

YC16XX SDK开发手册

V1.7

Yichip Microelectronics
©2022

Revision History

Version	Author	Description
1.0	sdk team	initial version
1.1	Hyh	同步LPM和RTC睡眠唤醒部分
1.2	Hyh	更新关于OTA升级的说明
1.3	sdk team	增加免晶体运行demo说明，优化部分说明
1.4	Hyh	更新BOOT方式OTA升级相关说明
1.5	sdk team	增加app_board_config.h文件使用说明
1.6	Scj	更新ble的说明
1.7	Kiven	更新睡眠相关demo说明

Confidentiality Level:

Confidential

目录

目录 3

1. board_config.h 文件说明 27

1.1. 宏定义说明 27

1.2. app_board_config.h 文件使用说明 28

2. SYSCTRL 31

2.1. 宏定义、枚举及结构体说明 31

2.1.1. 系统事件标志位宏定义 31

2.1.2. 系统时钟选择枚举 31

2.1.3. BBPLL 时钟选择枚举 31

2.1.4. 模块时钟枚举 32

2.2. SYSCTRL 接口汇总 32

2.3. 接口说明 33

2.3.1. void SYSCTRL_ChipInit(void) 33

2.3.2. void SYSCTRL_DpllInit(void) 33

2.3.3. void SYSCTRL_ClkInit(SYS_ClkEnum sysClkSource, uint8_t sysClkDiv,
DPLL_ClkEnum mcuDpllClkSel, DPLL_ClkEnum flashClkSel) 34

2.3.4. uint32_t SYSCTRL_GetMcuClk(void) 34

2.3.5. uint32_t SYSCTRL_GetSystemClk(void) 35

2.3.6. void SYSCTRL_ClkEn(CLK_EnEnum type,FunctionalState newState) 35

2.3.7. void SYSCTRL_CrystalCtrl(FunctionalState state) 35

2.3.8. void SYSCTRL_SoftReset(void) 36

2.3.9. void SYSCTRL_WaitLpoEdge(int edgeNum) 36
2.3.10. uint8_t SYSCTRL_GetSysEvent(void) 36
2.3.11. void SYSCTRL_ClearSysEvent(uint8_t event) 37
2.3.12. void SYSCTRL_IceCmd(FunctionalState newState) 37
2.4. 免晶体 demo 说明 38
3. UART 40
3.1. 宏定义、枚举及结构体说明 40
3.1.1. UART 停止位 bit 数 40
3.1.2. UART 奇偶校验位 40
3.1.3. UART 模式 40
3.1.4. UART 硬件流控控制 40
3.1.5. UART 中断状态 40
3.1.6. UART 初始化结构体 41
3.2. UART 操作接口汇总 41
3.3. 接口说明 42
3.3.1. void UART_Init(CORE_UARTA_Type* uartx, UART_InitStruct* initStruct) 42
3.3.2. void UART_DeInit(CORE_UARTA_Type* uartx) 43
3.3.3. void UART_SendData(CORE_UARTA_Type* uartx, uint8_t data) 43
3.3.4. uint8_t UART_ReceiveData(CORE_UARTA_Type* uartx) 44
3.3.5. uint32_t UART_ReceiveBuff(CORE_UARTA_Type* uartx, uint8_t* buff, uint32_t len) 44
3.3.6. uint32_t UART_SendBuff(CORE_UARTA_Type* uartx, uint8_t* buff, uint32_t len) 44

3.3.7. uint32_t UART_GetRxCount(CORE_UARTA_Type* uartx) 45
3.3.8. uint32_t UART_GetTxCount(CORE_UARTA_Type* uartx) 45
3.3.9. void UART_SetRxNumIrq(CORE_UARTA_Type* uartx, uint8_t num) 45
3.3.10. void UART_SetRxTimeout(CORE_UARTA_Type* uartx, uint16_t time) . 46
3.3.11. uint8_t UART_GetIrqStatus(CORE_UARTA_Type* uartx) 46
3.3.12. void UART_SetTxIrq(CORE_UARTA_Type* uartx, FunctionalState newState) 47
3.3.13. void UART_ClearTxIrq(CORE_UARTA_Type* uartx) 47
3.3.14. void UART_ClearRxNumIrq(CORE_UARTA_Type* uartx) 47
3.3.15. void UART_ClearRxTimeoutIrq(CORE_UARTA_Type* uartx) 47
3.3.16. void UART_SetAutoBaud(CORE_UARTA_Type* uartx, FunctionalState newState) 48
3.3.17. void UART_SetBaudRate(CORE_UARTA_Type* uartx, uint32_t baudRate) 48
3.4. demo 说明 48
3.4.1. uart polling demo 49
3.4.2. uart interrupt demo 49
4. 中断系统 52
4.1.1. PLIC 中断表 52
4.1.2. VECTOR 中断表 53
4.2. 宏定义、枚举及结构体说明 55
4.2.1. PLIC 中断号枚举 55
4.2.2. VECTOR 中断号枚举 56

4.3. 接口汇总 56
4.4. 接口说明 57
4.4.1. void PLIC_EnableIRQ(IRQn_Enum IRQn) 57
4.4.2. void PLIC_DisableIRQ(IRQn_Enum IRQn) 57
4.4.3. void PLIC_SetPriority(IRQn_Enum IRQn, uint32_t priority) 57
4.4.4. void inline VECTOR_EnableIRQ(VECTOR_IRQn_Enum IRQn) 58
4.4.5. void inline VECTOR_DisableIRQ(VECTOR_IRQn_Enum IRQn) 58
5. GPIO 59
5.1. 宏定义、枚举及结构体说明 59
5.1.1. GPIO 模式枚举 59
5.1.2. GPIO Pin 枚举 60
5.1.3. GPIO 端口枚举 61
5.1.4. GPIO 长按复位枚举 61
5.1.5. GPIO 长按复位时间枚举 62
5.1.6. GPIO 唤醒类型枚举 62
5.1.7. GPIO 中断模式枚举 62
5.2. GPIO 接口汇总 63
5.3. 接口说明 63
5.3.1. void GPIO_Config(GPIO_NumEnum gpiox, GPIO_ModeEnum func) 64
5.3.2. void GPIO_DeInit(void) 64
5.3.3. void GPIO_AnalogEn(GPIO_NumEnum gpiox, FunctionalState newState) 64
5.3.4. void GPIO_DsConfig(GPIO_NumEnum gpiox, FunctionalState newState) 65
5.3.5. uint8_t GPIO_ReadDataBit(GPIO_NumEnum gpiox) 65

5.3.6. uint8_t GPIO_ReadData(GPIO_PortEnum portx) 65
5.3.7. void GPIO_SetIrq(GPIO_NumEnum gpiox, FunctionalState newState) ...66
5.3.8. void GPIO_SetIrqMode(GPIO_NumEnum gpiox, GPIO_IrqEnum type) .. 66
5.3.9. uint8_t GPIO_GetIrqStatus(GPIO_NumEnum gpiox) 66
5.3.10. uint8_t GPIO_GetPortIrqStatus(GPIO_PortEnum portx) 67
5.3.11. void GPIO_ClearIrq(GPIO_NumEnum gpiox) 67
5.3.12. void GPIO_SetLongPressReset(GPIO_RstNumEnum GPIOx,
FunctionalState newState) 68
5.3.13. void GPIO_SetLongPressResetTimeout(GPIO_RstTimeEnum time) 68
5.3.14. uint32_t GPIO_Capture(GPIO_NumEnum gpiox,uint32_t minFrequency)68
5.3.15. void GPIO_WriteBit(GPIO_NumEnum gpiox, BitAction status) 69
5.3.16. void GPIO_SetWakeupType(GPIO_NumEnum gpiox,
GPIO_WakeUpEnum wakeType) 69
5.4. demo 说明 69
5.4.1. gpio capture demo 69
5.4.2. gpio interrupt demo 70
5.4.3. gpio_long_press_reset demo 72
6. 定时器(TIMER/PWM) 73
6.1. 宏定义、枚举及结构体说明 73
6.1.1. 获取定时器时钟宏定义 73
6.1.2. 计算定时器定时 1ms 宏定义 73
6.1.3. 计算定时器定时 1us 宏定义 73
6.1.4. 定时器端口枚举 73

6.1.5. 定时器初始化结构体 73
6.1.6. PWM 独立工作初始化结构体 74
6.1.7. PWM 差分输出初始化结构体 74
6.2. 定时器接口汇总 76
6.3. 定时器接口说明 76
6.3.1. void TIM_DeInit(TIM_PortEnum timx) 76
6.3.2. void TIM_Init(TIM_InitStruct *initStruct) 77
6.3.3. void TIM_Cmd(TIM_PortEnum timx, FunctionalState newState) 77
6.3.4. uint32_t TIM_Getclk(void) 77
6.3.5. void TIM_SetTimerPeriod(TIM_PortEnum timx, uint16_t period) 78
6.3.6. void TIM_ITConfig(TIM_PortEnum timx, FunctionalState newState) 78
6.3.7. void TIM_PwmInit(PWM_InitStruct *initStruct) 78
6.3.8. void TIM_SetPwmPeriod(TIM_PortEnum timx, uint16_t lowLevelPeriod, uint16_t highLevelPeriod) 79
6.3.9. void TIM_PwmSyncInit(PWM_SyncInitStruct* syncInitStruct) 79
6.3.10. void TIM_PwmSyncCmd(TIM_PortEnum timx, FunctionalState newState) 79
6.3.11. void TIM_SetPwmSyncPeriod(TIM_PortEnum timx, uint16_t leftBorderPeriod, uint16_t highLevelPeriod, uint16_t rightBorderPeriod, uint16_t lowLevelPeriod) 80
6.3.12. void TIM_ClearIrqStatus(TIM_PortEnum timx) 80
6.3.13. ITStatus TIM_GetITStatus(TIM_PortEnum timx) 80
6.3.14. uint8_t TIM_GetAllITStatus(void) 81

6.4. demo 说明 81
6.4.1. 定时器中断 demo 81
6.4.2. PWM demo 84
6.4.3. PWM 差分输出 demo 85
7. FLASH 88
7.1. 宏定义、枚举及结构体说明 88
7.1.1. FLASH 物理地址转 CPU(XIP 地址)宏定义 88
7.1.2. FLASH 读模式宏定义 88
7.2. FLASH 操作接口汇总 88
7.3. 接口说明 89
7.3.1. uint8_t FLASH_Write(uint32_t flashAddr, uint32_t tlen, uint8_t *tbuf) ... 89
7.3.2. uint8_t FLASH_Read(uint32_t flashAddr, uint32_t rlen, uint8_t *rbuf) 90
7.3.3. void FLASH_Lock(void) 90
7.3.4. uint8_t FLASH_Unlock(void) 90
7.3.5. uint8_t FLASH_PageErase(uint32_t flashAddr) 91
7.3.6. uint8_t FLASH_SectorErase(uint32_t flashAddr) 91
7.3.7. uint8_t FLASH_BlockErase_32k(uint32_t flashAddr) 92
7.3.8. uint8_t FLASH_BlockErase_64k(uint32_t flashAddr) 92
7.3.9. uint8_t FLASH_Powerdown(void) 92
7.3.10. uint8_t FLASH_SetReadMode(uint8_t readModeCmd) 93
7.3.11. uint32_t FLASH_GetJedecId(void) 93
7.3.12. void FLASH_SetParamAddr(uint32_t paramOffset) 93
7.4. demo 说明 93

8. HSPI 97
8.1. 宏定义、枚举及结构体说明 97
8.1.1. HSPI 极性宏定义 97
8.1.2. HSPI 相位宏定义 97
8.1.3. HSPI 状态类型枚举 97
8.1.4. HSPI 初始化结构体 97
8.2. HSPI 操作接口汇总 98
8.3. 接口说明 99
8.3.1. void HSPI_Init(HSPI_InitStruct* initStruct) 99
8.3.2. void HSPI_DeInit(void) 100
8.3.3. void HSPI_EnableCmd(FunctionalState newState) 100
8.3.4. void HSPI_DmaEnableCmd(FunctionalState newState) 100
8.3.5. uint32_t HSPI_GetClk(void) 101
8.3.6. void HSPI_FifoAutoStartCmd(FunctionalState newState) 101
8.3.7. void HSPI_SetIrq(HSPI_StatusEnum type, FunctionalState newState) 101
8.3.8. void HSPI_SlaveEnableCmd(FunctionalState newState) 102
8.3.9. void HSPI_ClearIrq(HSPI_StatusEnum type) 102
8.3.10. FlagStatus HSPI_GetSpiStatus(HSPI_StatusEnum type) 102
8.3.11. FlagStatus HSPI_GetIrqStatus(HSPI_StatusEnum type) 103
8.3.12. uint16_t HSPI_GetAllIrqStatus(void) 103
8.3.13. void HSPI_MoveRxFifoPtr(void) 104
8.3.14. void HSPI_MoveTxFifoPtr(void) 104
8.3.15. uint8_t HSPI_ReadRxFifoData(void) 104

8.3.16. void HSPI_WriteRxFifoData(uint8_t data) 104
8.3.17. uint8_t HSPI_ReadTxFifoData(void) 105
8.3.18. void HSPI_WriteTxFifoData(uint8_t data) 105
8.3.19. void HSPI_SendData(uint8_t data) 105
8.3.20. uint8_t HSPI_ReceiveData(void) 105
8.3.21. void HSPI_SendBuff(uint8_t *buff, uint32_t len) 105
8.3.22. void HSPI_ReceiveBuff(uint8_t *buff, uint32_t len) 106
8.3.23. void HSPI_SendAndReceiveData(uint8_t *txbuff, uint32_t txlen, uint8_t *rxbuff, uint32_t rxlen) 106
8.4. demo 说明 106
8.4.1. hspi flash demo 107
8.4.2. hspi master demo 109
8.4.3. hspi slave demo 110
9. IIC 112
9.1. 宏定义、枚举及结构体说明 112
9.1.1. IIC 初始化结构体 112
9.2. IIC 操作接口汇总 112
9.3. 接口说明 113
9.3.1. void IIC_Init(CORE_IIC_Type* iicx, IIC_InitStruct *iicInitStruct) 113
9.3.2. void IIC_DeInit(CORE_IIC_Type* iicx) 113
9.3.3. void IIC_SendData(CORE_IIC_Type* iicx, uint8_t *src, uint16_t len) 113
9.3.4. void IIC_ReceiveData(CORE_IIC_Type* iicx, uint8_t *src, uint16_t srcLen, uint8_t *dest, uint16_t destLen) 114

9.3.5. void IIC_SetIrq(CORE_IIC_Type* iicx, FunctionalState newState) 114
9.3.6. void IIC_ClearIrq(CORE_IIC_Type* iicx) 115
9.4. demo 说明 115
10. LPM 119
10.1. 宏定义、枚举及结构体说明 119
10.1.1. GPIO 睡眠保持电平枚举 119
10.1.2. 睡眠模式枚举 119
10.1.3. 唤醒模式枚举 119
10.2. LPM 操作接口汇总 119
10.3. 接口说明 120
10.3.1. void LPM_Init(void) 120
10.3.2. void LPM_SetGpioLatch(GPIO_NumEnum gpiox,LPM_LatchEnum type) 120
10.3.3. void LPM_Sleep(uint32_t wakeTimeMs, LPM_ModeEnum sleepLevel,uint8_t retention_cfg,LPM_WakeupModeEnum wakeMode,lpmPreActionCb lpmPreCallback,lpmPostActionCb lpmPostCallback) 120
10.4. demo 说明 121
11. CAN 126
11.1. 宏定义、枚举及结构体说明 126
11.1.1. CAN 帧类型宏定义 126
11.1.2. CAN 模式宏定义 126
11.1.3. CAN 中断类型 126
11.1.4. CAN 标志位宏定义 126

11.1.5. CAN 过滤模式.....127
11.1.6. CAN 过滤 bit 数.....127
11.1.7. CAN 初始化结构体.....127
11.1.8. CAN 过滤器初始化结构体.....128
11.1.9. Tx 信息结构体.....129
11.1.10. Rx 信息结构体.....129
11.2. CAN 操作接口汇总.....130
11.3. 接口说明.....130
11.3.1. uint8_t CAN_Init(CAN_Type *canx, CAN_InitStruct *initStruct).....131
11.3.2. void CAN_DeInit(CAN_Type *canx).....131
11.3.3. void CAN_FilterInit(CAN_Type *canx, CAN_FilterInitStruct *filterInitStruct).....131
11.3.4. uint8_t CAN_SendData(CAN_Type *canx, CAN_TxMsgStruct *txMessage).....132
11.3.5. FlagStatus CAN_GetFlagStatus(CAN_Type *canx, uint32_t canFlag)..132
11.3.6. uint8_t CAN_MessagePending(CAN_Type *canx).....132
11.3.7. uint8_t CAN_ReceiveData(CAN_Type *canx, CAN_RxMsgStruct *rxMessage).....133
11.3.8. void CAN_SetIrq(CAN_Type* canx, uint32_t canIrq, FunctionalState newState).....133
11.3.9. uint8_t CAN_GetIrqStatus(CAN_Type* canx, uint32_t canIrq).....134
11.3.10. void CAN_ClearIrq(CAN_Type* canx, uint32_t canIrq).....134
11.3.11. void CAN_Sleep(CAN_Type* canx).....134

11.3.12. void CAN_WakeUp(CAN_Type* canx) 135
11.4. demo 说明 135
11.4.1. loopback mode 135
11.4.2. normal mode 137
12. CHARGE 141
12.1. 宏定义、枚举及结构体说明 141
12.1.1. 充电电源枚举 141
12.1.2. 充电中断枚举 141
12.1.3. 充电中断模式枚举 141
12.1.4. 充电电流枚举 141
12.2. CHARGE 操作接口汇总 142
12.3. 接口说明 143
12.3.1. void CHARGE_VolSel(uint8_t CutOffVol, int8_t RechgrVol) 143
12.3.2. void CHARGE_CurSel(CHARGE_CurrentEnum ChargCurtSel, uint32_t TricCurtSel, uint32_t CutOffCurtSel) 143
12.3.3. bool CHARGE_CcCvCtrl(uint8_t ChargCcMod, uint8_t ChargCvMod) 144
12.3.4. void CHARGE_SystemPowerSelect(CHARGE_SysPowerEnum powerSource) 145
12.3.5. void CHARGE_Cmd(FunctionalState newState) 145
12.3.6. bool CHARGE_InsertDet(void) 145
12.3.7. bool CHARGE_State(void) 146
12.3.8. void CHARGE_SetIrq(CHARGE_InterEnum chgr_intrtype, FunctionalState newState) 146

12.3.9. void CHARGE_SetIrqMode(CHARGE_InterEnum chgr_intrtype, CHARGE_IrqModeEnum mode) 146
12.3.10. uint8_t CHARGE_GetIrqStatus(CHARGE_InterEnum chgr_intrtype) ..147
12.3.11. uint8_t CHARGE_GetIrqMode(void) 147
12.4. demo 说明 148
13. WDT 151
13.1. 宏定义、枚举及结构体说明 151
13.1.1. WDT 模式枚举 151
13.1.2. WDT 初始化结构体 151
13.1.3. WDT 超时时间宏定义 151
13.2. WDT 操作接口汇总 151
13.3. 接口说明 152
13.3.1. void WDT_Init(WDT_InitStruct* initStruct) 152
13.3.2. void WDT_SetReload(uint8_t count) 152
13.3.3. void WDT_Enable(void) 152
13.3.4. void WDT_Disable(void) 153
13.3.5. void WDT_ClearIrqSatus(void) 153
13.3.6. void WDT_WfiHoldEn(FunctionalState newState) 153
13.4. demo 说明 153
14. NEC 156
14.1. 宏定义、枚举及结构体说明 156
14.1.1. NEC 初始化结构体 156
14.1.2. NEC 中断类型 156

14.1.3. NEC 增益电平枚举 156
14.1.4. NEC 输入源枚举 156
14.1.5. NEC 模式枚举 156
14.2. NEC 操作接口汇总 157
14.3. 接口说明 158
14.3.1. void NEC_Init(NEC_InitStruct *initStruct) 158
14.3.2. void NEC_DeInit(void) 158
14.3.3. void NEC_SetClk(uint32_t clk) 158
14.3.4. void NEC_SetMode(NEC_ModeEnum mode) 158
14.3.5. void NEC_RxSrcSel(NEC_RxSrcEnum necSource) 159
14.3.6. void NEC_SetRxGainLevel(NEC_GainLevelEnum gainLevel) 159
14.3.7. void NEC_PtrInit(void) 159
14.3.8. void NEC_Cmd(FunctionalState state) 159
14.3.9. void NEC_SetIrq(uint8_t irqType,FunctionalState state) 160
14.3.10. void NEC_ClearIrq(uint8_t irqType) 160
14.3.11. uint8_t NEC_GetIrqStatus(void) 160
14.3.12. int NEC_WriteData(uint16_t *buf,int len) 161
14.3.13. uint16_t NEC_ReadData(void) 161
14.3.14. uint32_t NEC_GetDataCount(void) 161
14.4. demo 说明 162
14.4.1. nec send demo 162
14.4.2. nec recv demo 168
15. 实时时钟(RTC) 173

15.1. 宏定义、枚举及结构体说明.....173
15.1.1. RTC 类型枚举.....173
15.1.2. RTC 时钟源选择枚举.....173
15.1.3. RTC 时间结构体.....173
15.2. RTC 接口汇总.....174
15.3. RTC 接口说明.....175
15.3.1. void RTC_Cmd(FunctionalState state).....175
15.3.2. void RTC_DeInit(void).....175
15.3.3. void RTC_Calibration(void).....176
15.3.4. uint32_t RTC_GetClk(void).....176
15.3.5. void RTC_ClkSel(RTC_ClkSelEnum clkSel).....176
15.3.6. void RTC_SetIrq(RTC_CompEnum type, FunctionalState state).....176
15.3.7. void RTC_WKConfig(RTC_CompEnum type, FunctionalState state).....177
15.3.8. void RTC_SetAlarm0(uint32_t alarmValue).....177
15.3.9. void RTC_SetAlarm1(uint32_t alarmValue).....177
15.3.10. uint32_t RTC_GetCount(RTC_CompEnum type).....178
15.3.11. uint8_t RTC_GetIrqStauts(RTC_CompEnum type).....178
15.3.12. void RTC_ClearIrq(RTC_CompEnum type).....178
15.3.13. void RTC_SetTime(struct tm dateStruct).....179
15.3.14. struct tm* RTC_GetTime(void).....179
15.3.15. time_t RTC_GetTimeStamp(void).....179
15.4.demo 说明.....179
15.4.1. RTC 日历时间模式.....180

15.4.2. RTC LPM 模式 181
16. QSPI 185
16.1. 宏定义、枚举及结构体说明 185
16.1.1. QSPI 传输模式宏定义 185
16.2. QSPI 操作接口汇总 185
16.3. 接口说明 186
16.3.1. void QSPI_SetMode(CORE_QSPI1_Type* qspix, QSPI_ModeEnum cmdMode, QSPI_ModeEnum addrMode, QSPI_ModeEnum dataMode) 186
16.3.2. void QSPI_ClkSel(CORE_QSPI1_Type* qspix, DPLL_ClkEnum qspiDpllClkSel) 186
16.3.3. void QSPI_ClkCmd(CORE_QSPI1_Type* qspix, FunctionalState state) 187
16.3.4. void QSPI_DmaStart(CORE_QSPI1_Type* qspix, FunctionalState state) 187
16.3.5. void QSPI_SetDummy(CORE_QSPI1_Type* qspix, uint8_t dly) 188
16.3.6. uint8_t QSPI_WaitDone(CORE_QSPI1_Type* qspix) 188
16.3.7. void QSPI_9bitModeEn(CORE_QSPI1_Type* qspix, FunctionalState state) 188
16.3.8. uint8_t Qspi_SendAndReceiveData(CORE_QSPI1_Type* qspix, uint8_t *cmdbuf, uint8_t cmdlen, uint8_t *txbuf, uint32_t txlen, uint8_t *rxbuf, uint32_t rxlen) 189
16.3.9. void QSPI_SetIrq(CORE_QSPI1_Type* qspix, FunctionalState state) ... 190
16.3.10. void QSPI_ClearIrq(CORE_QSPI1_Type* qspix) 190
16.4. demo 说明 191
17. SADC 196

17.1. 宏定义、枚举及结构体说明 196
17.1.1. SADC 量程选择宏定义 196
17.1.2. SADC 通道枚举 196
17.1.3. SADC 状态标志位及中断标志位枚举 196
17.1.4. SADC 通道模式枚举 197
17.1.5. SADC 触发方式枚举 197
17.1.6. SADC 初始化结构体 198
17.1.7. SADC 看门狗初始化结构体 199
17.2. SADC 操作接口汇总 199
17.3. SADC 接口说明 200
17.3.1. void SADC_Init(SADC_InitTypeDef *SADC_InitStruct) 200
17.3.2. void SADC_DeInit(void) 201
17.3.3. void SADC_InputDiv2(FunctionalState state) 201
17.3.4. void SADC_SetRegularChannel(uint16_t rlist) 201
17.3.5. void SADC_SetInjectedChannel(uint16_t ilist, uint8_t num) 202
17.3.6. void SADC_RegularTrigsel(SADC_EtsTypeDef ets) 203
17.3.7. void SADC_InjectTrigsel(SADC_EtsTypeDef ets) 204
17.3.8. void SADC_RegularTrigger(void) 205
17.3.9. void SADC_InjectTrigger(void) 205
17.3.10. void SADC_SetIrq(SADC_IrqEnum irqn) 205
17.3.11. uint8_t SADC_GetIrqStatus(void) 206
17.3.12. uint8_t SADC_GetStatus(void) 206
17.3.13. void SADC_ClearIrq(SADC_IrqEnum irqn) 207

17.3.14. void SADC_SetRegualrSampleMode(FunctionalState state) 207
17.3.15. void SADC_SetInjectSampleMode(FunctionalState state) 208
17.3.16. void SADC_DmaCmd(FunctionalState newState) 208
17.3.17. void SADC_DmaConfig(uint16_t *rsaddr, uint16_t *readdr, uint16_t *ieaddr) 209
17.3.18. void SADC_WdtConfig(SADC_WdtStruct *wdt,uint16_t lowLimit,uint16_t highLimit) 209
17.3.19. uint16_t SADC_GetRegularConversionValue(void) 209
17.3.20. uint16_t SADC_GetInjectedConversionValue(void) 210
17.3.21. uint16_t SADC_CalculateVoltage(SADC_ChxEnum ch,uint16_t adcValue) 210
17.4. demo 说明 210
17.4.1. sadc demo 210
17.4.2. sadc dma 215
18. MIC 218
18.1. 宏定义、枚举及结构体说明 218
18.1.1. MIC 音频数据长度计算宏定义 218
18.1.2. MIC 配置宏定义 218
18.1.3. MIC 采样率枚举 219
18.1.4. MIC 一级时钟分频系数枚举 219
18.1.5. MIC 二级时钟分频系数枚举 219
18.1.6. MIC ADC 数据输出类型枚举 219
18.1.7. 数字 MIC 数据采集数据类型枚举 220

18.1.8. 选择 DMA 输出 MIC 数据类型枚举 220
18.1.9. MIC 初始化结构体 220
18.2. MIC 操作接口汇总 221
18.3. MIC 接口说明 221
18.3.1. void AUDIO_AdcInit(AUDIO_AdcInitStruct *adcInitStruct) 221
18.3.2. void AUDIO_AdcDmaBufInit(uint8_t *buf,uint16_t bufSize) 222
18.3.3. void AUDIO_AdcCmd(FunctionalState state) 222
18.3.4. void AUDIO_AdcSetVol(uint16_t vol) 222
18.3.5. void AUDIO_AdcSetMute(FunctionalState newState) 223
18.3.6. uint32_t AUDIO_AdcGetDataCount(void) 223
18.3.7. uint32_t AUDIO_AdcGetData(uint8_t *buf,uint32_t len) 223
18.4. demo 说明 223
19. DAC 226
19.1. 宏定义、枚举及结构体说明 226
19.1.1. 计算 DAC 输出数据量宏定义 226
19.1.2. DAC 单次、连续输出模式宏定义 226
19.1.3. DAC 采样率枚举 226
19.2. DAC 接口汇总 226
19.3. DAC 接口说明 227
19.3.1. void AUDIO_DacInit(uint32_t sampleRate) 227
19.3.2. void AUDIO_DacSetVol(uint8_t vol) 227
19.3.3. void AUDIO_DacDmaBufInit(uint8_t *buf, uint32_t len) 228
19.3.4. uint32_t AUDIO_DacPutData(uint8_t *buf, uint32_t len) 228

19.3.5. uint32_t AUDIO_DacPutZero(uint32_t len) 228
19.3.6. void AUDIO_DacSetMode(uint8_t mode) 229
19.3.7. void AUDIO_DacSetMute(FunctionalState mute) 229
19.3.8. void AUDIO_DacCmd(FunctionalState state) 229
19.3.9. uint32_t AUDIO_DacGetAvailableBufSize(void) 229
19.3.10. void AUDIO_SetDacIrq(FunctionalState state) 230
19.3.11. void AUDIO_SetDacIrqLevel(uint16_t level) 230
19.3.12. void AUDIO_ClearDacIrq(void) 230
19.4. demo 说明 230
20. USB 235
20.1. 宏定义、枚举及结构体说明 235
20.1.1. USB CDC 235
20.1.2. USB HID 239
20.1.3. USB printer 244
20.2. USB 接口汇总 248
20.3. USB 接口说明 250
20.3.1. USB CDC 250
20.3.2. USB HID 254
20.3.3. USB printer 257
20.4. demo 说明 261
20.4.1. USB CDC 261
20.4.2. USB HID 263
20.4.3. 键鼠 demo 264

20.4.4. USB printer 266
21. OTA 268
21.1. 宏定义、枚举及结构体说明 268
21.1.1. OTA 配置结构体 268
21.1.2. OTA 固件大小及 A,B 区大小宏定义(A B 区方式 OTA 升级) 268
21.1.3. OTA 固件大小及 APP 起始地址宏定义(BOOT 方式 OTA 升级) 269
21.1.4. OTA 升级模式枚举 269
21.2. 接口说明 269
21.2.1. void ota_config_init(void) 269
21.2.2. void RunNewApp(bool appflag) 269
21.2.3. void GoToApp(uint8_t set_app_flag,uint8_t entry_ota_flag) 269
21.2.4. void Show_Current_Block(void) 270
21.2.5. void OTA_Process(OTA_MODE_Def ota_mode) 270
21.3. demo 说明 270
21.3.1 A B 区方式 OTA 升级 270
21.3.2 BOOT 方式 OTA 升级 272
21.4. OTA 升级流程和注意事项 275
OTA 固件注意事项 275
OTA A B 区升级注意事项 275
OTA BOOT 升级注意事项 275
21.4.1. OTA 通讯协议 277
21.4.2. USB 方式 OTA 升级 278
21.4.3. BLE 方式 OTA 升级 279

22. BLE 283
22.1. 宏定义、枚举及结构体说明 283
22.1.1. BLE 广播数据最大长度 283
22.1.2. RSSI 窗口大小 283
22.1.3. BLE 地址类型 283
22.1.4. NVRAM 相关 284
22.1.5. NVRAM 数据结构体 284
22.1.6. BLE 广播类型枚举 285
22.1.7. BLE 地址类型枚举 285
22.1.8. BLE 配对模式枚举 285
22.2. ble 接口汇总 285
22.3. 接口说明 288
22.4. yc_bt 288
22.4.1. uint16_t BT_GetLibVersion(void) 288
22.4.2. void BT_BleSetMac(uint8_t * bleAddr) 288
22.4.3. void BT_BleSetName(uint8_t* bleName, uint8_t len) 288
22.4.4. void BT_BleSetAdvParams(BLE_AdvTypeEnum type,BLE_AdvAddrType own_addr_type, BLE_AdvAddrType peer_addr_type, uint8_t *advData, uint8_t len) 289
22.4.5. void BT_BleSetScanData(uint8_t* scanData, int len) 289
22.4.6. void BT_BleSetAdvInterval(uint32_t interval) 289
22.4.7. void BT_SetVisibility(uint8_t discoverable) 290

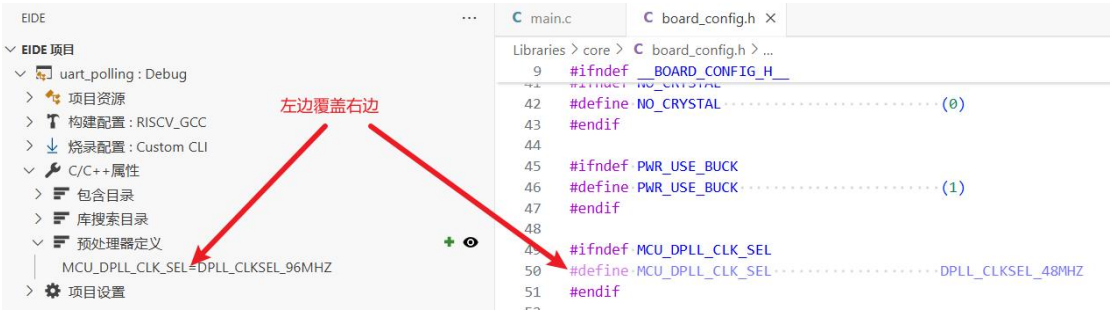
22.4.8. void BT_SetScanEnable(uint8_t discoverable,uint32_t scan_interval,uint16_t scan_window) 290
22.4.9. void BT_SetBleConnAddr(uint8_t *addr) 290
22.4.10. void BT_BleConnectParmUpdateReq(uint8_t *data,uint8_t len) 291
22.4.11. void BT_BleUpdateMtu(void) 291
22.4.12. void BT_BleSetMtu(uint16_t mtu) 291
22.4.13. void BT_BleSetPairMode(BLE_PairModeEnum mode) 292
22.4.14. _Bool BT_BleSlaveSendData(dev_handle_t dhandle,uint16_t handle, uint8_t * bleData, uint16_t len) 292
22.4.15. _Bool BT_BleMasterSendData(dev_handle_t dhandle,uint16_t handle, uint8_t * bleData, uint16_t len) 293
22.4.16. void BT_SetTXPower(char db) 293
22.4.17. void BT_SetLLParam(uint8_t *p) 293
22.4.18. void BT_NvramLoad(void) 293
22.4.19. uint8_t BT_NvramFindFromList(void) 294
22.4.20. void BT_BleNvramSave(void) 294
22.4.21. uint8_t BT_NvramUpdateToFlash(void) 294
22.4.22. void BT_BleDisconnectIND(uint8_t reason) 294
22.4.23. int8_t BT_GetRssi(void) 295
22.5. yc_ipc 295
22.5.1. void IPC_HandleRxPacket(void) 295
22.5.2. void IPC_ParseEvtCallback(uint32_t dataPtr) 295
22.6. yc_user 295

22.6.1. void U_BleSetMac(uint8_t * bleAddr) 295
22.6.2. void U_BleSetDeviceName(char *name_in,uint8_t *bleAddr) 296
22.6.3. void U_BleGenAdvScanData(uint8_t *adv_data,uint8_t *scan_rsp, uint8_t *name,uint8_t name_len,uint8_t *mac) 296
22.6.4. void U_BleParseScanAdvData(uint8_t *p) 296
22.6.5. int U_smp_passkey_wait_input(dev_handle_t dhandle) 297
22.6.6. void U_smp_passkey_wait_input_timeout(dev_handle_t dhandle) 297
22.6.7. int U_ATT_GenericAccess_CbRead(dev_handle_t dhandle, uint16_t att_handle, const struct ble_gatt_attr *attr, void *buf, uint16_t len, uint16_t offset) 297
22.6.8. int U_ATT_User_ServiceCbWrite(dev_handle_t dhandle, uint16_t att_handle, const struct ble_gatt_attr *attr, void *buf, uint16_t len, uint16_t offset) 298
22.6.9. void U_BleMasterParseNotifyData(dev_handle_t dhandle,uint8_t *rxp)298
22.7. demo 说明 299
22.7.1. ble_demo 299
22.7.2. ble_master_demo 302

1. board_config.h 文件说明

1.1. 宏定义说明

- 如果宏定义声明前有#ifndef 控制，个别应用 demo 需要更改此类宏定义的值时，不需要更改 board_config.h 文件，可以通过 VScode 的 EIDE 工程项目下 C/C++属性的预处理器定义中添加宏定义来覆盖 board_config.h 文件中的值，如下图



- BOARD_TYPE 宏定义有两个功能，一个是适配不同开发板或应用板的 IO 分配，另一个是适配不同型号芯片 flash 管脚，该宏定义的值用于适配 flash 引脚，从左到右每一个字节定义一个 flash 功能脚，字节的值为 IO 号；字节的位置对应不同 flash 功能脚，从左到右分别对应 CS、CLK、IO0(SI)、IO1(SO)、IO2(WP)、IO3(HOLD),后面还可以再跟 01 或 02 之类的序列号，用来区分同一个封装型号芯片不同开发板，例如：

  #define BOARD_2025_223_YC1161AA_EVB_V_1_0　(0x0c100f0d0b0e)

  表示 GPIO_12 为 flash 的 CS，GPIO_16 为 flash 的 CLK，GPIO_15 为 flash 的 IO0(SI)，GPIO_13 为 flash 的 IO1(SO)，GPIO_11 为 flash 的 IO2(WP)，GPIO_14 为 flash 的 IO3(HOLD)。

- 宏定义 PWR_USE_BUCK 定义是否使用芯片内部 BUCK 模块供电，需要与芯片封装和 PCB 适配，请联系原厂技术人员来确定该宏定义的值，不匹配会导致芯片工作异常。

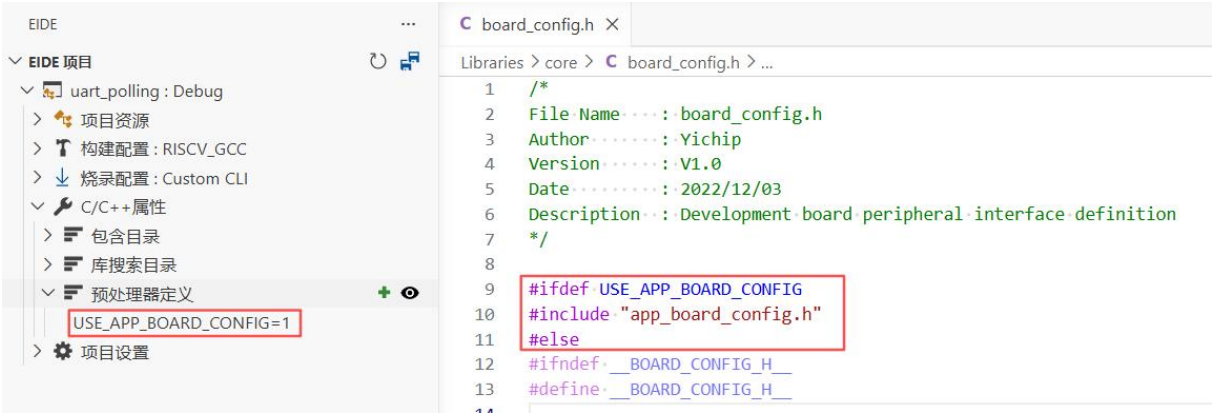
```
#ifndef PWR_USE_BUCK
#define PWR_USE_BUCK                    (0)
#endif
```

- 宏定义 NO_CRYSTAL 启用时将不会使用外部晶体而是使用内部 RC384M 时钟，但是该时钟精度不如外部晶体，不支持蓝牙链接
- 宏定义 SYS_CLK_DIV(系统时钟)需要与 MCU_CLK（CPU 主频）适配，系统时钟大小为 MCU_CLK/(SYS_CLK_DIV+1)，并且系统时钟最大支持 96MHz，推荐系统时钟为 48MHz

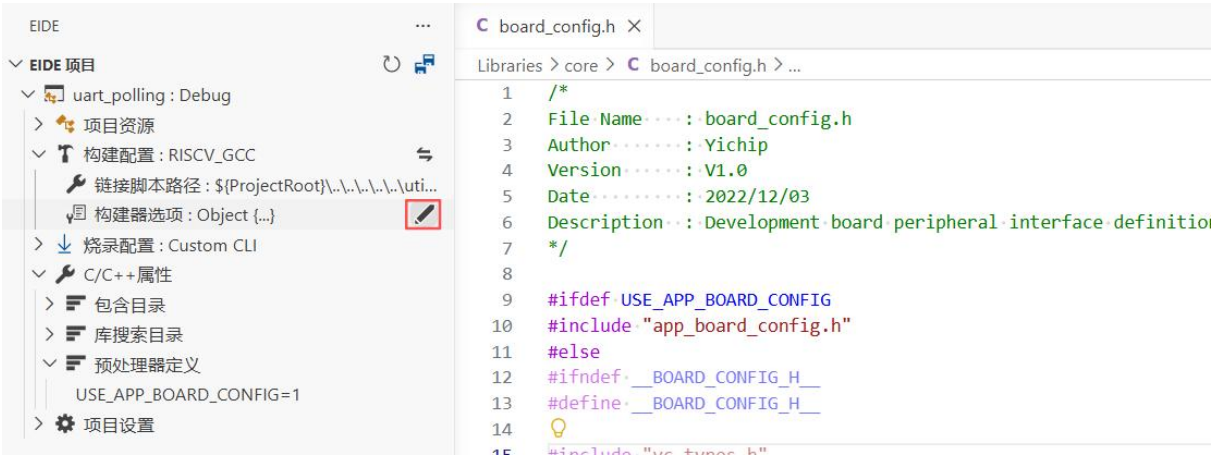
1.2. app_board_config.h 文件使用说明

board_config.h 文件适配了不同封装通用开发板的 IO 引脚分配，用户在开发应用时可参考 board_config.h 文件编写 app_board_config.h 文件，实现自定义的引脚映射

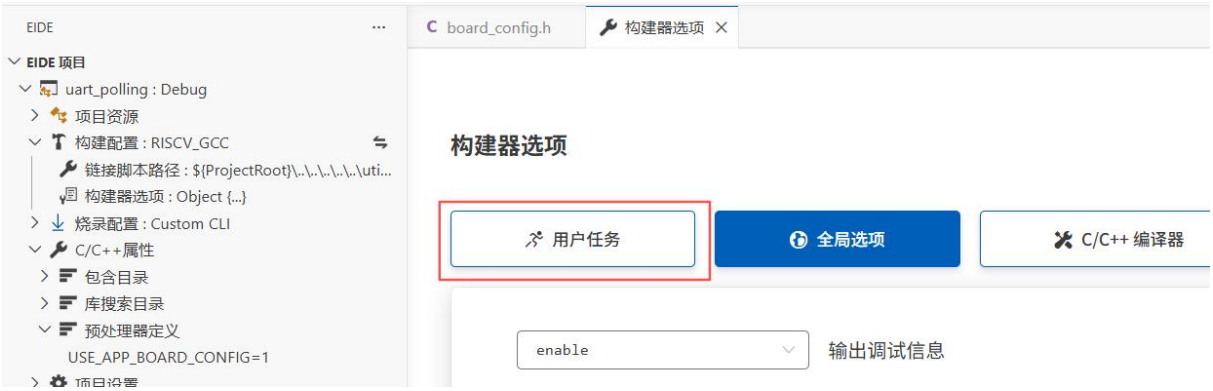
- 首先，在 VScode 的 EIDE 工程项目下 C/C++属性的预处理器定义中添加宏定义 USE_APP_BOARD_CONFIG=1 使 board_config.h 文件中的定义失效，并启用“app_board_config.h”的包含，如下图



- 然后，在构建配置中修改“gen dat”任务的传参，如下图，点击修改按钮



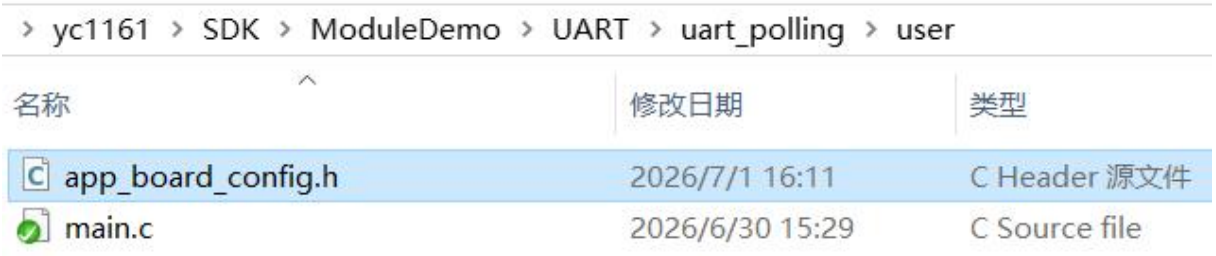
选择用户任务



下滑找到“gen dat”任务，修改命令中的第二项（app_board_config.h 文件路径）



"${ProjectRoot}/../../user/app_board_config.h"是 EIDE 中使用相对路径的写法。具体来说，"${ProjectRoot}"表示当前工程文件(*.code-workspace)所在的文件夹，"../"表示向上回退一级目录，此处共回退 2 级目录，然后进入 user 文件夹中的 app_board_config.h 文件。修改的内容取决于 app_board_config.h 文件的位置。例如，将 app_board_config.h 文件放入此 demo 的 user 文件夹中



应修改为："${ProjectRoot}/../../user/app_board_config.h"

- 注意：
  - 用户自定义的引脚映射文件必须命名为 app_board_config.h
  - 从 Libraries\core\board_config.h 文件复制过来的内容需删除 USE_APP_BOARD_CONFIG 这个条件编译语句，BOARD_TYPE 宏定义根据芯片封装修改

2. SYSCTRL

模块简介

系统控制主要作用是控制内外部时钟的选择、使能、关闭、分频，为 CPU 核心和各外设提供时钟。

2.1. 宏定义、枚举及结构体说明

2.1.1. 系统事件标志位宏定义

```
#define SYS_EVENT_SOFT_RESET              (1<<0)
#define SYS_EVENT_IO_RESET                (1<<1)
#define SYS_EVENT_WDT_RESET               (1<<2)
#define SYS_EVENT_CODE_CRC_ERR            (1<<4)   /* 固件 crc 校验错误 */
#define SYS_EVENT_CPU_INST_ERR            (1<<5)   /* CPU 指令异常 */
#define SYS_EVENT_CPU_DATA_ERR            (1<<6)   /* CPU 取数异常 */
#define SYS_EVENT_VIO_ERR                 (1<<7)   /* VIO 低压 */
#define SYS_EVENT_ALL                     (0xFF)
```

2.1.2. 系统时钟选择枚举

```
typedef enum
{
    SYS_CLKSEL_RC = 0,   /* 24M */
    SYS_CLKSEL_DPLL,     /* 24M,48M,64M,96M,128M,192M,256M */
    SYS_CLKSEL_CRYSTAL, /* normal 24M, or the select crystal */
    SYS_CLKSEL_LPO,      /* 32768hz */
} SYS_ClkEnum;
```

2.1.3. BBPLL 时钟选择枚举

```
typedef enum
{
    DPLL_CLKSEL_24MHZ = 0,
    DPLL_CLKSEL_48MHZ,
    DPLL_CLKSEL_64MHZ,
    DPLL_CLKSEL_96MHZ,
    DPLL_CLKSEL_128MHZ,
```

```
    DPLL_CLKSEL_192MHZ,
    DPLL_CLKSEL_256MHZ
} DPLL_ClkEnum;
```

2.1.4.模块时钟枚举

```
typedef enum
{
    CLK_CPU = 8,
    CLK_RAM,
    CLK_USB,
    CLK_UARTA,
    CLK_UARTB,
    CLK_AES,
    CLK_GDAC,
    CLK_BTDMA,
    CLK_SPI,
    CLK_IIC0,
    CLK_IIC1,
    CLK_MICDSM,
    CLK_MIC,
    CLK_ICE,
    CLK_NEC,
    CLK_UARTC,
    CLK_UARTD,
    CLK_LPM,
    CLK_QSPI,
    CLK_CAN
} CLK_EnEnum;
```

2.2. SYSCTRL 接口汇总

函数类型	函数名	说明
SYSCTRL 相关接口	SYSCTRL_ChipInit	芯片初始化
	SYSCTRL_DpllInit	芯片 DPLL 时钟初始化
	SYSCTRL_ClkInit	芯片时钟初始化
	SYSCTRL_ClkEn	芯片模块时钟使能

	SYSCTRL_GetMcuClk	获取 MCU 时钟
	SYSCTRL_GetSystemClk	获取系统时钟
	SYSCTRL_WaitLpoEdge	LPO 时钟同步
	SYSCTRL_CrystalCtrl	外部晶体振使能
	SYSCTRL_SoftReset	软件复位
	SYSCTRL_GetSysEvent	获取系统复位标志位
	SYSCTRL_ClearSysEvent	清除系统复位标志位
	SYSCTRL_IceCmd	芯片 ice 调试端口控制

2.3. 接口说明

2.3.1. void SYSCTRL_ChipInit(void)

- 描述：在该函数内将默认开启的看门狗功能关闭（应用如果需要启用看门狗，请参考 WDT 列程初始化），并对外部晶体、DPLL、系统时钟源、系统时钟、CPU 主频及 flash 时钟进行初始化
- 参数：none
- 返回值：none

2.3.2. void SYSCTRL_DpllInit(void)

- 描述：DPLL 模块初始化，由 SYSCTRL_ChipInit 函数内部调用
- 参数：none
- 返回值：none

2.3.3. void SYSCTRL_ClkInit(SYS_ClkEnum sysClkSource, uint8_t sysClkDiv, DPLL_ClkEnum mcuDpllClkSel, DPLL_ClkEnum flashClkSel)

- 描述：系统时钟初始化，由 SYSCTRL_ChipInit 函数内部调用
- 参数：
  - sysClkSource：设置系统时钟源，支持 4 个可选的系统时钟源，如下所示：
    - SYS_CLKSEL_RC：源自 24MHz RC 振荡器
    - SYS_CLKSEL_DPLL: 通过对晶体振荡或免晶体振荡(免晶体振荡时不支持蓝牙功能)产生, 可选择 24MHz, 48MHz, 64MHz, 96MHz, 128MHz, 192MHz, 256MHz
    - SYS_CLKSEL_CRYSTAL：源自 24M 晶体振荡器
    - SYS_CLKSEL_LPO：源自 RC32k 时钟源
  - sysClkDiv：设置系统时钟的分频，范围在 0-15 之间，SYS_CLK=MCU_CLK/(sysClkDiv+1)，系统时钟最高位 96M，提高 CPU 主频时，注意调整 sysClkDiv 使 SYS_CLK 保持在 96M 及以下
  - mcuDpllClkSel：MCU 时钟选择，sysClkSource 选择 SYS_CLKSEL_DPLL 时有效，可选值为 24MHz，48MHz，64MHz，96MHz，128MHz，192MHz，256MHz。
  - flashClkSel：flash 时钟选择，sysClkSource 选择 SYS_CLKSEL_DPLL 时有效，可选值为 24MHz，48MHz，64MHz，96MHz。
- 返回值：none

2.3.4. uint32_t SYSCTRL_GetMcuClk(void)

- 描述：获取 MCU 时钟
- 参数：none

- 返回值：MCU 时钟

2.3.5. uint32_t SYSCTRL_GetSystemClk(void)

- 描述：获取系统时钟
- 参数：none
- 返回值：系统时钟

2.3.6. void SYSCTRL_ClkEn(CLK_EnEnum type,FunctionalState newState)

- 描述：使能模块时钟
- 参数：
  - type：
    - 模块类型，详见 CLK_EnEnum 枚举
  - newState：
    - ENABLE：使能
    - DISABLE：失能
- 返回值：none

2.3.7. void SYSCTRL_CrystalCtrl(FunctionalState state)

- 描述：控制外部晶体振荡器使能
- 参数：
  - state：
    - ENABLE：使能
    - DISABLE：失能

- 返回值：none

2.3.8. void SYSCTRL_SoftReset(void)

- 描述：软件复位
- 参数：none
- 返回值：none

2.3.9. void SYSCTRL_WaitLpoEdge(int edgeNum)

- 描述：LPO 时钟与系统时钟同步函数，操作某些 LPM 寄存器时需要确保时钟沿同步，由驱动库内部调用
- 参数：
  - edgeNum：需要同步的 LPO 时钟沿个数
- 返回值：none

2.3.10. uint8_t SYSCTRL_GetSysEvent(void)

- 描述：获取系统复位事件标志位
- 参数：none
- 返回值：对应 SYS_EVENT_XX 宏定义组
  - bit0：软件复位
  - bit1：IO 复位
  - bit2：看门狗复位
  - bit3：固件 CRC 校验错误复位
  - bit4：CPU 指令异常复位
  - bit5：CPU 取数异常复位

- bit6：VIO 低压复位

2.3.11. void SYSCTRL_ClearSysEvent(uint8_t event)

- 描述：系统复位标志位清除
- 参数：
  - event：
    - bit0：软件复位
    - bit1：IO 复位
    - bit2：看门狗复位
    - bit3：固件 CRC 校验错误复位
    - bit4：CPU 指令异常复位
    - bit5：CPU 取数异常复位
    - bit6：VIO 低压复位
- 返回值：none

2.3.12. void SYSCTRL_IceCmd(FunctionalState newState)

- 描述：芯片 ice 调试端口控制
- 参数：
  - newState：芯片 ice 调试端口使能或失能
    - DISABLE：失能
    - ENABLE：使能
- 返回值：none

注意：

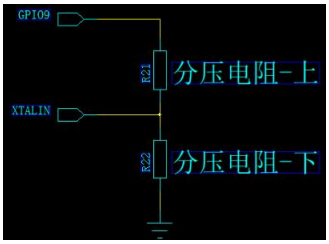
- GPIO[31]默认为芯片 ice 调试端口，在代码下载、程序调试中非常重要，不推荐作为普通 IO 口使用
- 若要将 GPIO[31]作为普通 IO 口使用，需在芯片初始化后延时 100ms，再调用此函数将芯片 ice 调试端口失能，避免关闭太早、以后无法下载代码

2.4. 免晶体 demo 说明

- 演示芯片免晶体运行，关键代码说明如下：
  - 免晶体控制宏定义，确保 NO_CRYSTAL 在预处理器定义中且为 1



  - GPIO9 需经分压电阻后与 XTALIN 引脚相连，此 GPIO 口可通过 board_config.h 文件中的宏 NO_CRYSTAL_GPIO 更改；分压电阻用于使 XTALIN 引脚电压接近 1.2V。下图为连接原理图：



  - 免晶体运行时，芯片无蓝牙功能

初始化系统和串口打印初始化，串口初始化必须在系统初始化之后，确保波特率配置正确

```
1    SYSCTRL_ChipInit();
2    printport_init();
```

Systick 定时器每 100ms 产生一次中断

```
    SYSTICK_Config(SYSTICK_ONE_MS*100);  //100ms
```

主循环持续将 LED 引脚翻转，用于验证芯片主频是否更改

```
    while(1)
    {
        GPIO_Config(GPIO_LED_PIN,GPIO_MODE_OUT_HIGH);
        GPIO_Config(GPIO_LED_PIN,GPIO_MODE_OUT_LOW);
    }
```

Systick 中断函数，每 100ms 打印一次，正常运行时串口应持续打印

```
_interrupt void VECTOR_SYSTICK_IRQ(void)
{
    yc_log("SYSTICK_IRQHandler");
    SYSTICK_ClearIrq(); //must call this function at last!
}
```

若出现串口打印乱码或不打印等情况：

- 检查 GPIO 是否配置正确、是否经分压电阻后与 XTALIN 引脚相连
- 与原厂 FAE 确认芯片是否经测试机台校准内部 RC 时钟

3. UART

3.1. 宏定义、枚举及结构体说明

3.1.1. UART 停止位 bit 数

```
#define UART_STOP_BIT_1                 (0)
#define UART_STOP_BIT_2                 (1)
#define UART_STOP_BIT_0                 (0xff)
```

3.1.2. UART 奇偶校验位

```
#define UART_PARITY_NONE                 (0)
#define UART_PARITY_EVEN                 (1)
#define UART_PARITY_ODD                  (2)
```

3.1.3. UART 模式

定义为全双工还是半双工

```
#define UART_MODE_SINGLE_LINE            (1)
#define UART_MODE_DUPLEX                 (0)
```

3.1.4. UART 硬件流控控制

```
#define UART_HARDWARE_FLOW_CONTROL_NONE          (0)
#define UART_HARDWARE_FLOW_CONTROL_ENABLE        (1)
```

3.1.5. UART 中断状态

```
#define UART_STATUS_TX_BUSY              (1<<0)
#define UART_STATUS_TX_FULL              (1<<1)
#define UART_STATUS_RX_EMPTY             (1<<2)
#define UART_STATUS_RX_FULL              (1<<3)
#define UART_STATUS_TX_INTR              (1<<4)
#define UART_STATUS_RX_INTR              (1<<5)
#define UART_STATUS_TIMEOUTINTR          (1<<6)
```

3.1.6. UART 初始化结构体

```
typedef struct
{
    uint32_t baudRate;     /*!< This member configures the UART communication baud rate(2400 - 3M bps)*/

    uint8_t stopBits;      /*!< UART_STOP_BIT_0 UART_STOP_BIT_1 or UART_STOP_BIT_2 */

    uint8_t parity;        /*!< UART_PARITY_NONE UART_PARITY_EVEN or UART_PARITY_ODD */

    uint8_t mode;           /*!< UART_MODE_SINGLE_LINE or UART_MODE_DUPLEX */

    uint8_t flowControl; /*!< UART_HARDWARE_FLOW_CONTROL_NONE or
                              UART_HARDWARE_FLOW_CONTROL_ENABLE */

    uint32_t txLen;         /*!< Specifies Tx DMA buff len */

    uint32_t rxLen;         /*!< Specifies Rx DMA buff len */

    uint32_t txBuffer;     /*!Tx ring buffer start addr */

    uint32_t rxBuffer;     /*!Rx ring buffer start addr */
} UART_InitStruct;
```

3.2. UART 操作接口汇总

函数类型	函数名	说明
UART 相关接口	UART_Init	UART 初始化
	UART_DeInit	清除 UART 相关配置
	UART_SendData	UART 发送 1 字节数据
	UART_ReceiveData	UART 接收 1 字节数据
	UART_ReceiveBuff	UART 获取 buffer 数据
	UART_SendBuff	UART 发送 buffer 数据

	UART_GetRxCount	UART 获取 RX DMA 已接收数据长度
	UART_GetTxCount	UART 获取 TX DMA 待发送数据长度
	UART_SetRxNumIrq	UART 设置 RX 中断长度阈值
	UART_SetRxTimeout	UART 设置 RX 字符间隔超时中断超时时间
	UART_GetIrqStatus	UART 获取中断状态
	UART_SetTxIrq	UART 设置 Tx 中断
	UART_ClearTxIrq	UART 清除 Tx 中断
	UART_ClearRxNumIrq	UART 清除 Rx 中断
	UART_ClearRxTimeoutIrq	UART 清除 Rx 超时中断
	UART_SetAutoBaud	UART 设置自动检测波特率
	UART_SetBaudRate	UART 设置波特率

3.3. 接口说明

3.3.1. void UART_Init(CORE_UARTA_Type* uartx, UART_InitStruct* initStruct)

- 描述：UART 初始化
- 参数：
  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D

- initStruct：初始值配置，结构体成员为
  - 波特率设置
  - 停止位
  - 奇偶校验位
  - 双工模式
  - 流控模式
  - TX 长度
  - RX 长度
  - TXbuf 地址
  - RXbuf 地址

- 返回值：none

3.3.2. void UART_DeInit(CORE_UARTA_Type* uartx)

- 描述：清除 UART 相关配置
- 参数：
  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D
- 返回值：none

3.3.3. void UART_SendData(CORE_UARTA_Type* uartx, uint8_t data)

- 描述：UART 发送 1 字节数据
- 参数：
  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D
  - data：待发送的 1 字节数据

- 返回值：none

3.3.4. uint8_t UART_ReceiveData(CORE_UARTA_Type* uartx)

- 描述：UART 接收 1 字节数据
- 参数：
  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D
- 返回值：
  - 接收的 1 字节数据

3.3.5. uint32_t UART_ReceiveBuff(CORE_UARTA_Type* uartx, uint8_t* buff, uint32_t len)

- 描述：获取 UART 接收多字节数据
- 参数：
  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D
  - buff：待接收的数据地址
  - len：接收长度
- 返回值：已接收数据字节数

3.3.6. uint32_t UART_SendBuff(CORE_UARTA_Type* uartx, uint8_t* buff, uint32_t len)

- 描述：UART 发送多字节数据
- 参数：
  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D
  - buff：待发送的数据地址

- len：发送数据长度

- 返回值：已发送数据字节数

3.3.7. uint32_t UART_GetRxCount(CORE_UARTA_Type* uartx)

- 描述：获取 UART 模块已接收待读取的字节数

- 参数：

  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D

- 返回值：已接收待读取的字节数

3.3.8. uint32_t UART_GetTxCount(CORE_UARTA_Type* uartx)

- 描述：获取 UART Tx DMA 中待发送字节个数

- 参数：

  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D

- 返回值：待发送字节个数

3.3.9. void UART_SetRxNumIrq(CORE_UARTA_Type* uartx, uint8_t num)

- 描述：设置 UART Rx 触发中断的长度阈值

- 参数：

  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D

  - num：Rx 中断阈值字节数(设置为 0 代表不启用 RX 字节中断)

- 返回值：none

3.3.10. void UART_SetRxTimeout(CORE_UARTA_Type* uartx, uint16_t time)

- 描述：设置 UART Rx 字符间隔超时中断时间
- 参数：
  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D
  - time：Rx 超时中断时间(设置为 0 代表不起用 Rx 字符间隔超时中断)，实际时间为 time/uart 模块时钟
- 返回值：none

3.3.11. uint8_t UART_GetIrqStatus(CORE_UARTA_Type* uartx)

- 描述：获取 UART 状态
- 参数：
  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D
- 返回值：
  - UART 状态，对应 UART_STATUS_XX 宏定义组,下列为各 bit 位代表状态：
    - bit 0：TX 忙
    - bit 1：TXbuffer 满
    - bit 2：RX 空
    - bit 3：RXbuffer 满
    - bit 4：TX 中断
    - bit 5：RX 中断
    - bit 6：超时中断

3.3.12. void UART_SetTxIrq(CORE_UARTA_Type* uartx, FunctionalState newState)

- 描述：设置 UART Tx 中断使能
- 参数：
  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D
  - newState：
    - ENABLE：使能
    - DISABLE：失能
- 返回值：none

3.3.13. void UART_ClearTxIrq(CORE_UARTA_Type* uartx)

- 描述：清除 Tx 中断状态
- 参数：
  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D
- 返回值：none

3.3.14. void UART_ClearRxNumIrq(CORE_UARTA_Type* uartx)

- 描述：清除 Rx 中断状态
- 参数：
  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D
- 返回值：none

3.3.15. void UART_ClearRxTimeoutIrq(CORE_UARTA_Type* uartx)

- 描述：清除 Rx 超时中断状态
- 参数：
  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D
- 返回值：none

3.3.16. void UART_SetAutoBaud(CORE_UARTA_Type* uartx, FunctionalState newState)

- 描述：设置 UART 自动适配波特率
- 参数：
  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D
  - newState：
    - ENABLE：使能
    - DISABLE：失能
- 返回值：none

3.3.17. void UART_SetBaudRate(CORE_UARTA_Type* uartx, uint32_t baudRate)

- 描述：UART 设置波特率大小
- 参数：
  - uartx：UART 类型，可选为 CORE_UARTA/B/C/D
  - baudRate：波特率大小
- 返回值：none

3.4. demo 说明

3.4.1. uart polling demo

- 演示以轮询方式读取 uart 接收到的数据并回发，关键代码说明如下：

初始化系统和串口打印初始化

```
    SYSCTRL_ChipInit();

    printport_init();
```

在whlie(1)中轮询uart，使用UART_GetRxCount判断收到串口数据，通过UART_ReceiveData 接收数据，再通过 UART_SendData 发送数据

```
    while(1)
    {
        if(UART_GetRxCount(PRINT_PORT)>0)
        {
            UART_SendData(PRINT_PORT,UART_ReceiveData(PRINT_PORT));
        }
    }
```

3.4.2. uart interrupt demo

- 演示通过 UART RX 中断接收数据以及 TX 中断的使用，关键代码说明如下：

初始化系统和串口打印初始化

```
    SYSCTRL_ChipInit();

    printport_init();
```

通过宏定义选择 plic 中断还是 vector 中断,通过 UART_SetRxIrqNum 设置接收多少个字符触发中断，并通过 UART_SetRxTimeout 设置 rx 字符间隔超时中断的超时时间

```
    UART_SetRxNumIrq(PRINT_PORT,RX_IRQ_NUM); //设置接收到多少个字符触发 RX 中断,希望接收到固定长度时触发中断场景使用
    UART_SetRxTimeout(PRINT_PORT,RX_TIMEOUT);//设置 RX 字符超时间隔中断的超时时间,包长不确定且希望接收完整包数据触发中断场景使用
    UART_SetTxIrq(PRINT_PORT,ENABLE);
    #if VECTOR_UARTA
    VECTOR_EnableIRQ(VECTOR_UARTA_IRQn);
```

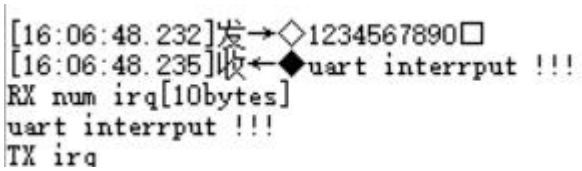
```
    #else
    PLIC_SetPriority(PRINT_IRQ_PORT,2);
    PLIC_EnableIRQ(PRINT_IRQ_PORT);
    #endif
```

uart 中断函数处理

```
#if VECTOR_UARTA
_interrupt void VECTOR_UARTA_IRQ(void)
#else
void UARTA_IRQHandler(void)
#endif
{
    uint8_t pRxBuf[256] = {0};
    uint16_t irq_status=UART_GetIrqStatus(PRINT_PORT);
    if(irq_status&UART_STATUS_RX_INTR)
    {
        yc_log("RX num irq[%dbytes]",RX_IRQ_NUM);
        UART_ReceiveBuff(PRINT_PORT,pRxBuf,RX_IRQ_NUM);
        UART_ClearRxNumIrq(PRINT_PORT);
    }
    if(irq_status&UART_STATUS_TIMEOUTINTR)
    {
        int rx_data_num=UART_GetRxCount(PRINT_PORT);
        yc_log("RX timeout irq[%dbytes]",rx_data_num);
        UART_ReceiveBuff(PRINT_PORT,pRxBuf,rx_data_num);
        UART_ClearRxTimeoutIrq(PRINT_PORT);
    }
    if(irq_status&UART_STATUS_TX_INTR)
    {
        yc_log("TX irq");
        while(UART_GetTxCount(PRINT_PORT)>0);//为避免打印"TX irq"日志导致再次触发 TX 中断,此处等待"TX irq"日志发送完成再清除 TX 中断
        UART_ClearTxIrq(PRINT_PORT);
    }
}
```

下图为中断触发示例，接收长度阈值为 10，接收到的数据字节长度未超过阈值触发超时中断，大于等于阈值则触发 Rx 中断，发送完成时产生 Tx 中断。

```
[16:07:14.085]发→◇123□
[16:07:14.090]收←◆uart interrput !!!
RX timeout irq[3bytes]
uart interrput !!!
TX irq
```


[16:06:48.232]发→◇1234567890□
[16:06:48.235]收←◆uart interrput !!!
RX num irq[10bytes]
uart interrput !!!
TX irq

4. 中断系统

4.1.1. PLIC 中断表

中断号	中断入口函数
0	RESERVE
1	BT_IRQHandler
2	SOFT_IRQHandler
3	BTTIMER_IRQHandler
4	UARTA_IRQHandler
5	USB_IRQHandler
6	AES_IRQHandler
7	IIC0_IRQHandler
8	IIC1_IRQHandler
9	HSPI_IRQHandler
10	MIC_IRQHandler
11	VIO_IRQHandler
12	GPIO_IRQHandler
13	SDMA_IRQHandler
14	HWAKE_IRQHandler
15	WDT_IRQHandler
16	TIMERGR0_IRQHandler

17	TIMERGR1_IRQHandler
18	CHGR_IRQHandler
19	GDAC_IRQHandler
20	SADC_IRQHandler
21	RESERVE
22	UARTC_IRQHandler
23	UARTD_IRQHandler
24	UARTB_IRQHandler
25	NEC_IRQHandler
26	QSPI_IRQHandler
27	QSPI1_IRQHandler
28	QSPI2_IRQHandler
29	RTC_IRQHandler
30	CAN_IRQHandler

4.1.2. VECTOR 中断表

中断号	中断入口函数
0	RESERVE
1	RESERVE
2	RESERVE

3	VECTOR_SOFT_IRQ
4	RESERVE
5	RESERVE
6	RESERVE
7	VECTOR_SYSTICK_IRQ
8	RESERVE
9	RESERVE
10	RESERVE
11	PLIC_IRQ(即 PLIC 中断)
12	RESERVE
13	RESERVE
14	RESERVE
15	RESERVE
16	VECTOR_RTC_IRQ
17	VECTOR_QSPI2_IRQ
18	VECTOR_QSPI1_IRQ
19	VECTOR_QSPI_IRQ
20	VECTOR_UARTA_IRQ
21	VECTOR_SADC_IRQ
22	VECTOR_GDAC_IRQ

23	VECTOR_TIMERGR0_IRQ
24	VECTOR_WDT_IRQ
25	VECTOR_SDMA_IRQ
26	VECTOR_GPIO_IRQ
27	VECTOR_MIC_IRQ
28	VECTOR_AES_IRQ
29	VECTOR_USB_IRQ
30	VECTOR_BTTIMER_IRQ
31	VECTOR_BT_IRQ

4.2. 宏定义、枚举及结构体说明

4.2.1. PLIC 中断号枚举

```
typedef enum
{
/* ----------------------   RV PLIC Interrupt ID   ---------------------- */
RSVD_IRQn                    =0,/* Reserved */
BT_IRQn                     =1,
SOFT_IRQn                    =2,
BTTIMER_IRQn                  =3,
UARTA_IRQn                    =4,
USB_IRQn                    =5,
AES_IRQn                    =6,
IIC0_IRQn                   =7,
IIC1_IRQn                   =8,
HSPI_IRQn                    =9,
MIC_IRQn                     =10,
VIO_IRQn                    =11,
GPIO_IRQn                    =12,
SDMA_IRQn                     =13,
HWAKE_IRQn                    =14,
WDT_IRQn                    =15,
```

```
TIMERGR0_IRQn                =16,/* TIM0-TIM3 */
TIMERGR1_IRQn                =17,/* TIM4-TIM7 */
CHGR_IRQn                    =18,
GDAC_IRQn                    =19,
SARADC_IRQn                  =20,
UARTC_IRQn                   =22,
UARTD_IRQn                   =23,
UARTB_IRQn                   =24,
NEC_IRQn                     =25,
QSPI_IRQn                    =26,
QSPI1_IRQn                   =27,
QSPI2_IRQn                   =28,
RTC_IRQn                     =29,
CAN_IRQn                     =30,
} IRQn_Enum;
```

4.2.2. VECTOR 中断号枚举

```
typedef enum
{
/* ----------------------   RV VECTOR Interrupt ID   ---------------------- */
VECTOR_SOFT_IRQn             =3,
VECTOR_SYSTICK_IRQn          =7,
VECTOR_RTC_IRQn              =16,
VECTOR_QSPI2_IRQn            =17,
VECTOR_QSPI1_IRQn            =18,
VECTOR_QSPI_IRQn             =19,
VECTOR_UARTA_IRQn            =20,
VECTOR_SADC_IRQn             =21,
VECTOR_GDAC_IRQn             =22,
VECTOR_TIMERGR0_IRQn         =23,/* TIM0-TIM3 */
VECTOR_WDT_IRQn              =24,
VECTOR_SDMA_IRQn             =25,
VECTOR_GPIO_IRQn             =26,
VECTOR_MIC_IRQn              =27,
VECTOR_AES_IRQn              =28,
VECTOR_USB_IRQn              =29,
VECTOR_BTTIMER_IRQn          =30,
VECTOR_BT_IRQn               =31,
} VECTOR_IRQn_Enum;
```

4.3. 接口汇总

函数类型	函数名	说明
PLIC 相关接口	PLIC_EnableIRQ	PLIC 中断使能
	PLIC_DisableIRQ	PLIC 中断失能
	PLIC_SetPriority	PLIC 中断优先级设置
VECTOR 相关接口	VECTOR_EnableIRQ	VECTOR 中断使能
	VECTOR_DisableIRQ	VECTOR 中断失能

4.4. 接口说明

4.4.1. void PLIC_EnableIRQ(IRQn_Enum IRQn)

- 描述：PLIC 中断使能
- 参数：
  - IRQn：PLIC 中断源的中断号，详见 PLIC 中断号枚举
- 返回值：none

4.4.2. void PLIC_DisableIRQ(IRQn_Enum IRQn)

- 描述：PLIC 中断失能
- 参数：
  - IRQn：PLIC 中断源的中断号，详见 PLIC 中断号枚举
- 返回值：none

4.4.3. void PLIC_SetPriority(IRQn_Enum IRQn, uint32_t priority)

- 描述：PLIC 中断优先级设置

- 参数：
  - IRQn：PLIC 中断源的中断号，详见 PLIC 中断号枚举
  - priority：中断优先级（可选值为 1、2、3，分别对应低、中、高中断优先级）
- 返回值：none

4.4.4. void inline VECTOR_EnableIRQ(VECTOR_IRQn_Enum IRQn)

- 描述：VECTOR 中断使能
- 参数：
  - IRQn：VECTOR 中断源的中断号，详见 VECTOR 中断号枚举
- 返回值：none

4.4.5. void inline VECTOR_DisableIRQ(VECTOR_IRQn_Enum IRQn)

- 描述：VECTOR 中断失能
- 参数：
  - IRQn：VECTOR 中断源的中断号，详见 VECTOR 中断号枚举
- 返回值：none

5. GPIO

5.1. 宏定义、枚举及结构体说明

5.1.1. GPIO 模式枚举

```
typedef enum
{
    GPIO_MODE_INPUT          = 0x00, /* 浮空输入 */
    GPIO_MODE_QSPI_NCS       = 0x02,
    GPIO_MODE_QSPI_SCK       = 0x03,
    GPIO_MODE_QSPI_IO0       = 0x04,
    GPIO_MODE_QSPI_IO1       = 0x05,
    GPIO_MODE_QSPI_IO2       = 0x06,
    GPIO_MODE_QSPI_IO3       = 0x07,
    GPIO_MODE_UARTA_TXD      = 0x08,
    GPIO_MODE_UARTA_RXD      = 0x09,
    GPIO_MODE_UARTA_RTS      = 0x0a,
    GPIO_MODE_UARTA_CTS      = 0x0b,
    GPIO_MODE_UARTB_TXD      = 0x0c,
    GPIO_MODE_UARTB_RXD      = 0x0d,
    GPIO_MODE_UARTB_RTS      = 0x0e,
    GPIO_MODE_UARTB_CTS      = 0x0f,
    GPIO_MODE_PWM_OUT        = 0x10, /* 从 GPIO0 开始，每 8 个 IO 为一组，分别对应 PWM0-PWM7 */
    GPIO_MODE_NEC_REC        = 0x11,
    GPIO_MODE_QSPI1_NCS      = 0x12,
    GPIO_MODE_QSPI1_SCK      = 0x13,
    GPIO_MODE_QSPI1_IO0      = 0x14,
    GPIO_MODE_QSPI1_IO1      = 0x15,
    GPIO_MODE_QSPI1_IO2      = 0x16,
    GPIO_MODE_QSPI1_IO3      = 0x17,
    GPIO_MODE_NEC_SEND       = 0x18,
    GPIO_MODE_DAC_OUT        = 0x1f, /* 从 GPIO0 开始，每 2 个 IO 为一组，分别对应 p 和 n */
    GPIO_MODE_PWM0_1_OUT     = 0x20, /* 从 GPIO0 开始，每 2 个 IO 为一组，分别对应 PWM0-PWM1 */
    GPIO_MODE_PWM0_3_OUT     = 0x21, /* 从 GPIO0 开始，每 4 个 IO 为一组，分别对应 PWM0-PWM3 */
    GPIO_MODE_SPID_MOSI      = 0x22,
    GPIO_MODE_SPID_MISO      = 0x23,
    GPIO_MODE_SPID_NCS       = 0x24,
    GPIO_MODE_SPID_SCK       = 0x25,
```

```
34      GPIO_MODE_UARTC             = 0x29, /* 从 GPIO0 开始，每 4 个 IO 为一组，分别对应
    UART_TX,UART_RX,UART_RTS,UART_CTS */
35      GPIO_MODE_UARTD             = 0x2a, /* 从 GPIO0 开始，每 4 个 IO 为一组，分别对应
    UART_TX,UART_RX,UART_RTS,UART_CTS */
36      GPIO_MODE_IIC0_SCL          = 0x2c,
37      GPIO_MODE_IIC0_SDA          = 0x2d,
38      GPIO_MODE_AUDIO_CLK         = 0x31,
39      GPIO_MODE_CAN_RX            = 0x32,
40      GPIO_MODE_CAN_TX            = 0x33,
41      GPIO_MODE_MICCLK_PDM        = 0x34,
42      GPIO_MODE_MICDAT_PDM        = 0x35,
43      GPIO_MODE_QSPI2_NCS         = 0x36,
44      GPIO_MODE_QSPI2_SCK         = 0x37,
45      GPIO_MODE_QSPI2_IO0         = 0x38,
46      GPIO_MODE_QSPI2_IO1         = 0x39,
47      GPIO_MODE_QSPI2_IO2         = 0x3a,
48      GPIO_MODE_QSPI2_IO3         = 0x3b,
49      GPIO_MODE_IIC1_SCL          = 0x3c,
50      GPIO_MODE_IIC1_SDA          = 0x3d,
51      GPIO_MODE_OUT_LOW           = 0x3e,
52      GPIO_MODE_OUT_HIGH          = 0x3f,
53      GPIO_MODE_PULL_UP           = 0x40,
54      GPIO_MODE_PULL_DOWN         = 0x80,
55      GPIO_MODE_HI_Z              = 0xc0    /* GPIO 高阻态 */
56  }GPIO_ModeEnum;
```

5.1.2. GPIO Pin 枚举

```
1   typedef enum
2   {
3       GPIO_0 = 0,
4       GPIO_1,
5       GPIO_2,
6       GPIO_3,
7       GPIO_4,
8       GPIO_5,
9       GPIO_6,
10      GPIO_7,
11      GPIO_8,
12      GPIO_9,
13      GPIO_10,
14      GPIO_11,
15      GPIO_12,
16      GPIO_13,
```

```
    GPIO_14,
    GPIO_15,
    GPIO_16,
    GPIO_17,
    GPIO_18,
    GPIO_19,
    GPIO_20,
    GPIO_21,
    GPIO_22,
    GPIO_23,
    GPIO_24,
    GPIO_25,
    GPIO_26,
    GPIO_27,
    GPIO_28,
    GPIO_29,
    GPIO_30,
    GPIO_31,
    GPIO_32,
    GPIO_33,
    GPIO_34,
    GPIO_35,
    GPIO_36,
    GPIO_37
}GPIO_NumEnum;
```

5.1.3. GPIO 端口枚举

```
typedef enum
{
    GPIOA = 0,
    GPIOB = 1,
    GPIOC = 2,
    GPIOD = 3,
    GPIOE = 4
} GPIO_PortEnum; /* 8 IO of each group */
```

5.1.4. GPIO 长按复位枚举

```
typedef enum
{
    GPIO_20_LONG_PRESS_RESET = 2,
    GPIO_26_LONG_PRESS_RESET,
```

```
    GPIO_31_LONG_PRESS_RESET
}GPIO_RstNumEnum;
```

5.1.5. GPIO 长按复位时间枚举

```
typedef enum
{
    LONG_PRESS_TIMEOUT_64S = 0,
    LONG_PRESS_TIMEOUT_56S,
    LONG_PRESS_TIMEOUT_48S,
    LONG_PRESS_TIMEOUT_40S,
    LONG_PRESS_TIMEOUT_32S,
    LONG_PRESS_TIMEOUT_24S,
    LONG_PRESS_TIMEOUT_16S,
    LONG_PRESS_TIMEOUT_8S
}GPIO_RstTimeEnum;
```

5.1.6. GPIO 唤醒类型枚举

```
typedef enum
{
    SLEEP_PULLUP = 1,
    SLEEP_LOWWAKE,
    SLEEP_PULLUP_LOWWAKE,
    SLEEP_HIGHWAKE,
    SLEEP_PULLDOWN_HIGHWAKE,
    SLEEP_PULLUP_QDECWAKE,
    SLEEP_TKEYBOARD_PRESSWAKE
} GPIO_WakeUpEnum;
```

5.1.7. GPIO 中断模式枚举

```
typedef enum
{
    HIGH_INTERRUPT = 0,
    LOW_INTERRUPT,
    RISING_INTERRUPT,
    FALL_INTERRUPT
}GPIO_IrqEnum;
```

5.2. GPIO 接口汇总

函数类型	函数名	说明
GPIO 相关接口	GPIO_Config	GPIO Pin 功能配置
	GPIO_DeInit	清除 GPIO 相关配置
	GPIO_AnalogEn	GPIO Pin 模拟功能使能
	GPIO_DsConfig	GPIO Pin 驱动能力配置
	GPIO_ReadDataBit	读取 GPIO Pin 高低电平
	GPIO_ReadData	读取 GPIO Port 高低电平
	GPIO_SetIrq	GPIO Pin 中断使能
	GPIO_SetIrqMode	配置 GPIO Pin 中断模式
	GPIO_GetIrqStatus	获取 GPIO Pin 中断状态
	GPIO_GetPortIrqStatus	获取 GPIO Port 中断状态
	GPIO_ClearIrq	清除 GPIO Pin 中断状态
	GPIO_SetLongPressReset	配置长按复位 GPIO Pin
	GPIO_SetLongPressResetTimeout	配置 GPIO 长按复位时间
	GPIO_Capture	检测 GPIO Pin 输入的方波频率
	GPIO_WriteBit	GPIO Pin 输出高低电平
	GPIO_SetWakeupType	GPIO Pin 设置唤醒类型

5.3. 接口说明

5.3.1. void GPIO_Config(GPIO_NumEnum gpiox, GPIO_ModeEnum func)

- 描述：GPIO Pin 功能配置
- 参数：
  - gpiox：GPIO Pin 号 ，详见 GPIO Pin 枚举
  - func：详见 GPIO 模式枚举，根据不同的硬件需求（GPIO_PULL_DOWN 和 GPIO_PULL_UP 可以和某些功能组合传参，如 GPIO_MODE_PULL_UP|GPIO_MODE_IIC0_SCL）
- 返回值：none

5.3.2. void GPIO_DeInit(void)

- 描述：清除 gpio 相关配置 ,将所有 GPIO 配置为高组态
- 参数：none
- 返回值：none

5.3.3. void GPIO_AnalogEn(GPIO_NumEnum gpiox, FunctionalState newState)

- 描述：GPIO Pin 模拟功能使能，使用此功能时建议先将 GPIO 的 func 配置为高阻模式，避免数字信号干扰模拟信号
- 参数：
  - gpiox：GPIO Pin 号 ，详见 GPIO Pin 枚举
  - newState：
    - ENABLE：使能
    - DISABLE：失能

- 返回值：none

5.3.4. void GPIO_DsConfig(GPIO_NumEnum gpiox, FunctionalState newState)

- 描述：GPIO Pin 驱动能力大小配置
- 参数：
  - gpiox：GPIO Pin 号，详见 GPIO Pin 枚举
  - newState：
    - ENABLE：将 GPIO 配置为强驱动
    - DISABLE：将 GPIO 配置为弱驱动
- 返回值：none

5.3.5. uint8_t GPIO_ReadDataBit(GPIO_NumEnum gpiox)

- 描述：获取 GPIO Pin 高低电平
- 参数：
  - gpiox：GPIO Pin 号，详见 GPIO Pin 枚举
- 返回值：
  - 输入电平状态：
    - 0：表示低电平
    - 1：表示高电平

5.3.6. uint8_t GPIO_ReadData(GPIO_PortEnum portx)

- 描述：获取 GPIO Port 高低电平
- 参数：

- portx：GPIO Port 号，详见 GPIO 端口枚举

- 返回值：
  - GPIO PORT 电平状态，一个 Port 代表 8 个 GPIO Pin，每个 bit 代表 1 个 GPIO Pin 状态

5.3.7. void GPIO_SetIrq(GPIO_NumEnum gpiox, FunctionalState newState)

- 描述：GPIO 中断使能
- 参数：
  - gpiox：GPIO Pin 号，详见 GPIO Pin 枚举
  - newState：
    - ENABLE：使能对应 GPIO Pin 中断
    - DISABLE：失能对应 GPIO Pin 中断
- 返回值：none

5.3.8. void GPIO_SetIrqMode(GPIO_NumEnum gpiox, GPIO_IrqEnum type)

- 描述：配置 GPIO 中断模式
- 参数：
  - gpiox：GPIO Pin 号，详见 GPIO Pin 枚举
  - type：GPIO 中断模式选择 详见 GPIO 中断模式枚举
- 返回值：none

5.3.9. uint8_t GPIO_GetIrqStatus(GPIO_NumEnum gpiox)

- 描述：获取 GPIO 中断状态
- 参数：
  - gpiox：GPIO Pin 号，详见 GPIO Pin 枚举
- 返回值：
  - GPIO 中断状态：
    - 1：中断标志位置起
    - 0：中断标志位未置起

5.3.10. uint8_t GPIO_GetPortIrqStatus(GPIO_PortEnum portx)

- 描述：获取 GPIO 端口中断状态
- 参数：
  - portx：GPIO Port 号，详见 GPIO 端口枚举
- 返回值：
  - GPIO Port 中断状态：一个 port 代表 8 个 GPIO Pin，每个 bit 代表 1 个 GPIO Pin
    - 1：中断标志位置起
    - 0：中断标志位未置起

5.3.11. void GPIO_ClearIrq(GPIO_NumEnum gpiox)

- 描述：GPIO 中断状态清除
- 参数：
  - gpiox：GPIO Pin 号，详见 GPIO Pin 枚举
- 返回值：none

5.3.12. void GPIO_SetLongPressReset(GPIO_RstNumEnum GPIOx, FunctionalState newState)

- 描述：长按复位 GPIO Pin 配置
- 参数：
  - GPIOx：只有特定的 GPIO Pin 才能设置长按重启，详见 GPIO 长按复位枚举
  - newState：
    - ENABLE：使能
    - DISABLE：失能
- 返回值：none

5.3.13. void GPIO_SetLongPressResetTimeout(GPIO_RstTimeEnum time)

- 描述：GPIO 长按复位时间配置
- 参数：
  - time：GPIO 长按时间配置，详见 GPIO 长按复位时间枚举
- 返回值：none

5.3.14. uint32_t GPIO_Capture(GPIO_NumEnum gpiox,uint32_t minFrequency)

- 描述：检测 GPIO 输入方波的频率
- 参数：
  - gpiox：GPIO Pin 号，详见 GPIO Pin 枚举
  - minFrequency：最小频率，单位赫兹（最小频率设置越高，检测相应速率越快）

- 返回值：
  - 输入方波的频率

5.3.15. void GPIO_WriteBit(GPIO_NumEnum gpiox, BitAction status)

- 描述：配置 GPIO Pin 输出高低电平
- 参数：
  - gpiox：GPIO Pin 号，详见 GPIO Pin 枚举
  - status：GPIO 输出电平，可选高电平或低电平
    - SET：高电平
    - RESET：低电平
- 返回值：none

5.3.16. void GPIO_SetWakeupType(GPIO_NumEnum gpiox, GPIO_WakeUpEnum wakeType)

- 描述：GPIO Pin 设置唤醒类型
- 参数：
  - gpiox：GPIO Pin 号，详见 GPIO Pin 枚举
  - wakeType：GPIO 唤醒类型，详见 GPIO 唤醒类型枚举
- 返回值：none

5.4. demo 说明

5.4.1. gpio capture demo

- 演示检测 GPIO 输入的方波频率功能，关键代码如下：

初始化系统和串口打印初始化

```
SYSCTRL_ChipInit();
printport_init();
```

配置 GPIO

```
GPIO_Config(GPIO_FREQUENCY_CAPTURE_PIN,GPIO_MODE_PULL_UP);//如果配置为高阻态则 GPIO 的数字功能无法使用
```

获取引脚频率，如果频率大于 0 则打印出来

```
while(1)
{
    frequency = GPIO_Capture(GPIO_FREQUENCY_CAPTURE_PIN, 1);
    if(frequency>0)
    {
      yc_log("test gpio %d ",GPIO_FREQUENCY_CAPTURE_PIN);
      yc_log("gpio_frequency = %d ",frequency);
      delay_ms(1000);
    }
}
```

5.4.2. gpio interrupt demo

- 演示 GPIO 中断触发功能，关键代码如下：

初始化系统和串口打印初始化

```
    SYSCTRL_ChipInit();
    printport_init();
```

通过 GPIO_Config 配置 pin 脚上下拉，通过 GPIO_SetIrqMode 设置 pin 脚中断类型，通过 GPIO_SetIrq 设置 pin 脚的 irq 使能

```
    GPIO_Config(GPIO_HIGH_IRQ_PIN,GPIO_MODE_PULL_DOWN);
    GPIO_SetIrqMode(GPIO_HIGH_IRQ_PIN,HIGH_INTERRUPT);
    GPIO_SetIrq(GPIO_HIGH_IRQ_PIN,ENABLE);

    GPIO_Config(GPIO_LOW_IRQ_PIN,GPIO_MODE_PULL_UP);
    GPIO_SetIrqMode(GPIO_LOW_IRQ_PIN,LOW_INTERRUPT);
    GPIO_SetIrq(GPIO_LOW_IRQ_PIN,ENABLE);
```

```
8
9       GPIO_Config(GPIO_RISING_IRQ_PIN,GPIO_MODE_PULL_DOWN);
10      GPIO_SetIrqMode(GPIO_RISING_IRQ_PIN,RISING_INTERRUPT);
11      GPIO_SetIrq(GPIO_RISING_IRQ_PIN,ENABLE);
12
13      GPIO_Config(GPIO_FALL_IRQ_PIN,GPIO_MODE_INPUT);
14      GPIO_SetIrqMode(GPIO_FALL_IRQ_PIN,FALL_INTERRUPT);
15      GPIO_SetIrq(GPIO_FALL_IRQ_PIN,ENABLE);
```

通过宏定义控制是 plic 中断还是 vector 中断

```
#ifdef VECTOR_INTERRUPT
    VECTOR_EnableIRQ(VECTOR_GPIO_IRQn);
#else
    PLIC_SetPriority(GPIO_IRQn,2);
    PLIC_EnableIRQ(GPIO_IRQn);
```

中断处理函数，中断发送通过获取 portGPIO_GetPortIrqStatus 每个 GPIO Pin 的中断状态，再通过宏 BIT_GET 获取是具体哪个 pin 脚发生中断，通过串口打印，并通过 GPIO_ClearIrq 清除中断

```
#if VECTOR_INTERRUPT
_interrupt void VECTOR_GPIO_IRQ(void)
#else
void GPIO_IRQHandler(void)
#endif
{
    uint8_t irq_status = 0;
    for(int i = 0;i <= GPIOE;i++) // find which group
    {
        irq_status=GPIO_GetPortIrqStatus(i);
        if(irq_status != 0)
        {
            for(int j=0;j<8;j++) // find which io
            {
                if(BIT_GET(irq_status,j))
                {
                    yc_log("gpio %d interrupt",j + (8*i));
                    GPIO_ClearIrq(j + (8*i));
                }
            }
        }
    }
```

```
}
```

5.4.3. gpio_long_press_reset demo

- 该 demo 演示 GPIO 长按复位功能，关键代码如下：

初始化系统和串口打印初始化

```
    SYSCTRL_ChipInit();

    printport_init();
```

通过宏定义来控制启用哪个 Pin 的长按复位功能，GPIO_Config 设置 GPIO Pin 上拉，GPIO_SetLongPressReset 接口设置长按重启 pin 脚，GPIO_SetLongPressResetTimeout 接口设置长按多久重启

```
#ifdef GPIO_Long_Press_Reset_1
    GPIO_Config(GPIO_Long_Press_Reset_1,GPIO_MODE_PULL_UP);
    GPIO_SetLongPressReset(GPIO_20_LONG_PRESS_RESET,ENABLE);
    GPIO_SetLongPressResetTimeout(LONG_PRESS_TIMEOUT);
    yc_log("GPIO_%d init end\r\n",GPIO_Long_Press_Reset_1);
#endif

#ifdef GPIO_Long_Press_Reset_2
    GPIO_Config(GPIO_Long_Press_Reset_2,GPIO_MODE_PULL_UP);
    GPIO_SetLongPressReset(GPIO_26_LONG_PRESS_RESET,ENABLE);
    GPIO_SetLongPressResetTimeout(LONG_PRESS_TIMEOUT);
    yc_log("GPIO_%d init end\r\n",GPIO_Long_Press_Reset_2);
#endif

#ifdef GPIO_Long_Press_Reset_3
    GPIO_Config(GPIO_Long_Press_Reset_3,GPIO_MODE_PULL_UP);
    GPIO_SetLongPressReset(GPIO_31_LONG_PRESS_RESET,ENABLE);
    GPIO_SetLongPressResetTimeout(LONG_PRESS_TIMEOUT);
    yc_log("GPIO_%d init end\r\n",GPIO_Long_Press_Reset_3);
#endif
```

6. 定时器(TIMER/PWM)

6.1. 宏定义、枚举及结构体说明

6.1.1.获取定时器时钟宏定义

```
#define TIMER_CLK                    (TIM_Getclk())
```

6.1.2.计算定时器定时 1ms 宏定义

```
#define TIMER_1MS(CLK_DIV)          (TIMER_CLK / (1 << CLK_DIV) / 1000)
```

CLK_DIV 为定时器时钟分频系数

6.1.3.计算定时器定时 1us 宏定义

```
#define TIMER_1US(CLK_DIV)          (TIMER_CLK / (1 << CLK_DIV) / 1000000)
```

CLK_DIV 为定时器时钟分频系数

6.1.4.定时器端口枚举

```
typedef enum
{
    TIM0 = 0,
    TIM1,
    TIM2,
    TIM3,
    TIM4,
    TIM5,
    TIM6,
    TIM7
} TIM_PortEnum;
```

6.1.5.定时器初始化结构体

```
typedef struct
{
```

```
    TIM_PortEnum timx;
    uint16_t        period;
    uint8_t         clockdiv;
    uint8_t         autoLoadEn;
} TIM_InitStruct;
```

- timx：选择使用的定时器,详见定时器端口枚举
- period：定时器周期，范围为 0-65535
- clockdiv：定时器时钟分频系数，范围为 0-7
- autoLoadEn：定时器自动重载功能使能
  - 0：失能
  - 1：使能

6.1.6. PWM 独立工作初始化结构体

```
typedef struct
{
    TIM_PortEnum     timx;
    uint16_t         lowlevelperiod;
    uint16_t         highlevelperiod;
    uint8_t          clockdiv;
} PWM_InitStruct;
```

- timx：选择使用的定时器
- lowlevelperiod：低电平周期
- highlevelperiod：高电平周期
- clockdiv：定时器分频系数，范围 0-7

6.1.7. PWM 差分输出初始化结构体

```
typedef struct
{
TIM_PortEnum timx;              /* PWM 组的第一路 TIMx(TIM0、TIM2、TIM4、TIM6) */
uint16_t lowLevelPeriod;
```

```
uint16_t highLevelPeriod;
uint16_t leftBorderPeriod;
uint16_t rightBorderPeriod;
uint8_t startLevel;            /* 0：偶数 PWM 通道先输出高电平；1：奇数 PWM 通道先输出高电平 */
uint8_t clockDiv;
uint8_t syncOutputInverse;
} PWM_SyncInitStruct;
```

- timx：定时器端口号
- lowLevelPeriod：第二路 PWM(端口序号为偶数)输出高电平的时间周期，范围 0~65535
- highLevelPeriod：第一路 PWM(端口序号为偶数)输出高电平的时间周期，范围 0~65535
- leftBorderPeriod：左死区的时间周期，范围 0~65535
- rightBorderPeriod：右死区的时间周期，范围 0~65535
- startLevel：
  - 0：奇数端口号 PWM 先输出高电平
  - 1：偶数端口号 PWM 先输出高电平
- clockDiv：定时器时钟分频系数，范围 0-7
- syncOutputInverse：决定哪一路 PWM 先输出高电平
  - 0：第一路 PWM(端口序号为偶数)先输出高电平
  - 1：第二路 PWM(端口序号为奇数)先输出高电平
- lowLevelPeriod、highLevelPeriod、leftBorderPeriod、rightBorderPeriod 的时间周期计算方法：

$$period = PWM_CLK / ((2^{CLK_DIV}) * HZ)$$

其中 HZ 为周期频率，单位为 Hz；period 为最终写入 lowLevelPeriod、highLevelPeriod、leftBorderPeriod、rightBorderPeriod 的值；PWM_CLK 为 TIMER/PWM 的时钟；CLK_DIV 为 TIMER/PWM 的分频系数，可以调用 demo 中计算 PWM 频率的宏定义

6.2. 定时器接口汇总

函数类型	函数名	说明
定时器相关接口	TIM_Init	定时器初始化
	TIM_DeInit	清除定时器相关配置
	TIM_Getclk	获取定时器模块时钟
	TIM_Cmd	定时器使能
	TIM_SetTimerPeriod	配置定时器周期
	TIM_ITConfig	定时器中断配置
	TIM_PwmInit	PWM 初始化
	TIM_SetPwmPeriod	设置 PWM 占空比
	TIM_PwmSyncInit	PWM 差分输出初始化
	TIM_PwmSyncCmd	PWM 差分输出使能
	TIM_SetPwmSyncPeriod	PWM 差分输出周期配置
	TIM_ClearIrqStatus	定时器中断标志位清除
	TIM_GetITStatus	获取指定定时器的中断标志位
	TIM_GetAllITStatus	获取所有定时器的中断标志位

6.3. 定时器接口说明

6.3.1.void TIM_DeInit(TIM_PortEnum timx)

- 描述：将对应定时器端口号的 pwmx_ctrl(x=0~7)寄存器的配置清除
- 参数：

- timx：定时器端口号，详见定时器端口枚举

- 返回值：none

6.3.2. void TIM_Init(TIM_InitStruct *initStruct)

- 描述：定时器初始化
- 参数：
  - initStruct：详见定时器初始化结构体
- 返回值：none

6.3.3. void TIM_Cmd(TIM_PortEnum timx, FunctionalState newState)

- 描述：定时器时钟使能
- 参数：
  - timx：定时器端口号，详见定时器端口枚举
  - newState：定时器时钟使能或失能
    - 0：失能
    - 1：使能
- 返回值：none

6.3.4. uint32_t TIM_Getclk(void)

- 描述：获取 TIMER/PWM 时钟
- 参数：none
- 返回值：
  - TIMER/PWM 模块时钟

6.3.5. void TIM_SetTimerPeriod(TIM_PortEnum timx, uint16_t period)

- 描述：设置定时器周期
- 参数：
  - timx：定时器端口号，详见定时器端口枚举
  - period：定时器周期，范围 0-65535
- 返回值：none

6.3.6. void TIM_ITConfig(TIM_PortEnum timx, FunctionalState newState)

- 描述：定时器中断使能
- 参数：
  - timx：定时器端口号，详见定时器端口枚举
  - newState：定时器中断使能或失能
    - 0：失能
    - 1：使能
- 返回值：none

6.3.7. void TIM_PwmInit(PWM_InitStruct *initStruct)

- 描述：定时器中断使能
- 参数：
  - initStruct：PWM 初始化结构体，详见 PWM 独立工作初始化结构体
- 返回值：none

6.3.8. void TIM_SetPwmPeriod(TIM_PortEnum timx, uint16_t lowLevelPeriod, uint16_t highLevelPeriod)

- 描述：PWM 设置高低电平时间周期
- 参数：
  - timx：PWM 使用的定时器端口号，详见定时器端口枚举
  - lowLevelPeriod：PWM 低电平时间周期
  - highLevelPeriod：PWM 高电平时间周期
- 返回值：none

6.3.9. void TIM_PwmSyncInit(PWM_SyncInitStruct* syncInitStruct)

- 描述：PWM 差分输出初始化
- 参数：
  - syncInitStruct：详见 PWM 差分输出初始化结构体
- 返回值：none

6.3.10. void TIM_PwmSyncCmd(TIM_PortEnum timx, FunctionalState newState)

- 描述：PWM 差分输出使能
- 参数：
  - timx：PWM 端口号
  - newState：PWM 差分输出使能或失能
    - 0：失能
    - 1：使能

- 返回值：none

6.3.11. void TIM_SetPwmSyncPeriod(TIM_PortEnum timx, uint16_t leftBorderPeriod, uint16_t highLevelPeriod, uint16_t rightBorderPeriod, uint16_t lowLevelPeriod)

- 描述：PWM 差分输出周期设置
- 参数：时间周期具体计算方法详见 PWM 差分输出初始化结构体
  - timx：定时器端口号(只能是偶数序号的定时器端口号)，详见定时器端口枚举
  - leftBorderPeriod：左死区时间周期
  - highLevelPeriod：第一路 PWM(端口序号为偶数)输出高电平的时间周期
  - rightBorderPeriod：右死区时间周期
  - lowLevelPeriod：第二路 PWM(端口序号为偶数)输出高电平的时间周期
- 返回值：none

6.3.12. void TIM_ClearIrqStatus(TIM_PortEnum timx)

- 描述：清除对应端口号的定时器中断标志位，必须在 TIMER_IRQHandler 中最后调用此函数。
- 参数：
  - timx：定时器端口号，详见定时器端口枚举
- 返回值：none

6.3.13. ITStatus TIM_GetITStatus(TIM_PortEnum timx)

- 描述：获取定时器对应端口号是否触发中断
- 参数：

- timx：定时器端口号，详见定时器端口枚举

- 返回值：
  - 对应定时器中断状态
    - 1：中断触发
    - 0：中断未触发

6.3.14. uint8_t TIM_GetAllITStatus(void)

- 描述：获取所有定时器中断状态
- 参数：none
- 返回值：
  - 所有定时器中断状态：每个 bit 代表一个定时器，如果 bit 为 1 则对应定时器触发了中断

6.4. demo 说明

6.4.1. 定时器中断 demo

- 演示定时器计数到达预设值时在中断中 GPIO Pin 输出脉冲，关键代码说明如下：

先初始化时钟和串口打印的 UART 端口

```
SYSCTRL_ChipInit();
printport_init();
```

有两种中断，分别为 PLIC 和 VECTOR 中断，利用宏定义选择使用哪种中断、再使能中断设置中断优先级

```
    #if VECTOR_IRQ
    VECTOR_EnableIRQ(VECTOR_TIMERGR0_IRQn);
    #else
    PLIC_SetPriority(TIMERGR0_IRQn, 1);
    PLIC_EnableIRQ(TIMERGR0_IRQn);
```

```
    #endif
    PLIC_SetPriority(TIMERGR1_IRQn, 1);
    PLIC_EnableIRQ(TIMERGR1_IRQn);
```

VECTOR 中断只对 TIM0-TIM3 生效，而 PLIC 中断对 TIM0-TIM7 都生效，默认开启 TIM4-TIM7 的 PLIC 中断

接下来对定时器进行设置和初始化配置，利用以下 API 设置定时器参数

```
static void TIM_CoreParaConfig(TIM_PortEnum timx, uint8_t clkDiv, uint16_t period,uint8_t autoloaden);
```

- timx：定时器端口号
- clkDiv：定时器时钟分频，范围 0~7
- period：定时器时间周期
- autoloaden：定时器自动重载功能是否使能，使能后定时器计数到达指定时间周期后会自动重载，关闭后定时器计算到达指定时间周期后会停止继续计数。

对 TIM0 和 TIM4 进行初始化

```
static void TIM_Configuration(void)
{

    TIM_CoreParaConfig((TIM_PortEnum)TIM0, TIMER_CLKDIV, TIMER_1MS(TIMER_CLKDIV) * 20);
    TIM_CoreParaConfig((TIM_PortEnum)TIM4, TIMER_CLKDIV, TIMER_1MS(TIMER_CLKDIV) * 50);
}
```

如果启用 VECTOR 中断，则 TIM0-TIM3 的定时器中断处理如下

```
_interrupt void VECTOR_TIMERGR0_IRQ(void)
{
    uint8_t timerIrqStatus = TIM_GetAllITStatus();
    for (int i = 0; i <= TIM3; i++)
    {
        if (timerIrqStatus & (1 << i))
        {
            yc_log("TIM%d INTERRUPT !!!", i); /* TIM_I service function */
            GPIO_Config(GPIO_PWM2_PIN, GPIO_MODE_OUT_HIGH);
            GPIO_Config(GPIO_PWM2_PIN, GPIO_MODE_OUT_LOW);
```

```
            TIM_ClearIrqStatus(i);
        }
    }
}
```

判断 TIM0~TIM3 中触发中断的定时器然后打印信息，GPIO_PWM2_PIN 输出一个脉冲后清除对应定时器的中断标志位

如果启用 PLIC 中断，则 TIM0~TIM3 的定时器中断处理如下

```
void TIMERGR0_IRQHandler(void)
{
    uint8_t timerIrqStatus = TIM_GetAllITStatus();
    for (int i = 0; i <= TIM3; i++)
    {
        if (timerIrqStatus & (1 << i))
        {
            yc_log("TIM%d INTERRUPT !!!", i); /* TIM_I service function */
            GPIO_Config(GPIO_PWM1_PIN,GPIO_MODE_OUT_HIGH);
            GPIO_Config(GPIO_PWM1_PIN,GPIO_MODE_OUT_LOW);
            TIM_ClearIrqStatus(i);
        }
    }
}
```

判断 TIM0~TIM3 中触发中断的定时器然后打印信息，GPIO_PWM1_PIN 输出一个脉冲后清除对应定时器的中断标志位

定时器 TIM4~TIM7 的 PLIC 中断处理如下

```
void TIMERGR1_IRQHandler(void)
{
    uint8_t timerIrqStatus = TIM_GetAllITStatus();
    for (int i = TIM4; i <= TIM7; i++)
    {
        if (timerIrqStatus & (1 << i))
        {
            yc_log("TIM%d INTERRUPT !!!", i); /* TIM_I service function */
            GPIO_Config(GPIO_PWM0_PIN,GPIO_MODE_OUT_HIGH);
            GPIO_Config(GPIO_PWM0_PIN,GPIO_MODE_OUT_LOW);
            TIM_ClearIrqStatus(i);
        }
    }
```

```
14 }
```

判断 TIM4~TIM7 中触发中断的定时器然后打印信息，GPIO_PWM0_PIN 输出一个脉冲后清除对应定时器的中断标志位

6.4.2. PWM demo

- 演示 GPIO_24-GPIO_27 输出 1000Hz PWM 波，关键代码说明如下：

设置 PWM 时钟分频系数，获取 PWM 时钟，计算 PWM 频率的宏定义

```
1 #define PWM_CLKDIV (1)
2 #define PWM_CLK TIMER_CLK
3 #define PWM_HZ(FREQ)    (PWM_CLK / (1 << PWM_CLKDIV) / FREQ)
4 #define PWM_KHZ(FREQ) (PWM_CLK / (1 << PWM_CLKDIV) / 1000 / FREQ)
5 #define PWM_MHZ(FREQ) (PWM_CLK / (1 << PWM_CLKDIV) / 1000000 / FREQ)
6 #define PWM_DUTY_RATIO          (0.5F)
```

假如要设置 1MHZ 频率的 PWM，则 PWM 配置时高低电平的时间周期都设置为：

PWM_MHZ(1)*PWM_DUTY_RATIO

先初始化时钟和串口打印的 UART 端口

```
1 SYSCTRL_ChipInit();
2 printport_init();
```

配置 PWM 的参数并使能 PWM 的 API

```
1 static void PWM_CoreParaConfig(TIM_PortEnum timx, uint8_t clkDiv, uint16_t lowLevelPeriod,
  uint16_t highLevelPeriod)
```

- timx：定时器端口号
- clkDiv：定时器分频系数
- lowLevelPeriod：低电平时间周期
- highLevelPeriod：高电平时间周期

将 GPIO 配置成对应的 PWM 复用模式，并配置 PWM 的频率为 1000Hz

```
GPIO_Config(GPIO_PWM0_PIN, GPIO_MODE_PWM0_1_OUT);
PWM_CoreParaConfig(TIM0,PWM_CLKDIV, (PWM_HZ(1000)*PWM_DUTY_RATIO),
(PWM_HZ(1000)*PWM_DUTY_RATIO));

GPIO_Config(GPIO_PWM1_PIN, GPIO_MODE_PWM0_3_OUT);
PWM_CoreParaConfig(TIM1, PWM_CLKDIV, (PWM_HZ(1000)*PWM_DUTY_RATIO),
(PWM_HZ(1000)*PWM_DUTY_RATIO));

GPIO_Config(GPIO_PWM2_PIN, GPIO_MODE_PWM_OUT);
PWM_CoreParaConfig(TIM2, PWM_CLKDIV,
(PWM_HZ(1000)*PWM_DUTY_RATIO),(PWM_HZ(1000)*PWM_DUTY_RATIO));

GPIO_Config(GPIO_PWM3_PIN, GPIO_MODE_PWM_OUT);
PWM_CoreParaConfig(TIM3, PWM_CLKDIV, (PWM_HZ(1000)*PWM_DUTY_RATIO),
(PWM_HZ(1000)*PWM_DUTY_RATIO));
```

PWM 输出在 GPIO 引脚上有三种复用方式：

- GPIO_MODE_PWM_OUT: 从 GPIO0 开始, 每 8 个 IO 为一组, 分别对应 PMW0~PWM7
- GPIO_MODE_PWM0_1_OUT：从 GPIO0 开始，每 2 个 IO 为一组，分别对应 PMW0~PWM1
- GPIO_MODE_PWM0_3_OUT：从 GPIO0 开始，每 4 个 IO 为一组，分别对应 PMW0~PWM3

6.4.3. PWM 差分输出 demo

- 演示 GPIO24 和 GPIO25 组合输出差分 PWM 波，关键代码说明如下：

设置 PWM 差分输出死区时间的宏定义

```
#define PWM_CLKDIV                  (6)
#define PWM_CLK                     TIMER_CLK
#define PWM_HZ(FREQ)                (PWM_CLK/(1 << PWM_CLKDIV)/FREQ)
#define PWM_KHZ(FREQ)               (PWM_CLK/(1 << PWM_CLKDIV)/1000/FREQ)
#define PWM_MHZ(FREQ)               (PWM_CLK/(1 << PWM_CLKDIV)/1000000/FREQ)

#define PWM_SYNC_PCNT               (PWM_HZ(200))//tim low
#define PWM_SYNC_NCNT               (PWM_HZ(400))
#define PWM_SYNC_PCNT_DEAD_TIME (TIMER_1MS(PWM_CLKDIV)*2)//tim high
```

```
#define PWM_SYNC_NCNT_DEAD_TIME (TIMER_1MS(PWM_CLKDIV)*4)
```

PWM 频率计算与 PWM demo 中的频率计算方法一致：

- PWM_SYNC_PCNT：第一路 PWM(端口序号为偶数)输出高电平的时间周期
- PWM_SYNC_NCNT：第二路 PWM(端口序号为奇数)输出高电平的时间周期
- PWM_SYNC_PCNT_DEAD_TIME：左死区时间周期
- PWM_SYNC_NCNT_DEAD_TIME：右死区时间周期

先初始化时钟和串口打印的 UART 端口

```
SYSCTRL_ChipInit();
printport_init();
```

设置 GPIO 复用为 PWM 模式

```
GPIO_Config(GPIO_PWM0_PIN, GPIO_MODE_PWM_OUT);
GPIO_Config(GPIO_PWM1_PIN, GPIO_MODE_PWM_OUT);
```

PWM 输出在 GPIO 引脚上有三种复用方式：

- GPIO_MODE_PWM_OUT: 从 GPIO0 开始, 每 8 个 IO 为一组, 分别对应 PMW0~PWM7
- GPIO_MODE_PWM0_1_OUT：从 GPIO0 开始，每 2 个 IO 为一组，分别对应 PMW0~PWM1
- GPIO_MODE_PWM0_3_OUT：从 GPIO0 开始，每 4 个 IO 为一组，分别对应 PMW0~PWM3

设置 PWM 差分输出配置的 API

```
 void PWM_SyncCoreParaConfig(TIM_PortEnum TIMx, uint8_t clk_div, uint32_t LeftBorderPeriod , uint32_t high_level_period,uint32_t RightBorderPeriod , uint32_t low_level_period , uint8_t SyncOutputInverse)
{
        _ASSERT(LeftBorderPeriod <= 0xffff);
        _ASSERT(high_level_period <= 0xffff);
        _ASSERT(RightBorderPeriod <= 0xffff);
        _ASSERT(low_level_period <= 0xffff);
```

```
    PWM_SyncInitStruct PWM_InitSyncStruct = {0};
    PWM_InitSyncStruct.timx = TIMx;
    PWM_InitSyncStruct.lowLevelPeriod = low_level_period;
    PWM_InitSyncStruct.highLevelPeriod = high_level_period;

    PWM_InitSyncStruct.leftBorderPeriod = LeftBorderPeriod;
    PWM_InitSyncStruct.rightBorderPeriod = RightBorderPeriod;

    PWM_InitSyncStruct.clockDiv = clk_div;
    PWM_InitSyncStruct.startLevel = 0;
    PWM_InitSyncStruct.syncOutputInverse = SyncOutputInverse;

    /*pwm core init*/
    TIM_PwmSyncInit(&PWM_InitSyncStruct);
    TIM_Cmd(TIMx,ENABLE);
    TIM_Cmd(TIMx+1,ENABLE);
}
```

- TIMx：两路 PWM 差分输出组合的第一路 PWM,端口序号为偶数，可以选择 TIM0,TIM2,TIM4,TIM6
- clk_div：PWM 时钟分频系数
- LeftBorderPeriod：左死区时间周期
- high_level_period：第一路 PWM（端口序号为偶数）高电平持续时间周期
- RightBorderPeriod：右死区时间周期
- low_level_period：第二路 PWM（端口序号为奇数）高电平持续时间周期
- SyncOutputInverse：决定哪一路 PWM 先输出高电平
  - 0：第一路 PWM (端口序号为偶数)先输出高电平
  - 1：第二路 PWM(端口序号为奇数)先输出高电平

7. FLASH

7.1. 宏定义、枚举及结构体说明

7.1.1. FLASH 物理地址转 CPU(XIP 地址)宏定义

```
#define PARAM_MCU_ADDR(flashAddr)      (((((flashAddr)-CORE_QSPI->flash_offset.bit.flash_offset)/34)*32)|0x1000000)
```

当需要使用 CPU 直接访问(读取)flash 明文参数区时需要使用这个宏将 flash 物理地址 flashAddr 转换为 CPU 逻辑地址（所选的 flashAddr 必须满足 (FLASH_PARAM_ADDR-CORE_QSPI->flash_offset.bit.flash_offset)/34 能够整除），且需要使用 FLASH_SetParamAddr 函数设置参数区起始地址，具体代码参考 7.4 说明

7.1.2. FLASH 读模式宏定义

```
#define QSPICFG_DUAL_MODE              (1 << 0)
#define QSPICFG_QUAD_MODE              (2 << 0)
#define QSPICFG_MBYTE                  (1 << 4)
#define QSPICFG_MBYTE_CONT             (1 << 5)
#define QCSFT_DUMMY                    (8)
#define QSPICFG_MODE_3B                (QSPICFG_DUAL_MODE | 8 << QCSFT_DUMMY)
#define QSPICFG_MODE_6B                (QSPICFG_QUAD_MODE | 8 << QCSFT_DUMMY)
#define QSPICFG_MODE_BB                (QSPICFG_DUAL_MODE | QSPICFG_MBYTE | 0x6000)
#define QSPICFG_MODE_EB                (QSPICFG_QUAD_MODE | QSPICFG_MBYTE | 0x6000 | 4 << QCSFT_DUMMY)
#define QSPICFG_MODE_E7                (QSPICFG_QUAD_MODE | QSPICFG_MBYTE | 0x6000 | 2 << QCSFT_DUMMY)
```

7.2. FLASH 操作接口汇总

函数类型	函数名	说明
FLASH 相关接口	FLASH_Write	向 FLASH 目标地址写数据
	FLASH_Read	读取 FLASH 目标地址数据

	FLASH_Lock	锁定 FLASH
	FLASH_Unlock	解锁 FLASH
	FLASH_PageErase	FLASH 页擦除
	FLASH_SectorErase	FLASH 扇区擦除
	FLASH_BlockErase_32k	擦除 FLASH 目标地址 32k 数据
	FLASH_BlockErase_64k	擦除 FLASH 目标地址 64k 数据
	FLASH_Powerdown	FLASH 进入 Powerdown 模式
	FLASH_SetReadMode	配置 FLASH 读数据模式
	FLASH_GetJedecId	获取 FLASH ID
	FLASH_SetParamAddr	FLASH 参数区偏移量配置函数

7.3. 接口说明

7.3.1. uint8_t FLASH_Write(uint32_t flashAddr, uint32_t tlen, uint8_t *tbuf)

- 描述：向 FLASH 目标地址写数据
- 参数：
  - flashAddr：FLASH 目标地址
  - tlen：写数据长度
  - tbuf：写数据地址
- 返回值：
  - FLASH 写状态 ：
    - 1：成功

- 0：失败

7.3.2. uint8_t FLASH_Read(uint32_t flashAddr, uint32_t rlen, uint8_t *rbuf)

- 描述：读取 FLASH 目标地址数据
- 参数：
  - flashAddr：FLASH 目标地址
  - rlen：读数据长度
  - rbuf：读数据地址
- 返回值：
  - FLASH 读状态 ：
    - 1：成功
    - 0：失败

7.3.3. void FLASH_Lock(void)

- 描述：启用 FLASH 块保护，将 FLASH 锁定
- 参数：none
- 返回值：
  - FLASH 锁定状态：
    - 1：成功
    - 0：失败

7.3.4. uint8_t FLASH_Unlock(void)

- 描述：解锁 FLASH

- 参数：none
- 返回值：
  - FLASH 解锁状态 ：
    - 1：成功
    - 0：失败

7.3.5. uint8_t FLASH_PageErase(uint32_t flashAddr)

- 描述：FLASH 页擦除
- 参数：
  - flashAddr：FLASH 目标地址
- 返回值：
  - 擦除 FLASH 页状态：
    - 1：成功
    - 0：失败

7.3.6. uint8_t FLASH_SectorErase(uint32_t flashAddr)

- 描述：FLASH 扇区擦除
- 参数：
  - flashAddr：FLASH 目标地址
- 返回值：
  - 擦除 FLASH 扇区状态：
    - 1：成功
    - 0：失败

7.3.7. uint8_t FLASH_BlockErase_32k(uint32_t flashAddr)

描述：擦除 FLASH 目标地址 32k 数据

- 参数：
  - flashAddr：FLASH 目标地址
- 返回值：
  - 擦除 FLASH 32k 状态：
    - 1：成功
    - 0：失败

7.3.8. uint8_t FLASH_BlockErase_64k(uint32_t flashAddr)

- 描述：擦除 FLASH 目标地址 64k 数据
- 参数：
  - flashAddr：目标 FLASH 地址
- 返回值：
  - 擦除 FLASH 64k 状态：
    - 1：成功
    - 0：失败

7.3.9. uint8_t FLASH_Powerdown(void)

- 描述：FLASH 进入 Powerdown 模式
- 参数：none
- 返回值：
  - FLASH 关闭状态：
    - 1：成功

- 0：失败

7.3.10. uint8_t FLASH_SetReadMode(uint8_t readModeCmd)

- 描述：配置 FLASH 读取数据模式
- 参数：
  - readModeCmd：读模式配置，详见 QSPICFG_MODE_XX 宏定义组
- 返回值：
  - 读模式配置状态：
    - 1：成功
    - 0：失败

7.3.11. uint32_t FLASH_GetJedecId(void)

- 描述：读取 FLASH JEDEC ID
- 参数：none
- 返回值：
  - FLASH 的 JEDEC ID

7.3.12. void FLASH_SetParamAddr(uint32_t paramOffset)

- 描述：FLASH 参数区偏移量配置函数
- 参数：
  - paramOffset：FLASH 参数区偏移量（CPU 逻辑地址）
- 返回值：none

7.4. demo 说明

- 演示对 FLASH 进行读写操作以及通过 CPU 直接访问明文写入的参数区数据，关键代码说明如下：

芯片上电后读取 FLASH ID，解锁 FLASH 数据，进行 4kb 的 FLASH 读写测试

```
int main(void)
{
    ......  //其他功能模块初始化
    yc_log("FLASH ID:%x\n",FLASH_GetJedecId());
    FLASH_LockUnlockTest();
    FLASH_RWTest(TEST_FLASH_ADDR, TEST_FLASH_LEN);

    while(1)
    {
    }
}
void FLASH_LockUnlockTest(void)
{
    if (FLASH_Lock() == SUCCESS)
        yc_log("FLASH_Lock success");
    else
        yc_log("FLASH_Lock Fail");

    if (FLASH_Unlock() == SUCCESS)
        yc_log("FLASH_Unlock success");
    else
        yc_log("FLASH_Unlock Fail");
}
```

64kb 的读写测试为对指定的地址先进行擦除，判断擦除是否成功，再向 FLASH 写入 4kb 的随机值；读取 FLASH 的值与写入的值进行对比，都一致判断读写成功；再向下个 4kb 地址循环，直到 64k 数据均进行完读写测试。

```
uint8_t FLASH_RWTest(uint32_t flash_addr, uint32_t len)
{
    uint8_t tbuf[SECTOR_SIZE];
    uint8_t rbuf[SECTOR_SIZE];
    uint32_t tested_len = 0;
    uint32_t packlen = 0;

    if(flash_addr%SECTOR_SIZE != 0)
    {
```

```
        yc_log("flash_addr unaligned[flash_addr=%x]",flash_addr);
        return ERROR;
    }

    gen_random_array(sizeof(tbuf), tbuf);
    while(tested_len < len)
    {
        packlen = (len-tested_len) > SECTOR_SIZE ? SECTOR_SIZE : (len-tested_len);
        yc_log("test flash_addr=%x",flash_addr);
        FLASH_SectorErase(flash_addr);
        FLASH_Read(flash_addr,packlen,rbuf);
         for(int j=0;j<packlen;j++)
        {
            if(rbuf[j] != 0xff)
            {
                yc_log("erase error break! rbuf[%d] = %d",j,rbuf[j]);
                return ERROR;
            }
        }
        FLASH_Write(flash_addr, packlen, tbuf);
        FLASH_Read(flash_addr, packlen, rbuf);
        if (0 != memcmp(tbuf, rbuf, packlen))
        {
            yc_log("FLASH_RWTest Fail[flash_addr=%x]",flash_addr);
            printv("tbuf",tbuf,sizeof(tbuf));
            printv("rbuf",rbuf,sizeof(rbuf));
            return ERROR;
        }

        flash_addr += packlen;
        tested_len += packlen;
    }
    yc_log("FLASH_RWTest success");
    return SUCCESS;
}
```

以下代码使用 FLASH_Write 函数写入参数区数据，并使用 FLASH_SetParamAddr 配置参数区起始地址，然后通过 CPU 直接读取打印

```
void cpu_read_param_area_test(void)
{
    uint8_t write_buf[256];
    yc_log("PARAM_FLASH_ADDR=%x",FLASH_PARAM_ADDR);
    yc_log("PARAM_MCU_ADDR=%x",PARAM_MCU_ADDR(FLASH_PARAM_ADDR));
```

```
    for(int i=0;i<sizeof(write_buf);i++)
    {
        write_buf[i]=i;
    }
    yc_log_buf("write buf",write_buf,sizeof(write_buf));
    FLASH_SectorErase(FLASH_PARAM_ADDR);
    FLASH_Write(FLASH_PARAM_ADDR,sizeof(write_buf),write_buf);
    FLASH_SetParamAddr(PARAM_MCU_ADDR(FLASH_PARAM_ADDR));
    yc_log_buf("cpu read",(uint8_t*)PARAM_MCU_ADDR(FLASH_PARAM_ADDR),sizeof(write_buf));

}
```

8. HSPI

8.1. 宏定义、枚举及结构体说明

8.1.1. HSPI 极性宏定义

```
#define HSPI_CPOL_LOW                    (0)
#define HSPI_CPOL_HIGH                   (1)
```

8.1.2. HSPI 相位宏定义

```
#define HSPI_CPHA_FIRST_EDGE             (0)
#define HSPI_CPHA_SECOND_EDGE            (1)
```

8.1.3. HSPI 状态类型枚举

```
typedef enum
{
    DMA_DONE = 0,              //DMA 完成
    TXFIFO_LOW,                //TXFIFO 低
    TXFIFO_OVERFLOW,           //TXFIFO 溢出
    RXFIFO_HIGH,               //RXFIFO 高
    RXFIFO_OVERFLOW,           //RXFIFO 溢出
    START_OVER_QUICK,          //开启过快
    TX_EMPTY,                  //TX 空
    RX_NOT_EMPTY,              //RX 非空
    RX_OVERRUN,                //RX 数据溢出
    ALL_SPI_STATUS = 0xff
} HSPI_StatusEnum;
```

8.1.4. HSPI 初始化结构体

```
typedef struct
{
    uint8_t rxAdjClk;          /*!< 接收数据相位调整，
                                  采样时间延迟 (RxAdjClk + 1)*spi_clk 时钟周期，不大于 HSPI 时分频，
                                  参数在 0x00 and 0x0F 之间 */
    uint8_t clkDiv;            /*!< HSPI 时钟分频，分频值为 (ClkDiv + 1)*2.
                                  参数在 0x00 and 0x0F 之间 */
```

```
    uint8_t txRxDly;              /*!< 发送和接收之间的保护间隔区，间隔 值为 spi_clk*(TxRxDly + 1)*4.
                                   参数在 0x00 and 0x0F 之间 */
    uint8_t ncsDly;               /*!< NCS 延时时间,延时值为 spi_clk*(NcsDly + 1).
                                   参数在 0x00 and 0x07 之间 */
    uint16_t rxAdjen;             /*!< 接收数据相位保护使能开关.
                                   参数在 0x00 and 0x01 之间 */
    uint16_t txRxSeq;             /*!< tx 和 rx 序列控制:
                                   参数值决定为半双工或全双工 */
    uint16_t cpha;                /*!< 时钟边沿捕获.
                                   参数值参考 HSPI 时钟边沿宏定义 */
    uint16_t cpol;                /*!< 时钟极性状态.
                                   参数值参考 HSPI 时钟极性宏定义 */
    uint8_t rxFifoWaterLevel;     /*!< rx fifo waterlevel 水位
                                   参数在 0x00 and 0x1F 之间 */
    uint8_t txFifoWaterLevel;     /*!<   tx fifo waterlevel 水位
                                   参数在 0x00 and 0x1F 之间 */

    uint8_t fifoSwEn;             /*!< FIFO 软件控制使能 */
    uint8_t halfDuplexEn;         /*!< 半双工模式使能(主机模式，使用 MOSI 用作双向通信通道从机模式，使用 MISO 用作双向通信通道) */
    uint8_t halfDuplexOe;         /*!< 半双工模式数据发送使能(只在从机模式生效) */
    uint8_t slaveMode;            /*!< 从机模式 */
} HSPI_InitStruct;
```

8.2. HSPI 操作接口汇总

函数类型	函数名	说明
HSPI 相关接口	HSPI_Init	HSPI 初始化
	HSPI_DeInit	清除 HSPI 相关配置
	HSPI_EnableCmd	HSPI 模块使能
	HSPI_DmaEnableCmd	HSPI DMA 功能使能
	HSPI_GetClk	获取当前 HSPI 时钟频率
	HSPI_FifoAutoStartCmd	配置 HSPI FIFO 自动复位功能

	HSPI_SetIrq	HSPI 配置中断使能
	HSPI_SlaveEnableCmd	HSPI 从机模式使能
	HSPI_ClearIrq	清除 HSPI 的中断和状态
	HSPI_GetSpiStatus	获取 HSPI 对应状态
	HSPI_GetIrqStatus	获取 HSPI 对应中断状态
	HSPI_GetAllIrqStatus	获取 HSPI 所有状态
	HSPI_MoveRxFifoPtr	移动 RxFIFO 指针
	HSPI_MoveTxFifoPtr	移动 TxFIFO 指针
	HSPI_ReadRxFifoData	读取 RxFIFO 1 字节数据
	HSPI_WriteRxFifoData	写 RxFIFO 1 字节数据
	HSPI_ReadTxFifoData	读取 TxFIFO 1 字节数据
	HSPI_WriteTxFifoData	写 TxFIFO 1 字节数据
	HSPI_SendData	HSPI 发送 1 字节数据
	HSPI_ReceiveData	HSPI 接收 1 字节数据
	HSPI_SendBuff	HSPI 发送多字节数据
	HSPI_ReceiveBuff	HSPI 接收多字节数据
	HSPI_SendAndReceiveData	HSPI 收发多字节数据

8.3. 接口说明

8.3.1. void HSPI_Init(HSPI_InitStruct* initStruct)

- 描述：HSPI 初始化
- 参数：
  - initStruct：指向 HSPI_InitStruct 结构体的指针，详见 HSPI 初始化结构体
- 返回值：none

8.3.2. void HSPI_DeInit(void)

- 描述：清除 HSPI 相关配置
- 参数：none
- 返回值：none

8.3.3. void HSPI_EnableCmd(FunctionalState newState)

- 描述：HSPI 模块使能
- 参数：
  - newState：
    - ENABLE：使能
    - DISABLE：失能
- 返回值：none

8.3.4. void HSPI_DmaEnableCmd(FunctionalState newState)

- 描述：HSPI DMA 使能
- 参数：
  - newState：
    - ENABLE：使能
    - DISABLE：失能

- 返回值：none

8.3.5. uint32_t HSPI_GetClk(void)

- 描述：获取当前 HSPI 时钟频率
- 参数：none
- 返回值：
  - 当前 HSPI 时钟频率

8.3.6. void HSPI_FifoAutoStartCmd(FunctionalState newState)

- 描述：配置 HSPI 在利用 FIFO 收发数据，完成后自动复位，FIFO 中数据清零
- 参数：
  - newState：
    - ENABLE：使能
    - DISABLE：失能
- 返回值：none

8.3.7. void HSPI_SetIrq(HSPI_StatusEnum type, FunctionalState newState)

- 描述：HSPI 配置中断使能
- 参数：
  - type：HSPI 状态枚举，详见 HSPI 枚举
  - newState：
    - ENABLE：使能

- DISABLE：失能

- 返回值：none

8.3.8. void HSPI_SlaveEnableCmd(FunctionalState newState)

- 描述：HSPI 从机模式使能
- 参数：
  - newState：
    - ENABLE：使能
    - DISABLE：失能
- 返回值：none

8.3.9. void HSPI_ClearIrq(HSPI_StatusEnum type)

- 描述：清除 HSPI 的中断和状态
- 参数：
  - type：HSPI 状态枚举，详见 HSPI 枚举
- 返回值：none

8.3.10. FlagStatus HSPI_GetSpiStatus(HSPI_StatusEnum type)

- 描述：获取 HSPI 对应状态
- 参数：
  - type：HSPI 状态枚举，详见 HSPI 枚举
- 返回值：
  - HSPI 查询目标当前状态标志位

- 1：已置起
- 0：未置起

8.3.11. FlagStatus HSPI_GetIrqStatus(HSPI_StatusEnum type)

- 描述：获取 HSPI 对应中断状态
- 参数：
  - type：HSPI 状态枚举，详见 HSPI 枚举
- 返回值：
  - HSPI 查询目标当前中断状态标志位
    - 1：已置起
    - 0：未置起

8.3.12. uint16_t HSPI_GetAllIrqStatus(void)

- 描述：获取 HSPI 所有状态
- 参数：none
- 返回值：
  - 返回值：HSPI 当前所有状态，以下为各 bit 代表的意义：
    - bit 0：dma 完成
    - bit 1：tx fifo 低
    - bit 2：tx fifo 溢出
    - bit 3：rx fifo 高
    - bit 4：rx fifo 溢出
    - bit 5：启动过快

- bit 6：tx 空
- bit 7：rx 非空
- bit 8：rx 数据溢出

8.3.13. void HSPI_MoveRxFifoPtr(void)

- 描述：移动 RxFIFO 指针
- 参数：none
- 返回值：none

8.3.14. void HSPI_MoveTxFifoPtr(void)

- 描述：移动 TxFIFO 指针
- 参数：none
- 返回值：none

8.3.15. uint8_t HSPI_ReadRxFifoData(void)

- 描述：读取 RxFIFO 1 字节数据
- 参数：none
- 返回值：
  - 1 字节 Rx 数据

8.3.16. void HSPI_WriteRxFifoData(uint8_t data)

- 描述：写 RxFIFO 1 字节数据
- 参数：
  - data：要写入的 1 字节数据
- 返回值：none

8.3.17. uint8_t HSPI_ReadTxFifoData(void)

- 描述：读取 TxFIFO 1 字节数据
- 参数：none
- 返回值：
  - 1 字节 Tx 数据

8.3.18. void HSPI_WriteTxFifoData(uint8_t data)

- 描述：写 TxFIFO 1 字节数据
- 参数：
  - data：要写入的 1 字节数据
- 返回值：none

8.3.19. void HSPI_SendData(uint8_t data)

- 描述：HSPI 发送 1 字节数据
- 参数：
  - data：要发送的 1 字节数据
- 返回值：none

8.3.20. uint8_t HSPI_ReceiveData(void)

- 描述：HSPI 接收 1 字节数据
- 参数：none
- 返回值：
  - 收取的 1 字节数据

8.3.21. void HSPI_SendBuff(uint8_t *buff, uint32_t len)

- 描述：HSPI 发送多字节数据
- 参数：
  - buff：待发送数据的起始地址
  - len：发送数据长度
- 返回值：none

8.3.22. void HSPI_ReceiveBuff(uint8_t *buff, uint32_t len)

- 描述：HSPI 接收多字节数据
- 参数：
  - buff：接收数据的起始地址
  - len：接收数据长度
- 返回值：none

8.3.23. void HSPI_SendAndReceiveData(uint8_t *txbuff, uint32_t txlen, uint8_t *rxbuff, uint32_t rxlen)

- 描述：HSPI 收发多字节数据
- 参数：
  - txbuff：待发送数据的起始地址
  - txlen：发送数据长度
  - rxbuff：接收数据的起始地址
  - rxlen：接收数据长度
- 返回值：none

8.4. demo 说明

8.4.1. hspi flash demo

- 演示使用 HSPI 对 FLASH 进行读写测试，关键代码说明如下：

初始化系统和串口打印初始化

```
    SYSCTRL_ChipInit();

    printport_init();
```

GPIO 初始化

```
  void W25Q16_Gpioinit(void)
{
    #if TRANSCEIVER_MODE==SEND_AND_RECEIVE_TOGETHER
    GPIO_Config(SPI_NCS_PIN, GPIO_MODE_SPID_NCS);
    #else
    GPIO_Config(SPI_NCS_PIN, GPIO_MODE_OUT_HIGH);
    #endif
    GPIO_Config(SPI_MISO_PIN, GPIO_MODE_SPID_MISO);
    GPIO_Config(SPI_MOSI_PIN, GPIO_MODE_SPID_MOSI);
    GPIO_Config(SPI_SCK_PIN, GPIO_MODE_SPID_SCK);
}
```

HSPI 初始化

```
void HSPI_Configuration(void)
{
    HSPI_InitStruct hspictrl = {0};
    uint32_t spi_module_clk=HSPI_GetClk();
    yc_log("SpiModuleClk=%d",spi_module_clk);

    hspictrl.rxAdjClk = 0;
    hspictrl.rxAdjen = 1;
    hspictrl.clkDiv = 0xf;      /* spi clock=spi_module_clk/((clkdiv+1)*2) */
    hspictrl.txRxDly = 0;
    hspictrl.ncsDly = 7;
    hspictrl.txRxSeq = HSPI_TXRX_SEQUENCE_TX_FIRST;
    hspictrl.cpha = HSPI_CPHA_FIRST_EDGE;
    hspictrl.cpol = HSPI_CPOL_LOW;
    hspictrl.rxFifoWaterLevel = 1;
    hspictrl.txFifoWaterLevel = 5;
    hspictrl.fifoSwEn = 0;
    hspictrl.slaveMode = 0;
    hspictrl.halfDuplexEn = 0;
```

```
    HSPI_Init(&hspictrl);
}
```

配置 DMA 使能

```
HSPI_DmaEnableCmd(ENABLE);
HSPI_EnableCmd(ENABLE);
```

对 FLASH 进行读写测试,读写一致判断为成功

```
yc_log("id=%x\r\n",W25Q16_ReadDeviceID());
 for(int i=0;i<TEST_SIZE;i+=SECTOR_SIZE)
{
    W25Q16_SectorErase(TEST_ADDR+i);
}
yc_log("flash erase end");

for(int i=0;i<TEST_SIZE;i+=SECTOR_SIZE)
{
    W25Q16_ReadData(TEST_ADDR+i, readBuf, sizeof(readBuf));
    for(int j=0;j<SECTOR_SIZE;j++)
    {
        if(readBuf[j]!=0xff)
        {
            test_error_flag = 1;
            yc_log("0x%x erase error[%02x]",TEST_ADDR+i+j,readBuf[j]);
            break;
        }
    }
    if(test_error_flag)
    {
        break;
    }
}
if(test_error_flag == 0)
{
    yc_log("flash erase check success");
}
test_error_flag = 0;
for(int i=0;i<TEST_SIZE;i+=SECTOR_SIZE)
{
    W25Q16_writeData(TEST_ADDR+i,writeBuf,sizeof(writeBuf));
}
yc_log("flash write end");

```

```
for(int i=0;i<TEST_SIZE;i+=SECTOR_SIZE)
{
    W25Q16_ReadData(TEST_ADDR+i, readBuf, sizeof(readBuf));
    for(int j=0;j<SECTOR_SIZE;j++)
    {
        if(readBuf[j]!=writeBuf[j])
        {
            test_error_flag = 1;
            yc_log("0x%x check error[%02x]",TEST_ADDR+i+j,readBuf[j]);
            break;
        }
    }
    if(test_error_flag)
    {
        break;
    }
}
if(test_error_flag == 0)
{
    yc_log("flash write check success");
}
```

8.4.2. hspi master demo

- HSPI 主模式收发测试程序，关键代码说明如下：

初始化系统和串口打印初始化

```
SYSCTRL_ChipInit();
printport_init();
```

GPIO 初始化

```
  void W25Q16_Gpioinit(void)
{
    #if TRANSCEIVER_MODE==SEND_AND_RECEIVE_TOGETHER
    GPIO_Config(SPI_NCS_PIN, GPIO_MODE_SPID_NCS);
    #else
    GPIO_Config(SPI_NCS_PIN, GPIO_MODE_OUT_HIGH);
    #endif
    GPIO_Config(SPI_MISO_PIN, GPIO_MODE_SPID_MISO);
    GPIO_Config(SPI_MOSI_PIN, GPIO_MODE_SPID_MOSI);
    GPIO_Config(SPI_SCK_PIN, GPIO_MODE_SPID_SCK);
```

```
}
```

HSPI 初始化

```
void HSPI_Configuration(void)
{
    HSPI_InitStruct hspictrl = {0};
    uint32_t spi_module_clk=HSPI_GetClk();
    yc_log("SpiModuleClk=%d",spi_module_clk);

    hspictrl.rxAdjClk = 0x0;
    hspictrl.rxAdjen = 0;
    hspictrl.clkDiv = 0xf;      /* spi clock=spi_module_clk/((clkdiv+1)*2) */
    hspictrl.txRxDly = 0x0b;
    hspictrl.ncsDly = 7;
    hspictrl.txRxSeq = HSPI_TXRX_SEQUENCE_TX_FIRST;
    hspictrl.cpha = HSPI_CPHA_SECOND_EDGE;
    hspictrl.cpol = HSPI_CPOL_HIGH;
    hspictrl.rxFifoWaterLevel = 0;
    hspictrl.txFifoWaterLevel = 0;
    hspictrl.fifoSwEn = 0;
    hspictrl.slaveMode = 0;
    hspictrl.halfDuplexEn = 0;
    hspictrl.halfDuplexOe = 0;
    HSPI_Init(&hspictrl);
}
```

HSPI DMA 配置

```
HSPI_DmaEnableCmd(ENABLE);
HSPI_EnableCmd(ENABLE);
```

收发数据

```
HSPI_SendAndReceiveData(write_test_buf,TEST_SIZE,read_test_buf,TEST_SIZE);
```

8.4.3. hspi slave demo

- HSPI 从模式收发测试程序，关键代码说明如下：

初始化系统和串口打印初始化

```
SYSCTRL_ChipInit();
```

```
printport_init();
```

GPIO 初始化

```
  void W25Q16_Gpioinit(void)
{
    #if TRANSCEIVER_MODE==SEND_AND_RECEIVE_TOGETHER
    GPIO_Config(SPI_NCS_PIN, GPIO_MODE_SPID_NCS);
    #else
    GPIO_Config(SPI_NCS_PIN, GPIO_MODE_OUT_HIGH);
    #endif
    GPIO_Config(SPI_MISO_PIN, GPIO_MODE_SPID_MISO);
    GPIO_Config(SPI_MOSI_PIN, GPIO_MODE_SPID_MOSI);
    GPIO_Config(SPI_SCK_PIN, GPIO_MODE_SPID_SCK);
}
```

HSPI 配置初始化，开启 slave

```
void HSPI_Configuration(void)
{
    HSPI_InitStruct hspictrl = {0};
    uint32_t spi_module_clk=HSPI_GetClk();
    yc_log("SpiModuleClk=%d",spi_module_clk);

    hspictrl.rxAdjClk = 0x0;
    hspictrl.rxAdjen = 0;
    hspictrl.clkDiv = 0xf;      /* spi clock=spi_module_clk/((clkdiv+1)*2) */
    hspictrl.txRxDly = 0x0b;
    hspictrl.ncsDly = 7;
    hspictrl.txRxSeq = HSPI_TXRX_SEQUENCE_TX_FIRST;
    hspictrl.cpha = HSPI_CPHA_SECOND_EDGE;
    hspictrl.cpol = HSPI_CPOL_HIGH;
    hspictrl.rxFifoWaterLevel = 16;
    hspictrl.txFifoWaterLevel = 16;
    hspictrl.fifoSwEn = 0;
    hspictrl.slaveMode = 1;
    hspictrl.halfDuplexEn = 0;
    hspictrl.halfDuplexOe = 0;
    HSPI_Init(&hspictrl);
}
```

配置 DMA

```
HSPI_DmaEnableCmd(ENABLE);
```

```
2  HSPI_EnableCmd(ENABLE);
```

slave 使能

```
1  HSPI_SlaveEnableCmd(ENABLE);
```

收发数据

```
1  HSPI_SendAndReceiveData(write_test_buf,TEST_SIZE,read_test_buf,TEST_SIZE);
```

9. IIC

9.1. 宏定义、枚举及结构体说明

9.1.1. IIC 初始化结构体

```
1  typedef struct{
2          uint8_t scll;       /*!< SCL 时钟低电平时间  */
3          uint8_t sclh;       /*!< SCL 时钟高电平时间  */
4          uint8_t stsu;       /*!< SDA 启动建立时间  */
5          uint8_t sthd;       /*!< SDA 启动保持时间  */
6          uint8_t sosu;       /*!< SDA 停止建立时间  */
7          uint8_t dtsu;       /*!< SDA 数据建立时间  */
8          uint8_t dthd;       /*!< SDA 数据保持时间  */
9  } IIC_InitStruct;
```

9.2. IIC 操作接口汇总

函数类型	函数名	说明
IIC 相关接口	IIC_Init	IIC 初始化
	IIC_DeInit	清除 IIC 相关配置
	IIC_SendData	IIC 发送数据
	IIC_ReceiveData	IIC 接收数据

	IIC_SetIrq	配置 IIC 中断使能
	IIC_ClearIrq	清除 IIC 中断

9.3. 接口说明

9.3.1. void IIC_Init(CORE_IIC_Type* iicx, IIC_InitStruct *iicInitStruct)

1. 描述：IIC 初始化
2. 参数：
   - iicx：
     - CORE_IIC
     - CORE_IIC1
   - iicInitStruct：iic 初始化，详见 iic 初始化结构体
3. 返回值：none

9.3.2. void IIC_DeInit(CORE_IIC_Type* iicx)

4. 描述：清除 IIC 相关配置
5. 参数：
   - iicx：
     - CORE_IIC
     - CORE_IIC1
6. 返回值：none

9.3.3. void IIC_SendData(CORE_IIC_Type* iicx, uint8_t *src, uint16_t len)

7. 描述：IIC 发送数据
8. 参数：
   - iicx：
     - CORE_IIC
     - CORE_IIC1
   - src：发送数据地址
   - len：发送数据长度
9. 返回值：none

9.3.4. void IIC_ReceiveData(CORE_IIC_Type* iicx, uint8_t *src, uint16_t srcLen, uint8_t *dest, uint16_t destLen)

10. 描述：IIC 接收数据
11. 参数：
    - iicx：
      - CORE_IIC
      - CORE_IIC1
    - src：发送数据地址
    - srclen：发送数据长度
    - dest：接收数据地址
    - destLen：接收数据长度
12. 返回值：none

9.3.5. void IIC_SetIrq(CORE_IIC_Type* iicx, FunctionalState newState)

13. 描述：配置 IIC 中断使能

14. 参数：

- iicx：
  - CORE_IIC
  - CORE_IIC1
- newState：
  - ENABLE：使能
  - DISABLE：失能

15. 返回值：none

9.3.6. void IIC_ClearIrq(CORE_IIC_Type* iicx)

16. 描述：清除 IIC 中断状态

17. 参数：

- iicx：
  - CORE_IIC
  - CORE_IIC1

18. 返回值：none

9.4. demo 说明

- 演示使用 IIC 对 EEPROM 的读写及一致性检查，关键代码说明如下：

IIC EEPROM 数据头结构体

```
typedef struct
{
    uint8_t deviceaddrwrite;    /*!< IIC 设备地址 */
    uint8_t writeaddr[2];       /*!< IIC 处理数据地址 */
```

```
}IIC_DataHeadStruct;
```

IIC EEPROM 写数据结构体

```
typedef struct
{
    IIC_DataHeadStruct sendhead;
    uint8_t data[EEPROM_PAGE_SIZE];
}IIC_SendDataStruct;
```

IIC EEPROM 读数据结构体

```
typedef struct
{
    IIC_DataHeadStruct sendhead;
    uint8_t data[EEPROM_PAGE_SIZE];
}IIC_SendDataStruct;
```

初始化系统和串口打印初始化

```
    SYSCTRL_ChipInit();
    printport_init();
```

GPIO 初始化

```
void EEPROM_GPIO_Init(void)
{
    if(IIC_PORT==CORE_IIC)
    {
        enable_clock(CLK_IIC0);
        GPIO_Config(IIC_SCL_PIN, GPIO_MODE_I2C_SCL);
        GPIO_Config(IIC_SDA_PIN, GPIO_MODE_I2C_SDA);
        yc_log("IIC0 MODE SET OK!\n");
    }
    else if(IIC_PORT==CORE_IIC1)
    {
        enable_clock(CLK_IIC1);
        GPIO_Config(IIC_SCL_PIN, GPIO_MODE_I2C_SCL1);
        GPIO_Config(IIC_SDA_PIN, GPIO_MODE_I2C_SDA1);
        yc_log("IIC1 MODE SET OK!\n");
    }
}
```

设置 IIC 初始化频率，开启中断

```
IIC_Configuration(100000);      /*!< IIC rate 100k */
IIC_SetIrq(IIC_PORT,IIC_INT_CONFIG);
if(IIC_INT_CONFIG==ENABLE)
{
    PLIC_SetPriority(IIC_IRQ_PORT,2);      /*!< IIC 中断使能 */
    PLIC_EnableIRQ(IIC_IRQ_PORT);
}
```

EEPROM 写数据，先写 7 位 IIC 设备地址和写位，再写 IIC 要写的地址

```
void Eeprom_Write(uint16_t wAddr,uint8_t *data,uint16_t len)
{
    IIC_SendDataStruct send;
    uint32_t writed_len=0,pack_len=0;
    send.sendhead.deviceaddrwrite = IIC_DEVICE_WRITE_ADDR;
    for(writed_len=0;writed_len<len;)
    {
        send.sendhead.writeaddr[0] = ((wAddr >> 8)&0xff);
        send.sendhead.writeaddr[1] = (wAddr &0xff);
        pack_len=(len-writed_len)>EEPROM_PAGE_SIZE?EEPROM_PAGE_SIZE:(len-writed_len);
        memcpy(&send.data,data,pack_len);
        IIC_SendData(IIC_PORT,(uint8_t *)&send,pack_len+3);
        delay_ms(5);                /*!< EEPROM 每页写时间 5ms */
        data+=pack_len;
        wAddr+=pack_len;
        writed_len+=pack_len;
    }
}
```

EEPROM 读数据，先写 7 位 IIC 设备地址和写位，再写 IIC 要读的地址

```
void Eeprom_Read(uint16_t rAddr,uint8_t *data,uint16_t len)
{
    uint8_t sendBuff[4];
    IIC_ReadDataStruct *rSend;
    rSend=(IIC_ReadDataStruct *)sendBuff;

    rSend->readhead.deviceaddrwrite=IIC_DEVICE_WRITE_ADDR;
    rSend->readhead.writeaddr[0]=((rAddr>>8)&0xff);
    rSend->readhead.writeaddr[1]=(rAddr&0xff);
    rSend->deviceaddrread=IIC_DEVICE_READ_ADDR;

    IIC_ReceiveData(IIC_PORT,sendBuff,4,data,len);
}
```

读写一致性检查

```
Eeprom_Write(0,write_buff,TEST_SIZE);
Eeprom_Read(0,read_buff,TEST_SIZE);
for(i=0;i<TEST_SIZE;i++)
{
    if(write_buff[i]!=read_buff[i])
    {
        yc_log("eeprom write&read error\nwrite_buff[%d]=%d\tread_buff[%d]=%d\n",i,write_buff[i],i,read_buff[i]);
        break;
    }
}
if(i == TEST_SIZE)
{
    yc_log("eeprom write&read success\n");
}
```

10. LPM

10.1. 宏定义、枚举及结构体说明

10.1.1. GPIO 睡眠保持电平枚举

```
typedef enum
{
    LATCH_NONE = 0,
    LATCH_PD,
    LATCH_PU,
    LATCH_OUTPUT_LOW,
    LATCH_OUTPUT_HIGH
}LPM_LatchEnum;
```

10.1.2. 睡眠模式枚举

```
typedef enum
{
   SLEEP_MODE_NONE = 0,
   SLEEP_MODE_LPM,
   SLEEP_MODE_DEEP
} LPM_ModeEnum;
```

10.1.3. 唤醒模式枚举

```
typedef enum
{
    WAKEUP_FROM_MAIN = 0,
    WAKEUP_FROM_SLEEP
}LPM_WakeupModeEnum;
```

10.2. LPM 操作接口汇总

函数类型	函数名	说明
LPM 相关接口	LPM_Init	LPM 初始化

	LPM_SetGpioLatch	LPM 休眠 GPIO 电平保持状态配置
	LPM_Sleep	LPM 休眠入口函数

10.3. 接口说明

10.3.1. void LPM_Init(void)

- 描述：LPM 初始化，
- 参数：none
- 返回值：none

10.3.2. void LPM_SetGpioLatch(GPIO_NumEnum gpiox,LPM_LatchEnum type)

- 描述：配置休眠时 GPIO 保持电平状态
- 参数：
  - gpiox：GPIO Pin 号 ，详见 GPIO Pin 枚举
  - type：GPIO 休眠电平状态，详见 GPIO 睡眠保持电平枚举
- 返回值：none

10.3.3. void LPM_Sleep(uint32_t wakeTimeMs, LPM_ModeEnum sleepLevel,uint8_t retention_cfg,LPM_WakeupModeEnum wakeMode,lpmPreActionCb lpmPreCallback,lpmPostActionCb lpmPostCallback)

- 描述：LPM 睡眠配置
- 参数：

- wakeTimeMs：定时唤醒时间，单位 ms，0 为不开启定时唤醒
- sleepLevel：睡眠模式，详见睡眠模式枚举
- 睡眠保电区域配置：
  - bit0：ram0[0x10000-0x11fff]，8k
  - bit1：ram1[0x12000-0x13fff]，8k
  - bit2：ram2[0x14000-0x17fff]，6k
  - bit3：ram3[0x18000-0x1ffff]，32k
  - bit4：ram4[0x20000-0x23fff]，16k
- wakeMode：唤醒模式，详见唤醒模式枚举
- lpmPreCallback：lpm 休眠前回调函数，休眠前的准备动作在此回调函数中实现
- lpmPostCallback：lpm 唤醒后回调函数，唤醒后的恢复操作在此回调函数中实现

- 返回值：none

10.4. demo 说明

- 演示 lpm 睡眠到唤醒过程，关键代码说明如下：

唤醒标志位宏定义

```
#define WAKEUP_FLAG_HOST                (1<<0)
#define WAKEUP_FLAG_KEY_SCAN            (1<<1)
#define WAKEUP_FLAG_RTCOV               (1<<2)
#define WAKEUP_FLAG_RTC1                (1<<3)
#define WAKEUP_FLAG_RTC0                (1<<4)
#define WAKEUP_FLAG_GPIO                (1<<5)
#define WAKEUP_FLAG_WAKE                (1<<6)
#define WAKEUP_FLAG_RAM_CRC_ERR         (1<<7)
```

初始化系统和串口打印初始化

```
    SYSCTRL_ChipInit();
    printport_init();
```

唤醒标志位分析

```
wakeup_FlagAnalysis();
```

唤醒标志位函数：通过标志位判断唤醒源，并判断 ram 是否正确保电

```
void wakeup_FlagAnalysis(void)
{
    yc_log("wakeup_flag=%x",LpmRecoverStruct->wakeup_flag);
    if(LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_WAKE)
    {
        if(LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_KEY_SCAN)
        {
            yc_log("key scan wakeup");
        }
        if(LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_RTCOV)
        {
            yc_log("rtc over wakeup");
            RTC_WKConfig(OV_INTR,DISABLE);
        }
        if(LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_RTC0)
        {
            yc_log("rtc0 wakeup");
            RTC_WKConfig(ALARM0,DISABLE);
        }
        if(LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_RTC1)
        {
            yc_log("rtc1 wakeup");
            RTC_WKConfig(ALARM1,DISABLE);
        }
        if(LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_GPIO)
        {
            yc_log("gpio wakeup");

yc_log_buf("gpio_in",LpmRecoverStruct->gpio_in,sizeof(LpmRecoverStruct->gpio_in));
            for(int i=0;i<GPIO_NUM;i++)
            {
                uint8_t wake_cfg=((LPM_PUWAKEUP->gpio_wake_cfg[i/2]>>((i%2)*4))&0x0f);
                if(LpmRecoverStruct->gpio_in[i/8]&(1<<(i%8)))/*gpio i input high*/
                {
                    if(wake_cfg==SLEEP_HIGHWAKE ||
wake_cfg==SLEEP_PULLDOWN_HIGHWAKE)
                    {
                        yc_log("gpio%d high wakeup",i);
                    }
```

```
                    }
                    else/*gpio i input low*/
                    {
                        if(wake_cfg==SLEEP_LOWWAKE || wake_cfg==SLEEP_PULLUP_LOWWAKE)
                        {
                            yc_log("gpio%d low wakeup",i);
                        }
                    }
                }
            }
            if(LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_RAM_CRC_ERR)
            {
                yc_log("retention ram no keep");
            }
        }
}
```

led 灯闪烁，发送串口指令进入休眠，唤醒后分析唤醒标志位：

1、串口发送 ‘l’ ，进入 retention ram 区域和栈顶区域保电且唤醒从睡眠处继续往后运行模式。

2、串口发送 ‘s’ ，进入深睡不保电模式，功耗最低，支持定时唤醒和 IO 唤醒。

```
while(1)
    {
        GPIO_Config(GPIO_LED_PIN,(GPIO_ModeEnum)led_flag);

        if(UART_GetRxCount(PRINT_PORT)>0)
        {
            switch (UART_ReceiveData(PRINT_PORT))
            {
                case 'l':
                    yc_log("lpm sleep");

LPM_Sleep(RTC_WAKEUP_TIME,SLEEP_MODE_DEEP,0x11,WAKEUP_FROM_SLEEP,LPM_prepareActionBeforeSleep,LPM_postActionWakeup);
                    yc_log("wakeup");
                    wakeup_FlagAnalysis();
                    break;

                case 's':
                    yc_log("deep sleep(no retention)");
```

```
18
LPM_Sleep(RTC_WAKEUP_TIME,SLEEP_MODE_DEEP,0x00,WAKEUP_FROM_MAIN,LPM_prepareActio
nBeforeSleep,LPM_postActionWakeup);
19                              yc_log("deep sleep wakeup");
20                              wakeup_FlagAnalysis();
21                              break;
22
23                          default:
24                              break;
25                  }
26              }
27          }
```

lpm 休眠前回调函数：配置部分 GPIO 休眠保持，gpio 引脚唤醒模式，关闭中断，休眠 dvdd 电压和 VDDlpm 电压配置，关闭串口打印

```
1  void LPM_prepareActionBeforeSleep(LPM_ModeEnum sleepLevel)
2  {
3          // gpio latch config
4          GPIO_Config(GPIO_PWM0_PIN,GPIO_MODE_PULL_DOWN);
5          LPM_SetGpioLatch(GPIO_PWM0_PIN,LATCH_PD);
6
7          GPIO_Config(GPIO_PWM1_PIN,GPIO_MODE_PULL_UP);
8          LPM_SetGpioLatch(GPIO_PWM1_PIN,LATCH_PU);
9
10         GPIO_Config(GPIO_PWM2_PIN,GPIO_MODE_OUT_LOW);
11         LPM_SetGpioLatch(GPIO_PWM2_PIN,LATCH_OUTPUT_LOW);
12
13         GPIO_Config(GPIO_PWM3_PIN,GPIO_MODE_OUT_HIGH);
14         LPM_SetGpioLatch(GPIO_PWM3_PIN,LATCH_OUTPUT_HIGH);
15         //gpio wakeup config
16         GPIO_Config(KEY1_PIN,GPIO_MODE_PULL_UP);
17         GPIO_SetWakeupType(KEY1_PIN,SLEEP_LOWWAKE);
18
19         /*
20         为了降低睡眠功耗，建议将睡眠后不使用的外设关闭
21         并根据外围电路将睡眠期间不使用的 GPIO 配置成 GPIO_MODE_HI_Z
22         */
23
24        __disable_irq();
25        /*保电电压配置*/
26        LpmRecoverStruct->rg_lpm_ldo_dig_retmem_vsel.bit.sleep=5;
27        LpmRecoverStruct->rg_lpm_ldo_dig_lpm_vosel.bit.sleep=0;
28        printport_deinit();
```

```
}
```

唤醒后回调函数：初始化系统和串口打印初始化

```
void LPM_postActionWakeup(LPM_ModeEnum sleepLevel)
{
    SYSCTRL_ChipInit();
    printport_init();
}
```

11. CAN

11.1. 宏定义、枚举及结构体说明

11.1.1. CAN 帧类型宏定义

```
#define CAN_ID_STD          ((uint32_t)0x00000000)          /*!< 标准帧 */
#define CAN_ID_EXT          ((uint32_t)0x00000001)          /*!< 扩展帧 */
#define CAN_RTR_DATA        ((uint32_t)0x00000000)          /*!< 数据帧 */
#define CAN_RTR_REMOTE      ((uint32_t)0x00000001)          /*!< 远程帧 */
```

11.1.2. CAN 模式宏定义

```
#define CAN_NORMAL                  (0)         /*!< 普通模式 */
#define CAN_SELF_RX_MODE            (1)         /*!< RX 自收模式 */
#define CAN_LOOPBACK_MODE           (2)         /*!< 回环模式 */
#define CAN_LISTEN_ONLY_MODE        (3)         /*!< 只听模式 */
```

11.1.3. CAN 中断类型

```
#define CAN_IT_RC               ((uint32_t)0x00000001)      /* 接收完成 */
#define CAN_IT_RXVLD            ((uint32_t)0x00000002)      /* rx fifo 非空 */
#define CAN_IT_RXFULL           ((uint32_t)0x00000004)      /* rx fifo 已满 */
#define CAN_IT_RXOVER           ((uint32_t)0x00000008)      /* rx fifo 溢出 */
#define CAN_IT_TC               ((uint32_t)0x00000010)      /* 发送完成，
                                                               即使被取消也会置 1 */
```

11.1.4. CAN 标志位宏定义

```
#define CAN_FLAG                                ((uint32_t)0x00000001)          /*!< Once a
frame is transmitting, buff busy turns '1', and user should not change the buff     */
#define CAN_FLAG_TX_BUSY                        ((uint32_t)0x00000001)          /*!< Once a
frame is transmitting, buff busy turns '1', and user should not change the buff     */
#define CAN_FLAG_TX_PEND                        ((uint32_t)0x00000002)          /*!< a frame is
transmitting or wait to send    */
#define CAN_FLAG_RX_STU                         ((uint32_t)0x00000004)          /*!< A frame is
ongoing, and CAN now act as Receiver            */
#define CAN_FLAG_TX_STU                         ((uint32_t)0x00000008)          /*!< A frame is
ongoing, and CAN now act as Transmitter */
```

```
#define CAN_FLAG_TX_VAL                 ((uint32_t)0x00000010)          /*!< The current value of TX data line.     */
#define CAN_FLAG_RX_VAL                 ((uint32_t)0x00000020)          /*!< The current value of RX data line. */
#define CAN_FLAG_SAM_BIT                ((uint32_t)0x00000040)          /*!< Current sample value.              */
#define CAN_FLAG_IDE_STU                ((uint32_t)0x00000080)          /*!< The controller is idle.*/
#define CAN_FLAG_ERR_PASSIVE            ((uint32_t)0x00000100)          /*!< The current value of error_passive.     */
#define CAN_FLAG_BUF_OFF                ((uint32_t)0x00000200)          /*!< The current value of bus_off. */
#define CAN_FLAG_ERR_FRAME              ((uint32_t)0x00000400)          /*!< Error frames are currently being sent.             */
#define CAN_FLAG_OVERLOAD_FRAME         ((uint32_t)0x00000800)          /*!< Overload frames are currently being sent.*/
```

11.1.5. CAN 过滤模式

```
#define CAN_FILTERMODE_IDMASK     ((uint8_t)0x00)     /*!< id 掩码模式 */
#define CAN_FILTERMODE_IDLIST     ((uint8_t)0x01)     /*!< 标识帧列表模式 */
```

11.1.6. CAN 过滤 bit 数

```
#define CAN_FILTERSCALE_16BIT     ((uint8_t)0x00) /*!< 2 个 16-bit 过滤 */
#define CAN_FILTERSCALE_32BIT     ((uint8_t)0x01) /*!< 1 个 32-bit 过滤 */
```

11.1.7. CAN 初始化结构体

```
typedef struct
{
    uint16_t prescaler; /*!< 这 bit 定义了时间量长度
                             Tq = (BRP[9:0]+1)*Tclk_can */

    uint8_t mode;/*!< 定义了 CAN 模式
                      参数定义了 CAN_NORMAL/CAN_SELF_RX_MODE/CAN_LOOPBACK_MODE/CAN_LISTEN_ONLY_MODE
                      详见 CAN 模式宏定义 */

    uint8_t sjw; /*!< 指定 CAN 硬件在进行重新同步时允许对位进行延长或缩短的最大时间量。*/

    uint8_t bs1; /*!< 指定时间量的数量写入 TSEG1 */
```

```
13
14      uint8_t bs2; /*!< 指定时间量的数量写入 TSEG2 */
15      uint8_t sam; /*!< 采样模式 0: 采样 1 次 1: 采样 3 次 */
16  } CAN_InitStruct;
```

11.1.8. CAN 过滤器初始化结构体

```
1   typedef struct
2   {
3       uint8_t filterMode;                 /*!< 初始化过滤模式
4                                               参数参考@CAN 过滤模式 */
5
6       uint8_t filterScale;                /*!< 过滤器 bit 数 */
7
8       FunctionalState filterActivation;   /*!< 使能或失能过滤器 */
9
10      uint16_t filterIdHigh;              /*!< 指定过滤器的识别号码（对于 32Bit 而言，为高
                                        字节部分）
11                                              (对 16 位而言是第 1 个）。
12                                              此参数的取值范围在 0x0000 到 0xFFFF 之间 */
13
14
15      uint16_t filterIdLow;               /*!< 指定过滤器的识别号码（对于 32Bit 而言，为低
                                        字节部分）
16                                              (对 16 位而言是第 2 个）。
17                                              此参数的取值范围在 0x0000 到 0xFFFF 之
                                        间 */
18
19      uint16_t filterMaskIdHigh;          /*!< 指定滤波器掩码编号或标识编号。（对于 32Bit
                                        而言，为高字节部分）
20                                              (对 16 位而言是第 1 个）。
21                                              此参数的取值范围在 0x0000 到 0xFFFF 之
                                        间 */
22
23      uint16_t filterMaskIdLow;           /*!< 指定滤波器掩码编号或标识编号。（对于 32Bit
                                        而言，为低字节部分）
24                                              (对 16 位而言是第 2 个）。
25                                              此参数的取值范围在 0x0000 到 0xFFFF 之
                                        间 */
26
27      uint8_t filterIde;                  /*!< 指定要传输的消息的标识符类型。
28                                              此参数详见 CAN 帧类型宏定义 */
29
30      uint8_t filterRtr;                  /*!< 指定将要传输的消息的帧类型。
```

```
                                        此参数详见 CAN 帧类型宏定义  */
}CAN_FilterStruct;
```

11.1.9. Tx 信息结构体

```
typedef struct
{
    uint32_t stdId;             /*!< 指定标准帧 id
                                    参数值在 0 到 0x7FF 之间  */

    uint32_t extId;             /*!< 指定扩展帧 id
                                    参数值在 0 到 0x1FFFFFFF 之间  */

    uint8_t ide;                /*!< 指定将要传输的消息的 id 类型
                                 0 为标准帧 1 为扩展帧  */

    uint8_t rtr;                /*!< 指定将要传输的消息的帧类型
                                 0 为数据帧 1 为远程帧  */

    uint8_t dlc;                /*!< 指定传输信息长度
                                 参数值在 0 到 8 之间  */

    uint8_t data[8];            /*!< 传输数据
                                    参数值在 0 到 0xFF 之间  */
} CAN_TxMsgStruct;
```

11.1.10. Rx 信息结构体

```
typedef struct
{
    uint32_t stdId;             /*!< 接收标准帧 id
                                    参数值在 0 到 0x7FF 之间  */

    uint32_t extId;             /*!< 接收扩展帧 id
                                    参数值在 0 到 0x1FFFFFFF 之间  */

    uint8_t ide;                /*!< 接收的消息的 id 类型
                                    0 为标准帧 1 为扩展帧  */

    uint8_t rtr;                /*!< 接收的消息的帧类型
                                    0 为数据帧 1 为远程帧  */

    uint8_t dlc;                /*!< 接收信息长度
```

```
                                    参数值在 0 到 8 之间 */

    uint8_t data[8];          /*!< 接收数据
                                    参数值在 0 到 0xFF 之间 */

    uint8_t fmi;              /*!< reserved */
} CAN_RxMsgStruct;
```

11.2. CAN 操作接口汇总

函数类型	函数名	说明
CAN 相关接口	CAN_Init	CAN 初始化
	CAN_DeInit	清除 CAN 相关配置
	CAN_FilterInit	CAN 过滤初始化
	CAN_SendData	CAN 发送数据
	CAN_GetFlagStatus	获取 CAN 标志位状态
	CAN_MessagePending	获取 CAN 是否有剩余数据
	CAN_ReceiveData	CAN 接收数据
	CAN_SetIrq	CAN 设置中断
	CAN_GetIrqStatus	CAN 获取中断状态
	CAN_ClearIrq	清除 CAN 中断状态
	CAN_Sleep	CAN 睡眠使能
	CAN_WakeUp	CAN 唤醒

11.3. 接口说明

11.3.1. uint8_t CAN_Init(CAN_Type *canx, CAN_InitStruct *initStruct)

- 描述：CAN 初始化
- 参数：
  - canx：CAN
  - initStruct：初始化结构体，详见 CAN 初始化结构体
- 返回值：
  - 初始化状态：
    - 1：成功
    - 0：失败

11.3.2. void CAN_DeInit(CAN_Type *canx)

- 描述：清除 CAN 相关配置
- 参数：
  - canx：CAN
- 返回值：none

11.3.3. void CAN_FilterInit(CAN_Type *canx, CAN_FilterInitStruct *filterInitStruct)

- 描述：CAN 过滤器初始化
- 参数：
  - canx：CAN
  - filterInitStruct：过滤器初始化，详见 CAN 过滤器初始化结构体
- 返回值：none

11.3.4. uint8_t CAN_SendData(CAN_Type *canx, CAN_TxMsgStruct *txMessage)

- 描述：CAN 发送数据
- 参数：
  - canx：CAN
  - txMessage：tx 信息结构体，详见 tx 信息结构体
- 返回值：
  - 错误状态 ：
    - 0：成功
    - 1：失败

11.3.5. FlagStatus CAN_GetFlagStatus(CAN_Type *canx, uint32_t canFlag)

- 描述：CAN 标志位状态
- 参数：
  - canx：CAN
  - canFlag：CAN 标志位，详见 CAN 标志位宏定义
- 返回值：
  - 标志位状态：
    - 1：置起
    - 0：未置起

11.3.6. uint8_t CAN_MessagePending(CAN_Type *canx)

- 描述：CAN rx 是否有剩余数据

- 参数：
  - canx：CAN
- 返回值：
  - rx 剩余数据状态：
    - 1：有剩余
    - 0：无剩余

11.3.7. uint8_t CAN_ReceiveData(CAN_Type *canx, CAN_RxMsgStruct *rxMessage)

- 描述：CAN 接收数据
- 参数：
  - canx：CAN
  - rxMessage：rx 数据，详见 rx 数据结构体
- 返回值：
  - 接收状态：
    - 1：成功
    - 0：失败

11.3.8. void CAN_SetIrq(CAN_Type* canx, uint32_t canIrq, FunctionalState newState)

- 描述：CAN 中断配置
- 参数：
  - canx：CAN
  - canIrq：CAN 中断类型，详见 CAN 中断类型宏定义

- 返回值：none

11.3.9. uint8_t CAN_GetIrqStatus(CAN_Type* canx, uint32_t canIrq)

- 描述：获取 CAN 中断状态
- 参数：
  - canx：CAN
  - canIrq：目标 CAN 中断类型，详见 CAN 中断类型宏定义
- 返回值：
  - 目标中断状态标志位：
    - 1：置起
    - 0：未置起

11.3.10. void CAN_ClearIrq(CAN_Type* canx, uint32_t canIrq)

- 描述：清除 CAN 目标中断状态
- 参数：
  - canx：CAN
  - canIrq：CAN 目标中断类型，详见 CAN 中断类型宏定义
- 返回值：none

11.3.11. void CAN_Sleep(CAN_Type* canx)

- 描述：CAN 睡眠使能
- 参数：

- canx：CAN

- 返回值：none

11.3.12. void CAN_WakeUp(CAN_Type* canx)

- 描述：CAN 睡眠状态唤醒使能

- 参数：

  - canx：CAN

- 返回值：none

11.4. demo 说明

11.4.1. loopback mode

- 演示 can 回环模式，关键代码说明如下：

can 模式控制宏定义

```
#define CAN_FLITER_MODE        (0)
#define CAN_INTR_MODE          (0)
```

初始化系统和串口打印初始化

```
    SYSCTRL_ChipInit();

    printport_init();
```

初始化 can 初始化结构体（can 模式和 can 波特率），选择回环模式

```
void Can_Confing()
{
    /* enable can/can1 clk */
    SYSCTRL_ClkEn(CLK_CAN, ENABLE);
    /* gpio set */
    GPIO_Config(GPIO_21, GPIO_MODE_CAN_RX);
    GPIO_Config(GPIO_22, GPIO_MODE_CAN_TX);

    CAN_InitStruct CAN_InitStruct;
```

```
    /*
        Tq= Tcan*(CAN_Prescaler+2)
        Tbit = Tq+(bs1 + 1)*Tq+(bs2 + 1)*Tq
    */
    CAN_InitStruct.prescaler = 8;
    CAN_InitStruct.bs1 = 6;
    CAN_InitStruct.bs2 = 3;
    CAN_InitStruct.sjw = 2;
    CAN_InitStruct.sam = 0;
    CAN_InitStruct.mode = CAN_NORMAL;
    CAN_Init(CAN, &CAN_InitStruct);
}
```

设置发送帧类型，帧 id，帧数据

```
void Can_SendMsgSet(void)
{
    TxMessage.extId = CAN_EXID;
    TxMessage.ide = 0x01;
    TxMessage.rtr = CAN_RTR_DATA;
    TxMessage.dlc = 0x08;
    for (uint8_t i = 0; i < TxMessage.dlc; i++)
        TxMessage.data[i] = '0' + i;
}
```

循环进行自发自收

```
    while (1)
    {
        CAN_SendData(CAN, &TxMessage);
        CAN_MsgInfoPrint(&TxMessage, NULL);
        for(uint8_t i = 0; i < 8; i++)
        {
            TxMessage.data[i] = TxMessage.data[i] + 1;
        }

        if(TxMessage.data[7] == 127)
        {
            for (uint8_t i = 0; i < TxMessage.dlc; i++)
            TxMessage.data[i] = '0' + i;
        }
        delay_ms(1000);

        if(CAN_ReceiveData(CAN, &RxMessage))
        {
```

```
            CAN_MsgInfoPrint(NULL, &RxMessage);
        }
    }
```

11.4.2. normal mode

- 演示 can 普通模式收发数据，关键代码说明如下：

初始化系统和串口打印初始化

```
    SYSCTRL_ChipInit();

    printport_init();
```

初始化 can 初始化结构体（can 模式和 can 波特率），波特率配置为 400k，模式为普通模式

```
void Can_Confing()
{
    /* enable can/can1 clk */
    SYSCTRL_ClkEn(CLK_CAN, ENABLE);
    /* gpio set */
    GPIO_Config(GPIO_21, GPIO_MODE_CAN_RX);
    GPIO_Config(GPIO_22, GPIO_MODE_CAN_TX);

    CAN_InitStruct CAN_InitStruct;
    /*
        Tq= Tcan*(CAN_Prescaler+2)
        Tbit = Tq+(bs1 + 1)*Tq+(bs2 + 1)*Tq
    */
    CAN_InitStruct.prescaler = 8;
    CAN_InitStruct.bs1 = 6;
    CAN_InitStruct.bs2 = 3;
    CAN_InitStruct.sjw = 2;
    CAN_InitStruct.sam = 0;
    CAN_InitStruct.mode = CAN_NORMAL;
    CAN_Init(CAN, &CAN_InitStruct);
}
```

发送信息初始化

```
void Can_SendMsgSet(void)
{
    TxMessage.extId = 0x01;
    TxMessage.ide = CAN_EXID;              /* ide 位的作用是确定是扩展帧(1)还是标准帧(0) */
```

```
    TxMessage.rtr = CAN_RTR_DATA;
    TxMessage.dlc = 0x08;
    for (uint8_t i = 0; i < TxMessage.dlc; i++)
        TxMessage.data[i] = '0' + i;
}
```

过滤器初始化，通过 CAN_FLITER_MODE 宏控制是否需要过滤

```
    #if CAN_FLITER_MODE
    /* 过滤器的配置，32bit 的列表模式下，只接受 ID 为 0x12 和 0x34 的两个标准帧 */
    CAN_FilterConfig(CAN_FILTER0, CAN_ID_STD, CAN_FILTERMODE_IDLIST, CAN_DATA, ENABLE, CAN_FILTERSCALE_32BIT, 0x0000, 0x0012, 0x0000, 0x0034);
    /* 过滤器的配置，32bit 的列表模式下，只接受 ID 为 0x7fc、0x7fd、0x7fe、0x7ff 的四个标准帧 */
    CAN_FilterConfig(CAN_FILTER1, CAN_ID_STD, CAN_FILTERMODE_IDMASK, CAN_DATA, ENABLE, CAN_FILTERSCALE_32BIT, 0x0000, 0x07ff, 0x0000, 0xfffc);
    /* 过滤器的配置，16bit 的列表模式下，只接受的四个标准帧 */
    CAN_FilterConfig(CAN_FILTER2, CAN_ID_STD, CAN_FILTERMODE_IDLIST, CAN_DATA, ENABLE, CAN_FILTERSCALE_16BIT, 0x0023, 0x0022, 0x0056, 0x0078);
    /* 过滤器的配置，16bit 的掩码模式下，只接受 ID 为 0x1a、0x1b、0x2a、0x2b 的四个标准帧 */
    CAN_FilterConfig(CAN_FILTER3, CAN_ID_STD, CAN_FILTERMODE_IDMASK, CAN_DATA, ENABLE, CAN_FILTERSCALE_16BIT, 0x001a, 0x00fc, 0x002a, 0x00fc);

    CAN_FilterInit(CAN, &filterInitStruct);
    #endif
```

can 过滤信息函数，将数据写入 CAN_FilterConfig 结构体，再由 CAN_FilterInit 进行初始化

```
 void CAN_FilterConfig(uint8_t group, uint8_t ide,uint8_t filterMode,uint8_t rtr,FunctionalState filterActivation,uint8_t filterScale, uint16_t filterIdLow, uint16_t filterIdHigh, uint16_t filterMaskIdLow, uint16_t filterMaskIdHigh)
{
    filterInitStruct.filterStruct[group].filterIde = ide;
    filterInitStruct.filterStruct[group].filterMode = filterMode;
    filterInitStruct.filterStruct[group].filterRtr = rtr;
    filterInitStruct.filterStruct[group].filterScale = filterScale;
    filterInitStruct.filterStruct[group].filterActivation = filterActivation;
    filterInitStruct.filterStruct[group].filterIdLow = filterIdLow;
    filterInitStruct.filterStruct[group].filterIdHigh = filterIdHigh;
    filterInitStruct.filterStruct[group].filterMaskIdLow = filterMaskIdLow;
    filterInitStruct.filterStruct[group].filterMaskIdHigh = filterMaskIdHigh;
}
```

通过 CAN_INTR_MODE 宏控制中断是否开启

```
    #if CAN_INTR_MODE
```

```
    CAN_SetIrq(CAN, CAN_IT_RXVLD|CAN_IT_TC, ENABLE);
    PLIC_SetPriority(CAN_IRQn, 2);
    PLIC_EnableIRQ(CAN_IRQn);
    #endif
```

按下按键，发送 can 扩展帧，每次发送完 id 增加，若中断未开启则轮询接收 can 数据

```
while (1)
    {
        if(GPIO_ReadDataBit(KEY1_PIN)==0)
        {
            if( CAN_SendData(CAN,&TxMessage)==0)
            #if CAN_INTR_MODE == 0
            {
                CAN_MsgInfoPrint(&TxMessage,NULL);
                TxMessage.extId = (TxMessage.extId+1)&0X1FFFFFFF;
            }
            #endif
            delay_ms(10);
        }
        #if CAN_INTR_MODE == 0
        if(CAN_ReceiveData(CAN, &RxMessage))
        {
            CAN_MsgInfoPrint(NULL, &RxMessage);
            memset(&RxMessage, 0, sizeof(RxMessage));
        }
        #endif
    }
```

若中断开启，则在数据发送成功后打印发送信息，在接收数据成功后打印数据信息

```
void CAN_IRQHandler(void)
{
    if(CAN_GetIrqStatus(CAN, CAN_IT_RXVLD))
    {
        yc_log("CAN Rx Fifo Not Empty Irqhandler!\n");
        if(CAN_ReceiveData(CAN, &RxMessage))
        {
            CAN_MsgInfoPrint(NULL, &RxMessage);
            memset(&RxMessage, 0, sizeof(RxMessage));
        }
        CAN_ClearIrq(CAN, CAN_IT_RXVLD);
    }

    if(CAN_GetIrqStatus(CAN, CAN_IT_TC))
```

```
    {
        yc_log("CAN Tx Complete Irqhandler!\n");
        CAN_MsgInfoPrint(&TxMessage, NULL);
        TxMessage.extId = (TxMessage.extId+1)&0X1FFFFFFF;
        CAN_ClearIrq(CAN, CAN_IT_TC);
    }
}
```

12. CHARGE

12.1. 宏定义、枚举及结构体说明

12.1.1. 充电电源枚举

```
typedef enum
{
    POWER_CHGR_IN,
    POWER_VBAT
}CHARGE_SysPowerEnum;
```

POWER_CHGR_IN：插入 USB 时系统电源自动切换为 USB 供电，拔出 USB 切换为锂电池供电

POWER_VBAT：系统电源固定由锂电池供电

12.1.2. 充电中断枚举

```
typedef enum
{
    CHGR_IN_DET_AON     = 0x1,
    CHGR_STATE_IND      = 0x2,
    CHGR_UVLO_OK_AON    = 0x4,
    CHGR_PGOOD          = 0x8,
    CHGR_ALL_IRQ        = 0xf

}CHARGE_IrqEnum;
```

12.1.3. 充电中断模式枚举

```
typedef enum
{
    CHGR_IRQ_MODE_1,
    CHGR_IRQ_MODE_0
}CHARGE_IrqModeEnum;
```

12.1.4. 充电电流枚举

```
typedef enum
{
    CHARGE_CURRENT_30MA,
    CHARGE_CURRENT_40MA,
    CHARGE_CURRENT_50MA,
    CHARGE_CURRENT_60MA,
    CHARGE_CURRENT_70MA,
    CHARGE_CURRENT_80MA,
    CHARGE_CURRENT_90MA,
    CHARGE_CURRENT_100MA,
    CHARGE_CURRENT_110MA,
    CHARGE_CURRENT_120MA,
    CHARGE_CURRENT_130MA,
    CHARGE_CURRENT_140MA,
    CHARGE_CURRENT_150MA,
    CHARGE_CURRENT_160MA,
    CHARGE_CURRENT_170MA,
    CHARGE_CURRENT_180MA
}CHARGE_CurrentEnum;
```

12.2. CHARGE 操作接口汇总

函数类型	函数名	说明
CHARGE 相关接口	CHARGE_VolSel	充电电压配置
	CHARGE_CurSel	充电电流配置
	CHARGE_CcCvCtrl	充电模式配置
	CHARGE_SystemPowerSelect	充电电源配置
	CHARGE_Cmd	充电模块使能
	CHARGE_InsertDet	USB 插入检测
	CHARGE_State	充电状态检测
	CHARGE_GetIrqStatus	获取充电中断状态

	CHARGE_SetIrq	充电中断使能配置
	CHARGE_SetIrqMode	充电中断模式配置
	CHARGE_GetIrqMode	获取充电中断模式

12.3. 接口说明

12.3.1. void CHARGE_VolSel(uint8_t CutOffVol, int8_t RechgrVol)

- 描述：充电电压配置
- 参数：
  - CutOffVol：截止电压，步进 53mv，一般配置为 otp 中的校准值（4.2V）
  - RechgrVol：复充电压，步进 53mv，一般配置为 otp 中的校准值-2（4.1V）
- 返回值：none

12.3.2. void CHARGE_CurSel(CHARGE_CurrentEnum ChargCurtSel, uint32_t TricCurtSel, uint32_t CutOffCurtSel)

- 描述：充电电流配置
- 参数：
  - ChargCurtSel：充电电流，详见充电电流枚举
  - TricCurtSel：当 VBAT<3V 时，涓流电流配置
    - 0：10%充电电流
    - 1：20%充电电流
    - 2：30%充电电流

- 3：40%充电电流

- CutOffCurtSel：截止电流，当当前充电电流小于或等于所设定的截止电流时停止充电
  - 0：10%充电电流
  - 1：20%充电电流
  - 2：30%充电电流
  - 3：40%充电电流

- 返回值：none

12.3.3. bool CHARGE_CcCvCtrl(uint8_t ChargCcMod, uint8_t ChargCvMod)

- 描述：充电模式配置
- 参数：
  - ChargCcMod：是否强制切换到恒流充电
    - 0：正常模式
    - 1：强制使能
  - ChargCvMod：是否强制切换到恒压模式
    - 0：正常模式
    - 1：强制使能
- 返回值：
  - 配置成功值：
    - 1：成功
    - 0：失败

12.3.4. void CHARGE_SystemPowerSelect(CHARGE_SysPowerEnum powerSource)

- 描述：充电电源配置
- 参数：
  - powerSource：详见充电电源枚举
- 返回值：none

12.3.5. void CHARGE_Cmd(FunctionalState newState)

- 描述：充电模块使能
- 参数：
  - newState：
    - ENABLE：使能充电模块
    - DISABLE：失能充电模块
- 返回值：none

12.3.6. bool CHARGE_InsertDet(void)

- 描述：USB 插入检测
- 参数：none
- 返回值：
  - USB 状态：
    - 1：已插入
    - 0：未插入

12.3.7. bool CHARGE_State(void)

- 描述：充电状态检测
- 参数：none
- 返回值：
  - USB 状态：
    - 1：正在充电
    - 0：未充电

12.3.8. void CHARGE_SetIrq(CHARGE_InterEnum chgr_intrtype, FunctionalState newState)

- 描述：充电中断使能配置
- 参数：
  - chgr_intrtype：充电中断类型，详见充电中断枚举
  - newState：
    - ENABLE：使能
    - DISABLE：失能
- 返回值：none

12.3.9. void CHARGE_SetIrqMode(CHARGE_InterEnum chgr_intrtype, CHARGE_IrqModeEnum mode)

- 描述：充电中断模式配置
- 参数：
  - chgr_intrtype：充电中断类型，详见充电中断枚举
  - mode：充电中断模式：

- 0：状态为 1 时触发中断
- 1：状态为 0 时触发中断

- 返回值：none

12.3.10. uint8_t CHARGE_GetIrqStatus(CHARGE_InterEnum chgr_intrtype)

- 描述：获取充电中断状态
- 参数：
  - chgr_intrtype：充电中断类型。详见充电中断枚举
- 返回值：
  - 目标充电状态标志位：
    - 1：已置起
    - 0：未置起

12.3.11. uint8_t CHARGE_GetIrqMode(void)

- 描述：获取充电中断模式
- 参数：none
- 返回值：
  - 充电中断模式标志位：各 bit 位为 0 中断模式是状态为 1 时触发中断，bit 位为 1 则中断模式是状态为 0 时触发中断，下列为各 bit 位代表状态：
    - bit0：CHGR_IN_DET_AON
    - bit1：CHGR_STATE_IND
    - bit2：CHGR_UVLO_OK_AON
    - bit 3：CHGR_PGOOD

12.4. demo 说明

- 演示轮询获取充电状态和 usb 插入状态，并通过中断获取哪个事件触发中断，关键代码说明如下：

初始化系统和串口打印初始化

```
    SYSCTRL_ChipInit();
    printport_init();
```

充电截止电压获取

```
  OTP_Read(OTP_CHGR_CUT_OFF_VOL_ADDR,(uint8_t*)&chgr_cut_off_vol,1);/* 读取 otp 中的截止电压校准值 */
    if((chgr_cut_off_vol&0x80) == 0)
    {
        chgr_cut_off_vol = 8;
    }
    else
    {
        chgr_cut_off_vol = (chgr_cut_off_vol&0x3f);
    }
    yc_log("chgr_cut_off_vol=%d",chgr_cut_off_vol);
```

配置充电电压和电流

```
    CHARGE_VolSel(chgr_cut_off_vol,chgr_cut_off_vol-2);
    CHARGE_CurSel(CHARGE_CURRENT_100MA,2,1);
```

中断使能和中断模式配置

```
    CHGR_SetIrqMode(ALL_CHGR_INTR,CHRG_EQ_0);
    CHGR_SetIrq(ALL_CHGR_INTR,ENABLE);
    PLIC_EnableIRQ(CHGR_IRQn);
    PLIC_SetPriority(CHGR_IRQn,2);
```

轮询充电状态和 usb 插入检测

```
    while(1)
    {
        if(CHARGE_InsertDet())
        {
            yc_log("usb insert");
```

```
        }
        else
        {
            yc_log("usb not insert");
        }
        if(CHARGE_State())
        {
            yc_log("charging");
        }
        else
        {
            yc_log("not charge");
        }
        delay_ms(500);
    }
```

中断打印中断状态，判断为哪个事件触发中断，触发后更改该事件中断触发模式

```
void CHGR_IRQHandler(void)
{
    uint8_t status = CHARGE_GetIrqStatus(CHGR_ALL_IRQ);
    uint8_t mode = CHARGE_GetIrqMode();
    yc_log("status=0x%02x,mode=0x%02x",status,mode);
    if(status&CHGR_IN_DET_AON)
    {
        if(mode&CHGR_IN_DET_AON)
        {
            yc_log("CHGR_IN_DET_AON 1");
            CHARGE_SetIrqMode(CHGR_IN_DET_AON,CHGR_IRQ_MODE_0);
        }
        else
        {
            yc_log("CHGR_IN_DET_AON 0");
            CHARGE_SetIrqMode(CHGR_IN_DET_AON,CHGR_IRQ_MODE_1);
        }
    }
    if(status&CHGR_STATE_IND)
    {
        if(mode&CHGR_STATE_IND)
        {
            yc_log("CHGR_STATE_IND 1");
            CHARGE_SetIrqMode(CHGR_STATE_IND,CHGR_IRQ_MODE_0);
        }
        else
        {
```

```
            yc_log("CHGR_STATE_IND 0");
            CHARGE_SetIrqMode(CHGR_STATE_IND,CHGR_IRQ_MODE_1);
        }
    }
    if(status&CHGR_UVLO_OK_AON)
    {
        if(mode&CHGR_UVLO_OK_AON)
        {
            yc_log("CHGR_UVLO_OK_AON 1");
            CHARGE_SetIrqMode(CHGR_UVLO_OK_AON,CHGR_IRQ_MODE_0);
        }
        else
        {
            yc_log("CHGR_UVLO_OK_AON 0");
            CHARGE_SetIrqMode(CHGR_UVLO_OK_AON,CHGR_IRQ_MODE_1);
        }
    }
    if(status&CHGR_PGOOD)
    {
        if(mode&CHGR_PGOOD)
        {
            yc_log("CHGR_PGOOD 1");
            CHARGE_SetIrqMode(CHGR_PGOOD,CHGR_IRQ_MODE_0);
        }
        else
        {
            yc_log("CHGR_PGOOD 0");
            CHARGE_SetIrqMode(CHGR_PGOOD,CHGR_IRQ_MODE_1);
        }
    }
}
```

13. WDT

13.1. 宏定义、枚举及结构体说明

13.1.1. WDT 模式枚举

```
typedef enum
{
    RESET_MODE = 0,
    INTR_MODE
}ModeEnum;
```

13.1.2. WDT 初始化结构体

```
typedef struct
{
    ModeEnum mode;        //模式设置，详见@WDT 模式枚举
    uint8_t reload;     //WDT 超时时间设置，详见@WDT 超时时间宏定义
} WDT_InitStruct;
```

13.1.3. WDT 超时时间宏定义

```
#define WDT_TIMER_125MS              0x7F
#define WDT_TIMER_250MS              0x7e
#define WDT_TIMER_500MS              0x7c
#define WDT_TIMER_1_5S              0x74
#define WDT_TIMER_10S               0x30
#define WDT_TIMER_MAX                0x00 //16.384s
```

13.2. WDT 操作接口汇总

函数类型	函数名	说明
WDT 相关接口	WDT_Init	WDT 初始化
	WDT_SetReload	设置 WDT 重载值
	WDT_Enable	WDT 使能

	WDT_Disable	WDT 失能
	WDT_ClearIrqSatus	WDT 清除中断状态
	WDT_WfiHoldEn	进入 WFI 后暂停看门狗计数

13.3. 接口说明

13.3.1. void WDT_Init(WDT_InitStruct* initStruct)

- 描述：WDT 初始化
- 参数：
  - initStruct：wdt 初始化，详见 wdt 初始化结构体
- 返回值：none

13.3.2. void WDT_SetReload(uint8_t count)

- 描述：WDT 重新加载超时时间计数
- 参数：
  - count：重载计数值，取值范围 0x00~0xFF。

    超时时间为 ：（ 0x80000 - (count<<12) ）/ (32768)，32768 是 wdt 时钟。
- 返回值：none

13.3.3. void WDT_Enable(void)

- 描述：WDT 使能
- 参数：none
- 返回值：none

13.3.4. void WDT_Disable(void)

- 描述：WDT 失能
- 参数：none
- 返回值：none

13.3.5. void WDT_ClearIrqSatus(void)

- 描述：WDT 清除中断状态
- 参数：none
- 返回值：none

13.3.6. void WDT_WfiHoldEn(FunctionalState newState)

- 描述：进入 WFI 后暂停看门狗计数，即 WFI 期间不会触发看门狗超时
- 参数：
  - newState：
    - ENABLE：使能
    - DISABLE：失能
- 返回值：none

13.4. demo 说明

- 演示 WDT 软复位和中断功能，关键代码说明如下：

初始化系统和串口打印初始化

```
SYSCTRL_ChipInit();
printport_init();
```

判断系统是否为 wdt 软复位并清除标志位

```
if(SYSCTRL_GetSysEvent()&(SYS_EVENT_WDT_RESET))
{
    yc_log("wdt reset");
    SYSCTRL_ClearSysEvent(SYS_EVENT_WDT_RESET);
}
```

WDT 初始化

```
void WDt_Configuration(void)
{
    WDT_InitStruct initstruct;
    initstruct.mode = WDT_MODE;
    initstruct.reload = WDT_COUNTER;
    WDT_Init(&initstruct);
}
```

判断中断/软复位模式，若为中断模式则开启中断，再通过宏定义判断是否为 vector 中断

```
if(WDT_MODE==INTR_MODE)
{
    #ifdef VECTOR_IRQ_ENBALE
    VECTOR_EnableIRQ(VECTOR_WDT_IRQn);
    #else
    PLIC_SetPriority(WDT_IRQn,1);
    PLIC_EnableIRQ(WDT_IRQn);
    #endif
}
```

重载次数小于 3，测试重载超时时间

```
while (1)
{
    if(wdt_reload_cnt<3)
    {
        delay_ms(1000);
        WDT_SetReload(WDT_COUNTER);
        yc_log("wdt reload");
        wdt_reload_cnt++;
    }
}
```

中断处理，到了超时时间则触发中断进行串口打印，重置重载次数

```
#ifdef VECTOR_IRQ_ENBALE
```

```
_interrupt void VECTOR_WDT_IRQ(void)
#else
void WDT_IRQHandler(void)
{
    yc_log("WDT interrupt");
    wdt_reload_cnt = 0;
    WDT_ClearIrqSatus();
}
#endif
```

14. NEC

14.1. 宏定义、枚举及结构体说明

14.1.1. NEC 初始化结构体

```
typedef struct
{
    uint32_t            bufSaddr;       /*!< nec 接收 dma 开始地址. */
    uint32_t            bufSize;        /*!< nec 接收 dma buf 大小. */
    uint32_t            clk;            /*!< nec 模块 时钟频率. */
    NEC_ModeEnum        mode;
} NEC_InitStruct;
```

14.1.2. NEC 中断类型

```
#define NEC_IRQ_DONE        (1<<0)    //NEC 完成中断
#define NEC_IRQ_HALF_DONE   (1<<1)    //NEC 半完成中断
```

14.1.3. NEC 增益电平枚举

```
typedef enum
{
    GAIN_LOWEST,
    GAIN_LOW,
    GAIN_HIGH,
    GAIN_HIGHEST
} NEC_GainLevelEnum;
```

14.1.4. NEC 输入源枚举

```
typedef enum{
    NEC_FROM_GPIO,
    NEC_FROM_IR
} NEC_RxSrcEnum;
```

14.1.5. NEC 模式枚举

```
typedef enum
{
    NEC_TX_CARRIER_NONE,                /*!< nec tx,无载波. */
    NEC_TX_CARRIER_HIGH,                /*!< nec tx,高载波. */
    NEC_TX_CARRIER_LOW,                 /*!< nec tx,低载波. */
    NEC_RX                              /*!< nec rx. */
} NEC_ModeEnum;
```

14.2. NEC 操作接口汇总

函数类型	函数名	说明
NEC 相关接口	NEC_Init	NEC 初始化
	NEC_DeInit	NEC 反初始化
	NEC_SetClk	NEC 时钟配置
	NEC_SetMode	NEC 模式配置
	NEC_RxSrcSel	Rx 数据源选择
	NEC_SetRxGainLevel	NEC 配置 RX 增益电平
	NEC_PtrInit	NEC 指针初始化
	NEC_Cmd	NEC 使能
	NEC_SetIrq	NEC 中断配置
	NEC_ClearIrq	NEC 中断清除
	NEC_GetIrqStatus	NEC 获取中断状态
	NEC_WriteData	NEC 写数据
	NEC_ReadData	读取 NEC 数据
	NEC_GetDataCount	获取 NEC 数据长度

14.3. 接口说明

14.3.1. void NEC_Init(NEC_InitStruct *initStruct)

- 描述：NEC 初始化
- 参数：
  - initStruct：NEC 初始化，详见 NEC 初始化结构体
- 返回值：none

14.3.2. void NEC_DeInit(void)

- 描述：NEC 反初始化
- 参数：none
- 返回值：none

14.3.3. void NEC_SetClk(uint32_t clk)

- 描述：NEC 时钟配置
- 参数：
  - clk：NEC 时钟频率，单位为 Hz
- 返回值：none

14.3.4. void NEC_SetMode(NEC_ModeEnum mode)

- 描述：NEC 模式配置
- 参数：
  - mode：NEC 模式，详见 NEC 模式枚举
- 返回值：none

14.3.5. void NEC_RxSrcSel(NEC_RxSrcEnum necSource)

- 描述：Rx 数据源选择
- 参数：
  - necSource：NEC 源，详见 NEC 输入源枚举
- 返回值：none

14.3.6. void NEC_SetRxGainLevel(NEC_GainLevelEnum gainLevel)

- 描述：NEC 配置 RX 增益电平
- 参数：
  - gainLevel：增益电平能力，详见 NEC 增益电平枚举
- 返回值：none

14.3.7. void NEC_PtrInit(void)

- 描述：NEC 指针初始化
- 参数：none
- 返回值：none

14.3.8. void NEC_Cmd(FunctionalState state)

- 描述：NEC 使能
- 参数：
  - state：
    - ENABLE：使能
    - DISABLE：失能

- 返回值：none

14.3.9. void NEC_SetIrq(uint8_t irqType,FunctionalState state)

- 描述：NEC 中断配置
- 参数：
  - irqType：中断类型，详见 NEC 中断类型
  - state：
    - ENABLE：使能
    - DISABLE：失能
- 返回值：none

14.3.10. void NEC_ClearIrq(uint8_t irqType)

- 描述：NEC 清除中断配置
- 参数：
  - irqType：中断类型，详见 NEC 中断类型
- 返回值：none

14.3.11. uint8_t NEC_GetIrqStatus(void)

- 描述：NEC 获取中断状态
- 参数：none
- 返回值：
  - 中断状态标志位：
    - 1：置起

- 0：未置起

14.3.12. int NEC_WriteData(uint16_t *buf,int len)

- 描述：NEC 写数据
- 参数：
  - buf：数据地址
  - len：数据长度（uint16_t 数据个数,不是字节数）
- 返回值：
  - 已写入数据的长度

14.3.13. uint16_t NEC_ReadData(void)

- 描述：NEC 读取数据
- 参数：none
- 返回值：
  - NEC 数据

14.3.14. uint32_t NEC_GetDataCount(void)

- 描述：获取 NEC 数据长度
- 参数：none
- 返回值：
  - NEC 数据长度（uint16_t 数据个数,不是字节数）
    - tx：未发送数据长度
    - rx：已接收数据长度

14.4. demo 说明

- 使用红外时，若发送或接收距离较短，请检查 IR_TX/RX 引脚上的限流电阻是否过大，推荐阻值为 10Ω

14.4.1. nec send demo

- 演示 nec 按下按键发送键值和重复码功能，关键代码说明如下：

NEC 参数宏定义

```
#define NEC_MODE                        NEC_TX_CARRIER_HIGH
#define NEC_CLK_US                      (3)
#define NEC_USE_EXTERNAL_DRIVE          (0)       /* 0:IO 直驱红外发射,1:IO+MOS 管驱动红外发射 */
#define NEC_CARRIER_DUTY_FREQ           (38000) /* 载波频率 */
#define NEC_SUB_FILL_DATA               (1)       /* 分包填充数据控制 */
#if (NEC_SUB_FILL_DATA)
#define NEC_DMA_BUF_SIZE                (36)
#else
#define NEC_DMA_BUF_SIZE                (72)
#endif

#define KEY_LONG_PRESS_TIME_MS          (300)    /* 300ms */

#define NEC_DATA_BIT_NUM                (1+4*8) /* 引导码+数据 */
#define NEC_REPEAT_INTERVAL_TIME_MS     (110)    /* 110ms */
#define NEC_BOOT_CODE_START_TIME        9000     /* 9ms */
#define NEC_BOOT_CODE_STOP_TIME         4500     /* 4.5ms */
#define NEC_DATA_START_TIME             560      /* 560us */
#define NEC_DATA_BIT0_STOP_TIME         560      /* 560us */
#define NEC_DATA_BIT1_STOP_TIME         1690     /* 1690us */
#define NEC_REPEAT_START_TIME           9000     /* 9ms */
#define NEC_REPEAT_STOP_TIME            2250     /* 2250us */
#define NEC_START_ERROR_VALUE           500      /* start time error value define */
#define NEC_STOP_ERROR_VALUE            300      /* stop time error value define */
#define NEC_ERROR_VALUE                 100      /* error value define */
```

初始化系统和串口打印初始化

```
    SYSCTRL_ChipInit();

```

```
    printport_init();
```

NEC 初始化，设置 systick，上拉按键 GPIO

```
   SYSTICK_Config(SYSTICK_ONE_MS*10);
    GPIO_Config(KEY1_PIN,GPIO_MODE_PULL_UP);
    yc_log("NEC send demo");
    NEC_TxInit();
```

NEC 初始化函数，配置 dma 地址和长度，时钟频率和模式，并配置 NEC 的 GPIO 管脚，若为载波模式，则会开启 TIM7 并配置占空比，最后使能 done 和 half 中断

```
void NEC_TxInit(void)
{
    NEC_InitStruct initStruct = {0};

    initStruct.bufSaddr =   (uint32_t)nec_dma_buf;
    initStruct.bufSize   = sizeof(nec_dma_buf);
    initStruct.clk         = NEC_CLK_US*1000000;
    initStruct.mode       = NEC_MODE;

    #if (NEC_USE_EXTERNAL_DRIVE)
    GPIO_Config(NEC_PIN, GPIO_MODE_HI_Z);/* 非 IO 直驱时可配任意 IO */
    #else
    GPIO_DsConfig(GPIO_0,ENABLE);/* IO 直驱时固定 IO0~3,且许将 IO 驱动能力配为大驱动 */
    GPIO_DsConfig(GPIO_1,ENABLE);
    GPIO_DsConfig(GPIO_2,ENABLE);
    GPIO_DsConfig(GPIO_3,ENABLE);
    GPIO_Config(GPIO_0, GPIO_MODE_HI_Z);
    GPIO_Config(GPIO_1, GPIO_MODE_HI_Z);
    GPIO_Config(GPIO_2, GPIO_MODE_HI_Z);
    GPIO_Config(GPIO_3, GPIO_MODE_HI_Z);
    #endif

    #if ((NEC_MODE==NEC_TX_CARRIER_HIGH) || (NEC_MODE==NEC_TX_CARRIER_LOW))
    uint32_t carrier_high_cnt = (((NEC_CLK_US*1000000+(NEC_CARRIER_DUTY_FREQ-1))/NEC_CARRIER_DUTY_FREQ)/3);/* 载波高定平 cnt,按 1/3 占空比算 */
    uint32_t carrier_low_cnt = (((NEC_CLK_US*1000000+(NEC_CARRIER_DUTY_FREQ-1))/NEC_CARRIER_DUTY_FREQ)-carrier_high_cnt);/* 载波低定平 cnt */
    _ASSERT(carrier_high_cnt<=0xffff && carrier_low_cnt<=0xffff);
    TIM_SetPwmPeriod(TIM7,carrier_low_cnt-1,carrier_high_cnt-1);/* 载波固定使用 TIM7 进行配置 */
```

```
    #endif

    NEC_Init(&initStruct);
    #if (NEC_IRQ_ENABLE==1 || NEC_SUB_FILL_DATA==1)
    PLIC_SetPriority(NEC_IRQn,1);
    PLIC_EnableIRQ(NEC_IRQn);
    #endif
}
```

判断按键是否按下，短按发送一次，长按则会发送重复码

```
 while(1)
{
    if(GPIO_ReadDataBit(KEY1_PIN)==0)
    {
        if(sended_key_value==false)
        {
            sended_key_value = true;
            nec_long_press = false;
            nec_send_bit_index=0;
            start_tick = SYSTICK_GetTick();
            if(NEC_Send(&nec_send_data,&nec_send_bit_index))
            {
                yc_log("nec send code ok");
            }
            else
            {
                yc_log("nec send code fail");
            }
        }
        else
        {
            if(nec_long_press == false)
            {
                nec_interval_time = KEY_LONG_PRESS_TIME_MS;
            }
            else
            {
                nec_interval_time = NEC_REPEAT_INTERVAL_TIME_MS;
            }
            if(SYSTICK_GetRelativeTime(start_tick)>=nec_interval_time)
            {
                nec_long_press = true;
                start_tick = SYSTICK_GetTick();
                if(NEC_SendRepeat())
```

```
                        {
                            yc_log("nec send Repeat ok");
                        }
                        else
                        {
                            yc_log("nec send Repeat fail");
                        }
                    }
                }
            }
            else
            {
                sended_key_value = false;
                nec_long_press = false;
            }
        }
}
```

NEC 发送函数：首次调用初始化模块指针，配置 GPIO 为 NEC 发送模式，并写入引导码，进入循环，直到 *bitIndex >= NEC_DATA_BIT_NUM，每写完一个 bit 就 (*bitIndex)++。写完所有位后，添加截止符（start + 0）并写入；如果是第一次调用，会启动 NEC 并启用中断。

```
bool NEC_Send(NEC_DataStruct *txData,int* bitIndex)
{
    uint16_t nec_bit_data[2];
    int start_index = *bitIndex;

    if(start_index == 0)
    {
        NEC_Cmd(DISABLE);
        NEC_PtrInit();
        #if (NEC_USE_EXTERNAL_DRIVE)
        GPIO_Config(NEC_PIN, GPIO_MODE_NEC_SEND);/* 非 IO 直驱时可配任意 IO */
        #else
        GPIO_Config(GPIO_0, GPIO_MODE_NEC_SEND);
        GPIO_Config(GPIO_1, GPIO_MODE_NEC_SEND);
        GPIO_Config(GPIO_2, GPIO_MODE_NEC_SEND);
        GPIO_Config(GPIO_3, GPIO_MODE_NEC_SEND);
        #endif
        _ASSERT((NEC_BOOT_CODE_START_TIME*NEC_CLK_US) < 0x7fff);/* 确保最长的引导码编码后不会越界 */
        /* 需要发送引导码 */
        nec_bit_data[0] = (NEC_BOOT_CODE_START_TIME*NEC_CLK_US)|(1<<15);
```

```
21            nec_bit_data[1] = NEC_BOOT_CODE_STOP_TIME*NEC_CLK_US;
22            (*bitIndex)++;
23            NEC_WriteData(nec_bit_data,2);
24        }
25        for(;*bitIndex < NEC_DATA_BIT_NUM;)
26        {
27            if(NEC_DMA_BUF_SIZE-(NEC_GetDataCount()) <= (2+2))/* 一个bit需要两个uint16_t,预留两
  uint16_t放截止符 */
28            {
29                #if (NEC_SUB_FILL_DATA == 0)
30                yc_log("dma buf not enough");
31                return false;
32                #else
33                if(start_index == 0)
34                {
35                    NEC_Cmd(ENABLE);
36                    NEC_SetIrq(NEC_IRQ_DONE|NEC_IRQ_HALF_DONE,ENABLE);
37                }
38                nec_need_fill_data = true;
39                return true;
40                #endif
41            }
42            nec_bit_data[0] = (NEC_DATA_START_TIME*NEC_CLK_US)|(1<<15);
43            if(((uint8_t*)txData)[(*bitIndex-1)/8]&(0x01<<((*bitIndex-1)%8)))
44            {
45                nec_bit_data[1] = NEC_DATA_BIT1_STOP_TIME*NEC_CLK_US;
46            }
47            else
48            {
49                nec_bit_data[1] = NEC_DATA_BIT0_STOP_TIME*NEC_CLK_US;
50            }
51            NEC_WriteData(nec_bit_data,2);
52            (*bitIndex)++;
53        }
54        nec_bit_data[0] = (NEC_DATA_START_TIME*NEC_CLK_US)|(1<<15);/* 添加截止符,模块启动一次
  发送后直到遇到数据0停止 */
55        nec_bit_data[1] = 0;
56        NEC_WriteData(nec_bit_data,2);
57        if(start_index == 0)
58        {
59            NEC_Cmd(ENABLE);
60            NEC_SetIrq(NEC_IRQ_DONE|NEC_IRQ_HALF_DONE,ENABLE);
61        }
62        nec_need_fill_data = false;
63        return true;
```

```
}
```

重复码发送函数

```
bool NEC_SendRepeat(void)
{
    uint16_t nec_bit_data[4];
    if(NEC_DMA_BUF_SIZE-(NEC_GetDataCount()) <= 3)/* uint16_t count */
    {
        return false;
    }
    NEC_Cmd(DISABLE);
    NEC_PtrInit();
    nec_bit_data[0] = (NEC_REPEAT_START_TIME*NEC_CLK_US)|(1<<15);
    nec_bit_data[1] = NEC_REPEAT_STOP_TIME*NEC_CLK_US;
    nec_bit_data[2] = (NEC_DATA_START_TIME*NEC_CLK_US)|(1<<15);
    nec_bit_data[3] = 0;/* 添加截止符,模块启动一次发送后直到遇到数据 0 停止 */
    NEC_WriteData(nec_bit_data,sizeof(nec_bit_data)/sizeof(uint16_t));
    #if (NEC_USE_EXTERNAL_DRIVE)
    GPIO_Config(NEC_PIN, GPIO_MODE_NEC_SEND);/* 非 IO 直驱时可配任意 IO */
    #else
    GPIO_Config(GPIO_0, GPIO_MODE_NEC_SEND);
    GPIO_Config(GPIO_1, GPIO_MODE_NEC_SEND);
    GPIO_Config(GPIO_2, GPIO_MODE_NEC_SEND);
    GPIO_Config(GPIO_3, GPIO_MODE_NEC_SEND);
    #endif
    NEC_Cmd(ENABLE);
    NEC_SetIrq(NEC_IRQ_DONE|NEC_IRQ_HALF_DONE,ENABLE);
    return true;
}
```

NEC 中断处理，触发完成中断将 GPIO 置为高阻态,触发半 half 中断继续发送数据

```
void NEC_IRQHandler(void)
{
    uint8_t necStatus=NEC_GetIrqStatus();
    if(necStatus&(NEC_IRQ_DONE))
    {
        #if (NEC_USE_EXTERNAL_DRIVE)
        GPIO_Config(NEC_PIN, GPIO_MODE_HI_Z);/* 非 IO 直驱时可配任意 IO */
        #else
        GPIO_Config(GPIO_0, GPIO_MODE_HI_Z);
        GPIO_Config(GPIO_1, GPIO_MODE_HI_Z);
        GPIO_Config(GPIO_2, GPIO_MODE_HI_Z);
        GPIO_Config(GPIO_3, GPIO_MODE_HI_Z);
```

```
            #endif
        }
        if(nec_need_fill_data == true)
        {
            NEC_Send(&nec_send_data,&nec_send_bit_index);
        }
        NEC_ClearIrq(necStatus);
}
```

14.4.2. nec recv demo

- 演示 nec 轮询接收数据，关键代码说明如下：

NEC 参数宏定义

```
#define NEC_CLK_US                      (1)
#define NEC_USE_EXTERNAL_DRIVE          (0)             /* 0:使用内部放大电路,1:使用外部放大电路 */
#define NEC_IRQ_ENABLE                  (0)             /* 0:不使能 NEC 中断,1:使能 NEC 中断 */
#define NEC_GAIN_LEVEL                  (GAIN_HIGHEST)  /* 红外内部放大电路增益 */
#define NEC_DMA_BUF_SIZE                (50)

#define NEC_BOOT_CODE_START_TIME        9000            /* 9ms */
#define NEC_BOOT_CODE_STOP_TIME         4500            /* 4.5ms */
#define NEC_DATA_START_TIME             560             /* 560us */
#define NEC_DATA_BIT0_STOP_TIME         560             /* 560us */
#define NEC_DATA_BIT1_STOP_TIME         1690            /* 1690us */
#define NEC_REPEAT_START_TIME           9000            /* 9ms */
#define NEC_REPEAT_STOP_TIME            2250            /* 2250us */
#define NEC_START_ERROR_VALUE           500             /* start time error value define */
#define NEC_STOP_ERROR_VALUE            300             /* stop time error value define */
#define NEC_ERROR_VALUE                 100             /* error value define */
```

初始化系统和串口打印初始化

```
    SYSCTRL_ChipInit();

    printport_init();
```

NEC 初始化，若开启 NEC 中断，则配置 1ms systick。

```
    #if (NEC_IRQ_ENABLE)
    SYSTICK_Config(SYSTICK_ONE_MS*10);
```

```
    #endif
    yc_log("NEC receive demo");
    NEC_RxInit();
```

NEC 接收初始化，配置 DMA 地址和长度，时钟频率和模式，并配置 NEC 的 GPIO 管脚，若宏定义开启中断则使能中断。

```
void NEC_RxInit(void)
{
    NEC_InitStruct initStruct = {0};

    initStruct.bufSaddr =   (uint32_t)nec_dma_buf;
    initStruct.bufSize   = sizeof(nec_dma_buf);
    initStruct.clk         = NEC_CLK_US*1000000;
    initStruct.mode       = NEC_RX;

    NEC_Init(&initStruct);
    #if (NEC_USE_EXTERNAL_DRIVE)
    GPIO_Config(NEC_PIN, GPIO_MODE_NEC_REC);/*  非 IO 直驱时可配任意 IO */
    NEC_RxSrcSel(NEC_FROM_GPIO);
    #else
    GPIO_AnalogEn(GPIO_0,ENABLE);
    GPIO_Config(GPIO_0, GPIO_MODE_NEC_REC);
    NEC_RxSrcSel(NEC_FROM_IR);
    NEC_SetRxGainLevel(NEC_GAIN_LEVEL);
    #endif
    #if (NEC_IRQ_ENABLE)
    NEC_SetIrq(NEC_IRQ_DONE|NEC_IRQ_HALF_DONE,ENABLE);
    PLIC_SetPriority(NEC_IRQn,1);
    PLIC_EnableIRQ(NEC_IRQn);
    #endif
    NEC_Cmd(ENABLE);
}
```

NEC 获取处理函数，根据 NEC_Receive 函数获取结果进行不同处理。

```
void nec_recv_process(void)
{
    switch(NEC_Receive(&nec_receive_data))
    {
        case NEC_RECV_SUCC:
            yc_log_buf("nec recv data",(uint8_t *)&nec_receive_data,sizeof(NEC_DataStruct));
            break;
        case NEC_RECV_REPEAT:
```

```
                yc_log("nec recv repeat");
                break;
            case NEC_ADDR_CHECK_ERR:
                yc_log("nec addr check err");
                break;
            case NEC_DATA_CHECK_ERR:
                yc_log("nec data check err");
                break;
            default:
                break;
        }
}
```

NEC 接收函数：把硬件时间样本解码为 NEC 协议的 4 字节数据并校验反码。

```
NEC_StatusEnum NEC_Receive(NEC_DataStruct *rxData)
{
    uint32_t nec_rev_data = 0;

    while(NEC_GetDataCount() >= 2)
    {
        nec_rev_data = NEC_ReadData();
        nec_rev_data = nec_rev_data / NEC_CLK_US;
        if(nec_rev_data > (NEC_DATA_BIT0_STOP_TIME - NEC_ERROR_VALUE))
        {
            if((nec_rev_data > (NEC_BOOT_CODE_STOP_TIME - NEC_ERROR_VALUE)) &&
(nec_rev_data < (NEC_BOOT_CODE_STOP_TIME + NEC_ERROR_VALUE)))
            {
                necByteIndex = 0;
                necBitIndex = 0;
            }
            if((nec_rev_data > (NEC_DATA_BIT0_STOP_TIME - NEC_ERROR_VALUE)) &&
(nec_rev_data < (NEC_DATA_BIT0_STOP_TIME + NEC_ERROR_VALUE)))
            {
                necByteData &= (~(1 << necBitIndex++));
            }
            else if((nec_rev_data > (NEC_DATA_BIT1_STOP_TIME - NEC_ERROR_VALUE)) &&
(nec_rev_data < (NEC_DATA_BIT1_STOP_TIME + NEC_ERROR_VALUE)))
            {
                necByteData |= (1 << necBitIndex++);
            }
            else if((nec_rev_data > (NEC_REPEAT_STOP_TIME - NEC_ERROR_VALUE)) &&
(nec_rev_data < (NEC_REPEAT_STOP_TIME + NEC_ERROR_VALUE)))
            {
                NEC_Cmd(DISABLE);
```

```
                NEC_PtrInit();
                NEC_Cmd(ENABLE);
                necByteIndex = 0;
                return NEC_RECV_REPEAT;
            }
            if(necBitIndex == 8)
            {
                ((uint8_t *)rxData)[necByteIndex++] = necByteData;
                necByteData = 0;
                necBitIndex = 0;
            }
            if(necByteIndex == 4)
            {
                if(rxData->address != ((~rxData->addressInverse) & 0x000000ff))
                {
                    memset((uint8_t *)&rxData, 0, sizeof(NEC_DataStruct));
                    NEC_Cmd(DISABLE);
                    NEC_PtrInit();
                    NEC_Cmd(ENABLE);
                    return NEC_ADDR_CHECK_ERR;
                }
                if(rxData->data != ((~rxData->dataInverse) & 0x000000ff))
                {
                    memset((uint8_t *)&rxData, 0, sizeof(NEC_DataStruct));
                    NEC_Cmd(DISABLE);
                    NEC_PtrInit();
                    NEC_Cmd(ENABLE);
                    return NEC_DATA_CHECK_ERR;
                }
                necByteIndex = 0;
                return NEC_RECV_SUCC;
            }
        }
    }
    return NEC_NONE_DATA;
}
```

不开启中断，则在轮询接收

```
    while(1)
    {
        #if (NEC_IRQ_ENABLE == 0)
        nec_recv_process();
        #endif
```

```
    }
```

若开启中断，systick 中断检查 nec 获取数据个数有无变化，检测到个数在 20ms 内稳定不变时，触发 nec_recv_process()，触发 NEC 中断则直接进行 nec_recv_process(),并清除中断。

```
#if (NEC_IRQ_ENABLE)
_interrupt void VECTOR_SYSTICK_IRQ(void)
{
    SystickCount++;
    uint32_t temp_items = NEC_GetDataCount();
    if(temp_items != last_items)
    {
        last_items = temp_items;
        start_tick = SystickCount;
    }
    else
    {
        if(SYSTICK_GetRelativeTime(start_tick) > 20)
        {
            nec_recv_process();
        }
    }
    SYSTICK_ClearIrq();
}
#endif

#if (NEC_IRQ_ENABLE)
void NEC_IRQHandler(void)
{
    uint8_t necStatus=NEC_GetIrqStatus();
    nec_recv_process();
    NEC_ClearIrq(necStatus);
}
#endif
```

15. 实时时钟(RTC)

15.1. 宏定义、枚举及结构体说明

15.1.1. RTC 类型枚举

```
typedef enum
{
    ALARM0 = 0,
    ALARM1,
    OV_INTR
} RTC_CompEnum;
```

- ALARM0：闹钟 0 类型
- ALARM1：闹钟 1 类型
- OV_INTR：溢出类型

15.1.2. RTC 时钟源选择枚举

```
typedef enum
{
    RC_32K = 0,
    XO_32K
} RTC_ClkSelEnum;
```

- RC_32K：RC 32KHz 时钟源
- XO_32K：晶振 32KHz 时钟源

15.1.3. RTC 时间结构体

```
struct tm
{
  int tm_sec;
  int tm_min;
  int tm_hour;
  int tm_mday;
  int tm_mon;
  int tm_year;
```

```
    int tm_wday;
    int tm_yday;
    int tm_isdst;
#ifdef __TM_GMTOFF
    long    __TM_GMTOFF;
#endif
#ifdef __TM_ZONE
    const char *__TM_ZONE;
#endif
};
```

实际上应用只用到以下几个变量来设置日期时间：

- tm_sec：秒
- tm_min：分钟
- tm_hour：小时
- tm_mday：日
- tm_mon：月
- tm_year：年

15.2. RTC 接口汇总

函数类型	函数名	说明
RTC 相关接口	RTC_Cmd	RTC 模块使能
	RTC_DeInit	RTC 模块反初始化
	RTC_Calibration	RTC 时钟校准
	RTC_GetClk	获取 RTC 时钟
	RTC_ClkSel	RTC 时钟源选择
	RTC_SetIrq	设置 RTC 中断

	RTC_WKConfig	RTC 唤醒源配置
	RTC_SetAlarm0	设置 RTC alarm0 预设值
	RTC_SetAlarm1	设置 RTC alarm1 预设值
	RTC_GetCount	获取 RTC 计数值
	RTC_GetIrqStauts	获取 RTC 中断标志位
	RTC_ClearIrq	清除 RTC 中断标志位
	RTC_SetTime	设置日期时间
	RTC_GetTime	获取日期时间
	RTC_GetTimeStamp	获取 UNIX 时间戳

15.3. RTC 接口说明

15.3.1. void RTC_Cmd(FunctionalState state)

- 描述：RTC 模块使能
- 参数：
  - state：
    - 0：失能 RTC 模块功能
    - 1：使能 RTC 模块功能
- 返回值：none

15.3.2. void RTC_DeInit(void)

- 描述：RTC 模块反初始化
- 参数：none

- 返回值：none

15.3.3. void RTC_Calibration(void)

- 描述：RTC 时钟校准
- 参数：none
- 返回值：none

15.3.4. uint32_t RTC_GetClk(void)

- 描述：获取 RTC 模块时钟
- 参数：none
- 返回值：RTC 模块的时钟

15.3.5. void RTC_ClkSel(RTC_ClkSelEnum clkSel)

- 描述：RTC 时钟源选择
- 参数：
  - clkSel：可选择 RC 32KHz 和 晶振 32KHz，详见 RTC 时钟源选择枚举
- 返回值：none

15.3.6. void RTC_SetIrq(RTC_CompEnum type, FunctionalState state)

- 描述：配置 RTC 中断
- 参数：
  - type：RTC 中断类型，可以设置为溢出中断、alarm0 中断和 alarm1 中断，详见 RTC 类型枚举
  - state：

- 0：失能该类型中断
- 1：使能该类型中断

- 返回值：none

15.3.7. void RTC_WKConfig(RTC_CompEnum type, FunctionalState state)

- 描述：RTC 唤醒源配置
- 参数：
  - type：RTC 中断类型，可以设置为溢出中断、alarm0 中断和 alarm1 中断,详见 RTC 类型枚举
  - state：
    - 0：失能该类型中断唤醒系统
    - 1：使能该类型中断唤醒系统
- 返回值：none

15.3.8. void RTC_SetAlarm0(uint32_t alarmValue)

- 描述：设置 alarm0 的预设值
- 参数：
  - alarmValue：当 RTC 计数寄存器的值等于改该值时触发 alarm0 中断，范围为 0-0xffffffff
- 返回值：none

15.3.9. void RTC_SetAlarm1(uint32_t alarmValue)

- 描述：设置 alarm1 的预设值

- 参数：
  - alarmValue：当 RTC 计数寄存器的值等于改该值时触发 alarm1 中断，范围为 0-0xffffffff
- 返回值：none

15.3.10. uint32_t RTC_GetCount(RTC_CompEnum type)

- 描述：获取 RTC 计数值
- 参数：
  - type：alarm0 和 alarm1 的预设中断值和 RTC 当前计数值，详见 RTC 类型枚举
- 返回值：RTC 预设值

15.3.11. uint8_t RTC_GetIrqStauts(RTC_CompEnum type)

- 描述：获取 RTC 中断标志位
- 参数：
  - type：可选择 alarm0、alarm1 和溢出三种中断类型，详见 RTC 类型枚举
- 返回值：
  - 0：该类型中断未触发中断
  - 1：该类型中断触发中断

15.3.12. void RTC_ClearIrq(RTC_CompEnum type)

- 描述：清除 RTC 中断标志位

参数：

  - type：可选择 alarm0、alarm1 和溢出三种中断类型，详见 RTC 类型枚举
- 返回值：none

15.3.13. void RTC_SetTime(struct tm dateStruct)

- 描述：设置日期时间
- 参数：
  - dateStruct：详见 RTC 时间结构体
- 返回值：none

15.3.14. struct tm* RTC_GetTime(void)

- 描述：获取日期时间
- 参数：none
- 返回值：当前时间结构体指针，详见 RTC 时间结构体

15.3.15. time_t RTC_GetTimeStamp(void)

- 描述：获取 UNIX 时间戳
- 参数：none
- 返回值：UNIX 时间戳

15.4.demo 说明

- 演示 RTC 正常计时功能和 LPM 唤醒功能，关键代码说明如下：

RTC LPM 宏定义控制

```
#define RTC_LPM_ENABLE                  (0)
#define RTC_WAKEUP_TIME                 (1000)   /* ms */
```

RTC_WAKEUP_TIME：设置 RTC 唤醒时间，单位为 ms

先初始化时钟和串口打印的 UART 端口

```
SYSCTRL_ChipInit();
printport_init();
```

对 RTC 配置并进行校准，并设置时间

```
if((LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_WAKE) == 0)
{
    delay_ms(500);/* 此处延时为方便下载 sync,应用里不需要加 */

    RTC_Calibration();
    yc_log("rtc_clk = %d",RTC_GetClk());
    RTC_Configuration();
    RTC_SetTime(timer_struct);
    last_calibration_time = RTC_GetTimeStamp();
    /* long long 型的 time_t 数据类型打印需要去掉编译器全局附加选项的--specs=nano.specs,并使用%lld 格式符,但会大大增加固件体积 */
    //yc_log("last_calibration_time = %lld",last_calibration_time);
}
```

RTC 配置

```
void RTC_Configuration(void)
{
    RTC_DeInit();
    #if (RTC_LPM_ENABLE==0)
    RTC_SetAlarm1(RTC_GetCount(OV_INTR)+RTC_GetClk()); /* alarm1 timer 1 seconds */
    RTC_SetIrq(ALARM1,ENABLE);
    #endif
    RTC_Cmd(ENABLE);
}
```

将 RTC 中断类型设置为 ALARM1 中断

15.4.1. RTC 日历时间模式

如果是正常 RTC 计时的 demo 则开启 RTC 中断

```
#if VECTOR_IRQ_ENBALE
VECTOR_EnableIRQ(VECTOR_RTC_IRQn);
#else
PLIC_SetPriority(RTC_IRQn,1);
PLIC_EnableIRQ(RTC_IRQn);
#endif
#endif
```

当 RTC 的 ALARM1 的计数值到达预设值时触发中断，预设值为当前时间的下一秒钟

在 RTC 中断中更新日历时间并打印，然后预设下次中断的时间

```
#if (RTC_LPM_ENABLE==0)
#if VECTOR_IRQ_ENBALE
_interrupt void VECTOR_RTC_IRQ(void)
#else
void RTC_IRQHandler(void)
#endif
{
    if(RTC_GetIrqStauts(ALARM1))    /* 将 alarm1 作为秒中断使用，实现日历功能 */
    {
        if(difftime(RTC_GetTimeStamp(),last_calibration_time) >= 3600)  /* 校准频率为一小时一次，可以根据时间精度调整；当前校准精度为 2048，理论耗时 62.5ms */
        {
            RTC_Calibration();
            last_calibration_time = RTC_GetTimeStamp();
            yc_log("rtc_clk = %d",RTC_GetClk());
        }
        struct tm *ptime = RTC_GetTime();
        yc_log("%04d-%02d-%02d %02d:%02d:%02d", ptime->tm_year, ptime->tm_mon, ptime->tm_mday, ptime->tm_hour, ptime->tm_min, ptime->tm_sec);
        RTC_SetAlarm1(RTC_GetCount(ALARM1)+RTC_GetClk());
        RTC_ClearIrq(ALARM1);
    }
    if(RTC_GetIrqStauts(OV_INTR))
    {
        yc_log("rtc reg over");
        RTC_ClearIrq(OV_INTR);
    }
}
#endif
```

将预设时间定为当前时间的下一秒钟

15.4.2. RTC LPM 模式

串口外设关闭

```
void printport_deinit(void)
{
    UART_DeInit(PRINT_PORT);
    GPIO_Config(PRINT_TX_IO_PIN,GPIO_MODE_HI_Z);
    GPIO_Config(PRINT_RX_IO_PIN,GPIO_MODE_HI_Z);
```

```
}
```

关闭中断，关闭外设，将 GPIO 设置为高阻

```
void LPM_prepareActionBeforeSleep(LPM_ModeEnum sleepLevel)
{
    /*
    为了降低睡眠功耗，建议将睡眠后不使用的外设关闭
    并根据外围电路将睡眠期间不使用的 GPIO 配置成 GPIO_MODE_HI_Z
    */

    __disable_irq();
    LpmRecoverStruct->rg_lpm_ldo_dig_retmem_vsel.bit.sleep=5;
    LpmRecoverStruct->rg_lpm_ldo_dig_lpm_vosel.bit.sleep=0;

    /*唤醒响应时间调整*/
    printport_deinit();
}
```

唤醒后的初始化函数

```
void LPM_postActionWakeup(LPM_ModeEnum sleepLevel)
{
    SYSCTRL_ChipInit();
    printport_init();
}
```

判断是什么唤醒源将芯片唤醒并做出相应处理

```
void wakeup_FlagAnalysis(void)
{
     yc_log("wakeup_flag=%x",LpmRecoverStruct->wakeup_flag);
     if(LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_WAKE)
     {
          if(LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_KEY_SCAN)
          {
               yc_log("key scan wakeup");
          }
          if(LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_RTCOV)
          {
               yc_log("rtc over wakeup");
               RTC_WKConfig(OV_INTR,DISABLE);
          }
          if(LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_RTC0)
          {
```

```
17                  yc_log("rtc0 wakeup");
18                  RTC_WKConfig(ALARM0,DISABLE);
19                  if(difftime(RTC_GetTimeStamp(),last_calibration_time) >= 3600)      /* 校准频率为一小
时一次，可以根据时间精度调整；当前校准精度为 2048，理论耗时 62.5ms */
20                  {
21                      RTC_Calibration();
22                      last_calibration_time = RTC_GetTimeStamp();
23                      yc_log("rtc_clk = %d",RTC_GetClk());
24                  }
25                  struct tm *ptime = RTC_GetTime();
26                  yc_log("%04d-%02d-%02d %02d:%02d:%02d", ptime->tm_year, ptime->tm_mon,
ptime->tm_mday, ptime->tm_hour, ptime->tm_min, ptime->tm_sec);
27              }
28              if(LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_RTC1)
29              {
30                  yc_log("rtc1 wakeup");
31                  RTC_WKConfig(ALARM1,DISABLE);
32              }
33              if(LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_GPIO)
34              {
35                  yc_log("gpio wakeup");
36
yc_log_buf("gpio_in",LpmRecoverStruct->gpio_in,sizeof(LpmRecoverStruct->gpio_in));
37                  for(int i=0;i<GPIO_NUM;i++)
38                  {
39                      uint8_t wake_cfg=((LPM_PUWAKEUP->gpio_wake_cfg[i/2]>>((i%2)*4))&0x0f);
40                      if(LpmRecoverStruct->gpio_in[i/8]&(1<<(i%8)))/*gpio i input high*/
41                      {
42                          if(wake_cfg==SLEEP_HIGHWAKE ||
wake_cfg==SLEEP_PULLDOWN_HIGHWAKE)
43                          {
44                              yc_log("gpio%d high wakeup",i);
45                          }
46                      }
47                      else/* gpio i input low */
48                      {
49                          if(wake_cfg==SLEEP_LOWWAKE || wake_cfg==SLEEP_PULLUP_LOWWAKE)
50                          {
51                              yc_log("gpio%d low wakeup",i);
52                          }
53                      }
54                  }
55              }
56              if(LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_RAM_CRC_ERR)
57              {
```

```
                yc_log("ram verify error");
            }
        }
}
```

判断如下几种唤醒源：

- 键扫描唤醒
- RTC 溢出唤醒
- RTC ALARM0 计时唤醒
- RTC ALARM1 计时唤醒
- GPIO 唤醒
- RAM CRC 校验错误唤醒

令芯片进入睡眠然后设置唤醒时间唤醒后分析唤醒源

```
while(1)
{
    #if RTC_LPM_ENABLE
    yc_log("lpm sleep");

LPM_Sleep(RTC_WAKEUP_TIME,SLEEP_MODE_DEEP,0x1f,WAKEUP_FROM_SLEEP,LPM_prepareActionBeforeSleep,LPM_postActionWakeup);
    yc_log("wakeup");
    wakeup_FlagAnalysis();
    #endif
}
```

16. QSPI

16.1. 宏定义、枚举及结构体说明

16.1.1. QSPI 传输模式宏定义

```
typedef enum
{
    QSPI_STANDARD_MODE = 0,    /* 单线模式 */
    QSPI_DUAL_MODE,            /* 双线模式 */
    QSPI_QSPI_MODE,            /* 四线模式 */
} QSPI_ModeEnum;
```

16.2. QSPI 操作接口汇总

函数类型	函数名	说明
QSPI 相关接口	QSPI_SetMode	QSPI 传输模式配置
	QSPI_ClkSel	QSPI 时钟速率配置
	QSPI_ClkCmd	QSPI 使能配置
	QSPI_DmaStart	QSPI DMA 使能配置
	QSPI_SetDummy	设置 QSPI dummy 数
	QSPI_WaitDone	QSPI 等待传输完成
	QSPI_9bitModeEn	QSPI 9bit 模式使能配置
	Qspi_SendAndReceiveData	QSPI 收发数据
	QSPI_SetIrq	QSPI 中断使能配置
	QSPI_ClearIrq	QSPI 中断状态清除

16.3. 接口说明

16.3.1. void QSPI_SetMode(CORE_QSPI1_Type* qspix, QSPI_ModeEnum cmdMode, QSPI_ModeEnum addrMode, QSPI_ModeEnum dataMode)

- 描述：QSPI 传输模式配置
- 参数：
  - qspix：
    - CORE_QSPI1
    - CORE_QSPI2
  - cmdMode：QSPI 发送命令传输模式，详见 QSPI 传输模式宏定义
  - addrMode：QSPI 发送地址传输模式，详见 QSPI 传输模式宏定义
  - dataMode：数据传输模式，详见 QSPI 传输模式宏定义
- 返回值：none

16.3.2. void QSPI_ClkSel(CORE_QSPI1_Type* qspix, DPLL_ClkEnum qspiDpllClkSel)

- 描述：QSPI 时钟速率配置
- 参数：
  - qspix：
    - CORE_QSPI1
    - CORE_QSPI2
  - qspiDpllClkSel：DPLL 时钟选择，详见 SYSCTRL 章节中的 BBPLL 时钟选择枚举
- 返回值：none

16.3.3. void QSPI_ClkCmd(CORE_QSPI1_Type* qspix, FunctionalState state)

- 描述：QSPI 使能配置
- 参数：
  - qspix：
    - CORE_QSPI1
    - CORE_QSPI2
  - state：
    - ENABLE：使能
    - DISABLE：失能
- 返回值：none

16.3.4. void QSPI_DmaStart(CORE_QSPI1_Type* qspix, FunctionalState state)

- 描述：QSPI DMA 使能配置
- 参数：
  - qspix：
    - CORE_QSPI1
    - CORE_QSPI2
  - state：
    - ENABLE：使能
    - DISABLE：失能
- 返回值：none

16.3.5. void QSPI_SetDummy(CORE_QSPI1_Type* qspix, uint8_t dly)

- 描述：设置 QSPI dummy 数
- 参数：
  - qspix：
    - CORE_QSPI1
    - CORE_QSPI2
  - dly：QSPI dummy 数
- 返回值：none

16.3.6. uint8_t QSPI_WaitDone(CORE_QSPI1_Type* qspix)

- 描述：QSPI 等待传输完成
- 参数：
  - qspix：
    - CORE_QSPI1
    - CORE_QSPI2
- 返回值：
  - 传输完成：
    - 1：完成
    - 0：超时未完成

16.3.7. void QSPI_9bitModeEn(CORE_QSPI1_Type* qspix, FunctionalState state)

- 描述：QSPI 9bit 模式使能配置

- 参数：
  - qspix：
    - CORE_QSPI1
    - CORE_QSPI2
  - state：
    - ENABLE：使能
    - DISABLE：失能
- 返回值：none

16.3.8. uint8_t Qspi_SendAndReceiveData(CORE_QSPI1_Type* qspix, uint8_t *cmdbuf, uint8_t cmdlen, uint8_t *txbuf, uint32_t txlen, uint8_t *rxbuf, uint32_t rxlen)

- 描述：QSPI 收发数据
- 参数：
  - qspix：
    - CORE_QSPI1
    - CORE_QSPI2
  - cmdbuf：发送命令数据地址（第一字节发送命令，之后字节发送地址）
  - cmdlen：发送命令数据长度
  - txbuf：发送数据地址
  - txlen：发送数据长度
  - rxbuf：接收数据地址
  - rxlen：接收数据长度

- 返回值：
  - 收发数据状态：
    - 1：收发成功
    - 0：收发超时

16.3.9. void QSPI_SetIrq(CORE_QSPI1_Type* qspix, FunctionalState state)

- 描述：QSPI 中断使能配置
- 参数：
  - qspix：
    - CORE_QSPI1
    - CORE_QSPI2
  - state：
    - ENABLE：使能
    - DISABLE：失能
- 返回值：none

16.3.10. void QSPI_ClearIrq(CORE_QSPI1_Type* qspix)

- 描述：QSPI 中断状态清除
- 参数：
  - qspix：
    - CORE_QSPI1
    - CORE_QSPI2
- 返回值：none

16.4. demo 说明

- 演示 QSPI 对 FLASH 读写及一致性检查，关键代码说明如下：

宏定义选择收发类型

```
#define SINGLE_BYTE_TRANSCEIVER                 1               /* 单字节收发 */
#define SEND_AND_RECEIVE_INDEPENDENCE           2               /* 收发独立 */
#define SEND_AND_RECEIVE_TOGETHER               3               /* 收发一次性完成 */

#define TRANSCEIVER_MODE SEND_AND_RECEIVE_TOGETHER
```

初始化系统和串口打印初始化

```
    SYSCTRL_ChipInit();

    printport_init();
```

GPIO 初始化

```
void W25Q16_GPIO_Init(void)
{
    if(QSPI_PORT==CORE_QSPI1)
    {
        GPIO_Config(QSPI_NCS_PIN, GPIO_MODE_QSPI1_NCS);
        GPIO_Config(QSPI_SCK_PIN, GPIO_MODE_QSPI1_SCK);
        GPIO_Config(QSPI_IO0_PIN, GPIO_MODE_QSPI1_IO0);
        GPIO_Config(QSPI_IO1_PIN, GPIO_MODE_QSPI1_IO1);
        GPIO_Config(QSPI_IO2_PIN, GPIO_MODE_QSPI1_IO2);
        GPIO_Config(QSPI_IO3_PIN, GPIO_MODE_QSPI1_IO3);
    }
    if(QSPI_PORT==CORE_QSPI2)
    {
        GPIO_Config(QSPI_NCS_PIN, GPIO_MODE_QSPI2_NCS);
        GPIO_Config(QSPI_SCK_PIN, GPIO_MODE_QSPI2_SCK);
        GPIO_Config(QSPI_IO0_PIN, GPIO_MODE_QSPI2_IO0);
        GPIO_Config(QSPI_IO1_PIN, GPIO_MODE_QSPI2_IO1);
        GPIO_Config(QSPI_IO2_PIN, GPIO_MODE_QSPI2_IO2);
        GPIO_Config(QSPI_IO3_PIN, GPIO_MODE_QSPI2_IO3);
    }
}
```

QSPI 时钟选择和使能

```
    QSPI_ClkSel(QSPI_PORT,DPLL_CLKSEL_24MHZ);
    QSPI_ClkCmd(QSPI_PORT, ENABLE);
```

QSPI 收发函数，通过宏控制模式：收发一次性完成/收发独立/单字节收发

```
uint8_t QSPI_ModeSelInterface( CORE_QSPI1_Type* qspix,
    uint8_t *cmdbuf,      uint8_t cmdlen,
    uint8_t *txbuf,       uint32_t txlen,
    uint8_t *rxbuf,       uint32_t rxlen)
{
#if TRANSCEIVER_MODE == SEND_AND_RECEIVE_TOGETHER
    return Qspi_SendAndReceiveData(qspix, cmdbuf, cmdlen, txbuf, txlen, rxbuf, rxlen);
#elif TRANSCEIVER_MODE == SEND_AND_RECEIVE_INDEPENDENCE
    GPIO_Config(QSPI_NCS_PIN, GPIO_MODE_OUT_LOW);
    if(cmdlen > 0)
    {
        Qspi_SendAndReceiveData(qspix, cmdbuf, cmdlen, 0, 0, 0, 0);
    }
    if(txlen > 0)
    {
        Qspi_SendAndReceiveData(qspix, 0, 0, txbuf, txlen, 0, 0);
    }
    if(rxlen > 0)
    {
        Qspi_SendAndReceiveData(qspix, 0, 0, 0, 0, rxbuf, rxlen);
    }
    GPIO_Config(QSPI_NCS_PIN, GPIO_MODE_OUT_HIGH);
    return 1;
#else
    uint32_t index = 0;
    GPIO_Config(QSPI_NCS_PIN, GPIO_MODE_OUT_LOW);
    while(cmdlen > 0)
    {
        Qspi_SendAndReceiveData(qspix, cmdbuf+index, 1, 0, 0, 0, 0);
        ++index;
        --cmdlen;
    }
    index = 0;
    while(txlen > 0)
    {
        Qspi_SendAndReceiveData(qspix, 0, 0, txbuf + index, 1, 0, 0);
        ++index;
        --txlen;
    }
    index = 0;
```

```
    while(rxlen > 0)
    {
        Qspi_SendAndReceiveData(qspix, 0, 0, 0, 0, rxbuf + index, 1);
        ++index;
        --rxlen;
    }

    GPIO_Config(QSPI_NCS_PIN, GPIO_MODE_OUT_HIGH);
    return 1;
#endif
}
```

对 flash 进行单、双、4 线不同指令的读写操作，读写一致为成功

```
 W25Q16_BLOCKErase_64K(TEST_ADDR);
    yc_log("flash erase end\n");
    for(uint32_t i=0;i<TEST_SIZE;i++)
    {
        test_wbuf[i]=i%256;
    }
    W25Q16_WriteFlashData(TEST_ADDR,test_wbuf,TEST_SIZE);
    yc_log("flash write end!\n");

    W25Q16_ReadFlashData_03(TEST_ADDR,test_rbuf,TEST_SIZE);
    for(int i=0;i<TEST_SIZE;i++)
    {
        if(test_rbuf[i]!=test_wbuf[i])
        {
            yc_log("qspi flash standard_03 mod write/read error\ntest_wbuf[%d]=%d\ttest_rbuf[%d]=%d\n",i,test_wbuf[i],i,test_rbuf[i]);
            break;
        }
        if(i==(TEST_SIZE-1))
        {
            yc_log("qspi flash standard_03 write/read success\n");
        }
    }

    W25Q16_ReadFlashData_DualMode_3B(TEST_ADDR,test_rbuf,TEST_SIZE);
    for(int i=0;i<TEST_SIZE;i++)
    {
        if(test_rbuf[i]!=test_wbuf[i])
        {
            yc_log("qspi flash dual_3B mod write/read error\ntest_wbuf[%d]=%d\ttest_rbuf[%d]=%d\n",i,test_wbuf[i],i,test_rbuf[i]);
```

```
30                  break;
31              }
32              if(i==(TEST_SIZE-1))
33              {
34                  yc_log("qspi flash dual_3B mod write/read success\n");
35              }
36          }
37          W25Q16_ReadFlashData_QuadMode_6B(TEST_ADDR,test_rbuf,TEST_SIZE);
38          for(int i=0;i<TEST_SIZE;i++)
39          {
40              if(test_rbuf[i]!=test_wbuf[i])
41              {
42                  yc_log("qspi flash quad_6B mod write/read
    error\ntest_wbuf[%d]=%d\ttest_rbuf[%d]=%d\n",i,test_wbuf[i],i,test_rbuf[i]);
43                  break;
44              }
45              if(i==(TEST_SIZE-1))
46              {
47                  yc_log("qspi flash quad_6B mod write/read success\n");
48              }
49          }
50          W25Q16_BLOCKErase_64K(TEST_ADDR);
51          for(uint32_t i=TEST_SIZE-1;i<TEST_SIZE;i--)
52          {
53              test_wbuf[i]=i%256;
54          }
55          W25Q16_WriteFlashData_QuadMode(TEST_ADDR,test_wbuf,TEST_SIZE);
56          yc_log("flash quad mod write end!\n");
57
58          W25Q16_ReadFlashData_0B(TEST_ADDR,test_rbuf,TEST_SIZE);
59          for(int i=0;i<TEST_SIZE;i++)
60          {
61              if(test_rbuf[i]!=test_wbuf[i])
62              {
63                  yc_log("qspi flash standard_0B write/read
    error\ntest_wbuf[%d]=%d\ttest_rbuf[%d]=%d\n",i,test_wbuf[i],i,test_rbuf[i]);
64                  break;
65              }
66              if(i==(TEST_SIZE-1))
67              {
68                  yc_log("qspi flash standard_0B write/read success\n");
69              }
70          }
71
72          W25Q16_ReadFlashData_DualMode_BB(TEST_ADDR,test_rbuf,TEST_SIZE);
```

```
73      for(int i=0;i<TEST_SIZE;i++)
74      {
75          if(test_rbuf[i]!=test_wbuf[i])
76          {
77              yc_log("qspi flash dual_BB mod write/read
error\ntest_wbuf[%d]=%d\ttest_rbuf[%d]=%d\n",i,test_wbuf[i],i,test_rbuf[i]);
78              break;
79          }
80          if(i==(TEST_SIZE-1))
81          {
82              yc_log("qspi flash dual_BB mod write/read success\n");
83          }
84      }
85
86      W25Q16_ReadFlashData_QuadMode_EB(TEST_ADDR,test_rbuf,TEST_SIZE);
87      for(int i=0;i<TEST_SIZE;i++)
88      {
89          if(test_rbuf[i]!=test_wbuf[i])
90          {
91              yc_log("qspi flash quad_EB mod write/read
error\ntest_wbuf[%d]=%d\ttest_rbuf[%d]=%d\n",i,test_wbuf[i],i,test_rbuf[i]);
92              break;
93          }
94          if(i==(TEST_SIZE-1))
95          {
96              yc_log("qspi flash quad_EB mod write/read success\n");
97          }
98      }
```

17. SADC

17.1. 宏定义、枚举及结构体说明

17.1.1. SADC 量程选择宏定义

```
#define SADC_GPIO_RANGE_3V3          0
#define SADC_GPIO_RANGE_1V8          1
#define SADC_GPIO_RANGE              SADC_GPIO_RANGE_1V8
```

SADC GPIO 模式的量程宏定义，有 1.8V 和 3.3V 两种量程

17.1.2. SADC 通道枚举

```
typedef enum
{
    SADC_CHANNEL_VBAT,
   SADC_CHANNEL_VDCDC,
    SADC_CHANNEL_GPIO4=6,
    SADC_CHANNEL_GPIO5,
    SADC_CHANNEL_GPIO9,
    SADC_CHANNEL_GPIO20,
    SADC_CHANNEL_GPIO21,
    SADC_CHANNEL_GPIO22,
    SADC_CHANNEL_GPIO23,
    SADC_CHANNEL_GPIO24,
    SADC_CHANNEL_GPIO25,
    SADC_CHANNEL_GPIO30,
}SADC_ChxTypeDef;
```

规则组和注入组都可以按照该枚举的序号配置，规则组按照 bit 配置，0-15bit 对应相同序号的通道，最多可以配置 12 通道；注入组每 4bit 配置一个通道，最多可以配置四个通道。

17.1.3. SADC 状态标志位及中断标志位枚举

```
typedef enum
{
    SADC_EORC       =BIT0,
    SADC_EOIC       =BIT1,
    SADC_WDT        =BIT2,
```

```
    SADC_HALF          =BIT3,
    SADC_IHALF         =BIT4,
    SADC_OVER          =BIT5,
    SADC_IOVER         =BIT6,
    SADC_EOC           =BIT7,
    SADC_ALL_EVENT   =0xff,
} SADC_IrqEnum;
```

- SADC_EORC：SADC 的规则组配置的所有通道每次采集完成后该中断标志位置一
- SADC_EOIC：SADC 的注入组配置的所有通道每次采集完成后该中断标志位置一
- SADC_WDT：SADC 的采样值大于设置的看门狗上限阈值或下限阈值时该中断标志位置一
- SADC_HALF：SADC 规则组 DMA 缓冲区写入达到一半时该中断标志位置一
- SADC_IHALF：SADC 注入组 DMA 缓冲区写入达到一半时该中断标志位置一
- SADC_OVER：SADC 规则组 DMA 缓冲区写满时该中断标志位置一
- SADC_IOVER：SADC 注入组 DMA 缓冲区写满时该中断标志位置一
- SDAC_EOC：规则组或注入组每次采集完成时该中断标志位置一
- SADC_ALL_EVENT：所有事件

17.1.4. SADC 通道模式枚举

```
typedef enum
{
    SADC_VBAT_MODE,
    SADC_GPIO_MODE,
} SADC_ChModeEnum;
```

该枚举在计算电压时使用，如果通道选择的是 VBAT 则选择 VBAT 模式进行计算，如果通道选择 GPIO 则选择 GPIO 模式进行计算

17.1.5. SADC 触发方式枚举

```
typedef enum
{
```

```
    SADC_TIMER0 = 0,
    SADC_TIMER1,
    SADC_TIMER2,
    SADC_TIMER3,
    SADC_TIMER4,
    SADC_TIMER5,
    SADC_TIMER6,
    SADC_TIMER7,
    SADC_SOFTWARE = 0xf,
} SADC_EtsTypeDef;
```

17.1.6. SADC 初始化结构体

```
typedef struct
{
    FunctionalState continuous_mode;    /* 连续采样模式 */
    FunctionalState auto_inject;        /* 自动注入使能 */
    SADC_EtsEnum etsrc;                 /* 规则组采样触发源 */
    SADC_EtsEnum etsic;                 /* 注入组采样触发源 */
} SADC_InitStruct;
```

- cont_mode_en：SADC 规则组单次、连续采样模式配置
  - 0：配置规则组单次采样模式
  - 1：配置规则组连续采样模式
- autoinject：SADC 注入组单次、自动采样模式配置
  - 0：配置注入组单次采样模式
  - 1：配置注入组自动采样模式
- dma_en：SADC 是否开启 DMA 功能
  - 0：关闭 DMA 功能
  - 1：开启 DMA 功能
- etsrc：SADC 规则组采样触发方式，详见 SADC 触发方式枚举
- etsic：SADC 注入组采样触发方式，详见 SADC 触发方式枚举

17.1.7. SADC 看门狗初始化结构体

```
typedef struct
{
    FunctionalState wdt_ren; /* watch dog regular enable */
    FunctionalState wdt_ien; /* watch dog injected enable */
    FunctionalState wdt_sc;   /* watch dog all channel */
    SADC_ChxTypeDef wdt_ch;
}
SADC_Wdt_TypeDef;
```

- wdt_ren：SADC 规则组看门狗功能使能
  - 0：失能
  - 1：使能
- wdt_ien：SADC 注入组看门狗功能使能
  - 0：失能
  - 1：使能
- wdt_sc：SADC 单通道还是全通道使用看门狗功能
  - 0：全通道使用看门狗功能
  - 1：单通道使用看门狗功能
- wdt_ch：SADC 看门狗通道配置，可配置通道参考 SADC 通道枚举

17.2. SADC 操作接口汇总

函数类型	函数名	说明
SADC 相关接口	SADC_Init	SADC 初始化
	SADC_DeInit	SADC 反初始化
	SADC_InputDiv2	SADC 输入分频使能
	SADC_SetRegularChannel	SADC 规则组通道配置

	SADC_SetInjectedChannel	SADC 注入组通道配置
	SADC_RegularTrigsel	SADC 规则组触发源配置
	SADC_InjectTrigsel	SADC 注入组触发源配置
	SADC_RegularTrigger	SADC 规则组软件触发
	SADC_InjectTrigger	SADC 注入组软件触发
	SADC_GetRegularConversionValue	获取 SADC 规则组采样值
	SADC_GetInjectedConversionValue	获取 SADC 注入组采样值
	SADC_SetRegualrSampleMode	SADC 规则组模式配置
	SADC_SetInjectSampleMode	SADC 注入组模式配置
	SADC_DmaCmd	SADC DMA 使能
	SADC_DmaConfig	SADC DMA 配置
	SADC_WdtConfig	SADC 看门狗配置
	SADC_GetStatus	SADC 状态获取
	SADC_SetIrq	SADC 中断配置
	SADC_GetIrqStatus	获取 SADC 中断状态
	SADC_ClearIrq	清除 SADC 中断标志位
	SADC_CalculateVoltage	计算 SADC 的采样电压

17.3. SADC 接口说明

17.3.1. void SADC_Init(SADC_InitTypeDef *SADC_InitStruct)

- 描述：SADC 的初始化函数，详细参数配置参考 SADC 初始化结构体
- 参数：
  - SADC_InitStruct：SADC 初始化结构体
- 返回值：none

17.3.2. void SADC_DeInit(void)

- 描述：将 SADC 时钟和使能关闭并清空 sadc0_ctrl 的配置
- 参数：none
- 返回值：none

17.3.3. void SADC_InputDiv2(FunctionalState state)

- 描述：SADC 输入分频使能
- 参数：
  - state
    - DISABLE：失能输入电压除以 2
    - ENABLE：使能输入电压除以 2
- 返回值：none

17.3.4. void SADC_SetRegularChannel(uint16_t rlist)

```
1 void SADC_SetRegularChannel(uint16_t rlist)
```

- 描述：SADC 规则组通道配置
- 参数：
  - rlist：要配置的通道，每 bit 代表一个通道，最多可配置 12 个通道具体配置值和含义如下：

- 1<<SADC_CHANNEL_VBAT：采集 VBAT 的通道
- 1<<SADC_CHANNEL_VDCDC：采集 VDCDC 的通道
- 1<<SADC_CHANNEL_GPIO4：采集 GPIO4 的通道
- 1<<SADC_CHANNEL_GPIO5：采集 GPIO5 的通道
- 1<<SADC_CHANNEL_GPIO9：采集 GPIO9 的通道
- 1<<SADC_CHANNEL_GPIO20：采集 GPIO20 的通道
- 1<<SADC_CHANNEL_GPIO21：采集 GPIO21 的通道
- 1<<SADC_CHANNEL_GPIO22：采集 GPIO22 的通道
- 1<<SADC_CHANNEL_GPIO23：采集 GPIO23 的通道
- 1<<SADC_CHANNEL_GPIO24：采集 GPIO24 的通道
- 1<<SADC_CHANNEL_GPIO25：采集 GPIO25 的通道
- 1<<SADC_CHANNEL_GPIO30：采集 GPIO30 的通道

- 返回值：none

17.3.5. void SADC_SetInjectedChannel(uint16_t ilist, uint8_t num)

- 描述：SADC 注入组通道配置
- 参数：
  - ilist：要配置的通道，每 4bit 代表一个通道：
    - 第一个通道配置：配置为通道枚举
      - SADC_CHANNEL_VBAT：采集 VBAT 的通道
      - SADC_CHANNEL_GPIO4：采集 GPIO4 的通道
      - SADC_CHANNEL_GPIO5：采集 GPIO5 的通道

- SADC_CHANNEL_GPIO9：采集 GPIO9 的通道
- SADC_CHANNEL_GPIO20：采集 GPIO20 的通道
- SADC_CHANNEL_GPIO21：采集 GPIO21 的通道
- SADC_CHANNEL_GPIO22：采集 GPIO22 的通道
- SADC_CHANNEL_GPIO23：采集 GPIO23 的通道
- SADC_CHANNEL_GPIO24：采集 GPIO24 的通道
- SADC_CHANNEL_GPIO25：采集 GPIO25 的通道
- SADC_CHANNEL_GPIO30：采集 GPIO30 的通道

  - 第二个通道配置：将通道枚举左移 4 位
  - 第三个通道配置：将通道枚举左移 8 位
  - 第四个通道配置：将通道枚举左移 12 位

- num：注入组通道数量

- 返回值：none

17.3.6. void SADC_RegularTrigsel(SADC_EtsTypeDef ets)

- 描述：SADC 规则组触发方式配置
- 参数：
  - ets：触发方式选择，可选择定时器触发和软件触发，定时器触发当定时器周期到达时会触发一次采样。如果选择软件触发则需要与函数 SADC_RegularSampleTrigger() 配合使用；如果配置为单次采样，则定时器每次定时结束或软件 trigger 后进行一次采样；如果配置为连续采样则第一次触发采样后一直连续采样：
    - SADC_TIMER0：TIMER0 触发
    - SADC_TIMER1：TIMER1 触发

- SADC_TIMER2：TIMER2 触发
- SADC_TIMER3：TIMER3 触发
- SADC_TIMER4：TIMER4 触发
- SADC_TIMER5：TIMER5 触发
- SADC_TIMER6：TIMER6 触发
- SADC_TIMER7：TIMER7 触发
- SADC_SOFTWARE：软件触发

- 返回值：none

17.3.7. void SADC_InjectTrigsel(SADC_EtsTypeDef ets)

- 描述：SADC 注入组触发方式配置
- 参数：
  - ets：触发方式选择，可选择定时器触发和软件触发，定时器触发当定时器周期到达时会触发一次采样。如果选择软件触发则需要与函数 SADC_InjectSampleTrigger()配合使用；如果配置为单次采样，则定时器每次定时结束或软件 trigger 后进行一次采样；如果配置为连续采样则第一次触发采样后一直连续采样：
    - SADC_TIMER0：TIMER0 触发
    - SADC_TIMER1：TIMER1 触发
    - SADC_TIMER2：TIMER2 触发
    - SADC_TIMER3：TIMER3 触发
    - SADC_TIMER4：TIMER4 触发
    - SADC_TIMER5：TIMER5 触发
    - SADC_TIMER6：TIMER6 触发

- SADC_TIMER7：TIMER7 触发
- SADC_SOFTWARE：软件触发

- 返回值：none

17.3.8. void SADC_RegularTrigger(void)

- 描述：SADC 规则组软件触发
- 参数：none
- 返回值：none

17.3.9. void SADC_InjectTrigger(void)

- 描述：SADC 注入组软件触发
- 参数：none
- 返回值：none

17.3.10. void SADC_SetIrq(SADC_IrqEnum irqn)

- 描述：配置 SADC 中断
- 参数：
  - irqn：使能对应的中断：
    - SADC_EORC：规则组单次采样完成中断
    - SADC_EOIC：注入组单次采样完成中断
    - SADC_WDT：看门狗中断
    - SADC_HALF：规则组 DMA 数据写入过半中断
    - SADC_IHALF：注入组 DMA 数据写入过半中断
    - SADC_OVER：规则组 DMA 数据写完中断

- SADC_IOVER：注入组 DMA 数据写完中断
- SADC_EOC：单次采样完成中断(不区分规则组注入组)

- 返回值：none

17.3.11. uint8_t SADC_GetIrqStatus(void)

- 描述：获取 SADC 中断标志位
- 参数：none
- 返回值：
  - 中断标志位：
    - bit0：规则组单次采样完成中断标志位
    - bit1：注入组单次采样完成中断中断标志位
    - bit2：看门狗中断标志位
    - bit3：规则组 DMA 数据写入过半中断标志位
    - bit4：注入组 DMA 数据写入过半中断标志位
    - bit5：规则组 DMA 数据写完中断标志位
    - bit6：注入组 DMA 数据写完中断标志位
    - bit7：单次采样完成中断(不区分规则组注入组)标志位

17.3.12. uint8_t SADC_GetStatus(void)

- 描述：获取 SADC 状态
- 参数：none
- 返回值：状态位：
  - bit0：规则组单次采样完成状态位
  - bit1：注入组单次采样完成中断状态位

- bit2：看门狗状态位
- bit3：规则组 DMA 数据写入过半状态位
- bit4：注入组 DMA 数据写入过半状态位
- bit5：规则组 DMA 数据写完状态位
- bit6：注入组 DMA 数据写完状态位
- bit7：单次采样完成(不区分规则组注入组)状态位

17.3.13. void SADC_ClearIrq(SADC_IrqEnum irqn)

- 描述：清除 SADC 中断标志位
- 参数：
  - irqn：清除对应的中断标志位：
    - SADC_EORC：规则组单次采样完成中断
    - SADC_EOIC：注入组单次采样完成中断
    - SADC_WDT：看门狗中断
    - SADC_HALF：规则组 DMA 数据写入过半中断
    - SADC_IHALF：注入组 DMA 数据写入过半中断
    - SADC_OVER：规则组 DMA 数据写完中断
    - SADC_IOVER：注入组 DMA 数据写完中断
    - SADC_EOC：单次采样完成中断(不区分规则组注入组)
- 返回值：none

17.3.14. void SADC_SetRegualrSampleMode(FunctionalState state)

- 描述：配置 SADC 规则组采样模式

- 参数：
  - state：设置规则组采样模式
    - DISABLE：为单次采样模式
    - ENABLE：为连续采样模式
- 返回值：none

17.3.15. void SADC_SetInjectSampleMode(FunctionalState state)

- 描述：配置 SADC 注入组采样模式
- 参数：
  - state：设置注入组采样模式
    - DISABLE：单次采样模式
    - ENABLE：自动采样模式，注入组会在规则组采样完成之后自动采样
- 返回值：none

17.3.16. void SADC_DmaCmd(FunctionalState newState)

- 描述：SADC DMA 功能使能
- 参数：
  - newState：使能或失能 DMA 功能：
    - DISABLE：失能
    - ENABLE：使能
- 返回值：none

17.3.17. void SADC_DmaConfig(uint16_t *rsaddr, uint16_t *readdr, uint16_t *ieaddr)

- 描述：SADC DMA 配置
- 参数：
  - rsaddr：规则组数据存放数组的起始地址
  - readdr：规则组数据存放数组的结束地址(规则组的结束地址+1 为注入组数据存放的起始地址)
  - ieaddr：注入组数据存放数组的结束地址（ieaddr 必须紧接着 readdr）
- 返回值：none

17.3.18. void SADC_WdtConfig(SADC_WdtStruct *wdt,uint16_t lowLimit,uint16_t highLimit)

- 描述：SADC 看门狗配置
- 参数：
  - wdt：看门狗功能初始化结构体，详见 SADC 看门狗初始化结构体说明
  - highLimit：看门狗电压上限阈值
  - lowLimit：看门狗电压下限阈值
- 返回值：none

17.3.19. uint16_t SADC_GetRegularConversionValue(void)

- 描述：获取 SADC 规则组采样值
- 参数：none
- 返回值：12 位 ADC 采样结果，范围 0~4095

17.3.20. uint16_t SADC_GetInjectedConversionValue(void)

- 描述：获取 SADC 注入组采样值
- 参数：none
- 返回值：12 位 ADC 采样结果，范围 0~4095

17.3.21. uint16_t SADC_CalculateVoltage(SADC_ChxEnum ch,uint16_t adcValue)

- 描述：计算 SADC 采样电压
- 参数：
  - ch：获取采样值时配置的通道
  - adcValue：ADC 采样值
- 返回值：计算得到的电压值，单位为 mV

17.4. demo 说明

- 使用 SADC 时，注意：
  - 若芯片供电小于等于 3.4V，SADC 推荐使用 1.8V 量程
  - 使用 1.8V 量程时若要测量 1.8V 以上电压，请使用 SADC_InputDiv2 函数将 SADC 输入分压功能使能

17.4.1. sadc demo

- 演示 SADC 单次采样，连续采样，定时器触发采样和中断功能关键代码说明如下：

SADC 功能配置宏定义

```
1 #define SADC_CONTINUOUS_MODE            0    /* SADC_CONTINUOUS_MODE==1 时不推荐开启 ADC 中断,也不能与 SADC_TIMER_TRIGGER_MODE 同时开启 */
2 #define SADC_TIMER_TRIGGER_MODE         0    /* SADC_TIMER_TRIGGER_MODE==1 时推荐开启 ADC 中断 */
```

```
#define SADC_WDT_EN                     0
#define SADC_IRQ                        0
```

开启看门狗功能后默认开启 SADC 的看门狗中断，开启定时器中断以后默认开启 EORC 中断，将宏定义的值设置为 1 时开启对应功能。

SADC 通道和引脚宏定义

```
#define ADC_GPIO_CH                     SADC_CHANNEL_GPIO24
#define ADC_GPIO_PIN                    GPIO_24
```

选择 SADC 采样通道和 GPIO 的宏定义两者需要相对应

连续采样单次采样配置

芯片初始化

```
SYSCTRL_ChipInit();
printport_init();
```

SADC 初始化

```
static void ADC_Configuration(void)
{
    SADC_InitStruct SADC_InitStruct;
    #if (SADC_CONTINUOUS_MODE==1)
    SADC_InitStruct.continuous_mode=ENABLE;
    #else
    SADC_InitStruct.continuous_mode=DISABLE;
    #endif
    SADC_InitStruct.auto_inject=DISABLE;
    SADC_InitStruct.etsrc=SADC_SOFTWARE;
    SADC_InitStruct.etsic=SADC_SOFTWARE;
    SADC_Init(&SADC_InitStruct);
    SADC_SetRegularChannel(1<<ADC_GPIO_CH);
}
static void ADC_GpioConfig()
{
    GPIO_Config(ADC_GPIO_PIN,GPIO_MODE_HI_Z);
    GPIO_AnalogEn(ADC_GPIO_PIN,ENABLE);
}
```

配置 SADC 开启连续还是单次采样

```
 #if (SADC_CONTINUOUS_MODE==1)
SADC_InitStruct.continuous_mode=ENABLE;
#else
SADC_InitStruct.continuous_mode=DISABLE;
#endif
```

通过宏定义来控制

SADC 看门狗功能配置

```
static void ADC_WDT_Configuration(void)
{
    SADC_WdtStruct wdt;
    wdt.wdt_ren=ENABLE;
    wdt.wdt_ien=DISABLE;
    wdt.wdt_ch=ADC_GPIO_CH;
    wdt.wdt_sc=1;
    SADC_WdtConfig(&wdt,100,3000);
    SADC_SetIrq(SADC_WDT,ENABLE);
}
```

设置 SADC 看门狗的上下限阈值电压并使能 SADC 看门狗中断

SADC 定时器功能配置

定时器通道分频宏定义

```
#define TIMER_CH                    SADC_TIMER6
#define TIMER_CLKDIV                (7)
```

配置触发采样的定时器，定时器时钟分频的宏定义

定时器初始化函数

```
static void TIM_CoreParaConfig(TIM_PortEnum timx, uint8_t clkDiv, uint32_t period)
{
    _ASSERT(period<=0xffff);
    TIM_InitStruct TIM_InitStruct;
    TIM_InitStruct.timx = timx;
    TIM_InitStruct.period = period;
    TIM_InitStruct.clockdiv = clkDiv;
    TIM_InitStruct.autoLoadEn=ENABLE;
    /* 定时器初始化 */
    TIM_Init(&TIM_InitStruct);
```

```
    /* 定时器使能 */
    TIM_Cmd(timx, ENABLE);
}
```

SADC 触发采样的定时器配置，定时器参数配置

```
SADC_RegularTrigsel(TIMER_CH);
TIM_CoreParaConfig((TIM_PortEnum)TIMER_CH, TIMER_CLKDIV, TIMER_1MS(TIMER_CLKDIV) * 100);
```

SADC 连续采样时代码(使能连续采样时不推荐开启定时器功能，原因详见 SADC 触发方式配置函数说明)

```
regular_adc_value=SADC_CalculateVoltage(ADC_GPIO_CH,SADC_GetRegularConversionValue());
yc_log("GPIO Voltage=%d",regular_adc_value);
```

一直计算采样值电压并打印

单次采样不开定时器时代码

未开启中断时

```
if(current_ch==ADC_GPIO_CH)
{
    current_ch=SADC_CHANNEL_VBAT;
}
else
{
    current_ch=ADC_GPIO_CH;
}
SADC_SetRegularChannel(1<<current_ch);
SADC_RegularTrigger();
#if (SADC_IRQ==0)
regular_adc_value=SADC_CalculateVoltage(current_ch,SADC_GetRegularConversionValue());
if(current_ch==ADC_GPIO_CH)
{
    yc_log("gpio voltage=%d",regular_adc_value);
}
else
{
    yc_log("vbat Voltage=%d",regular_adc_value);
}
```

开启中断时

```
if(irq_status&SADC_EORC)
{
    if(current_ch==ADC_GPIO_CH)
    {
        yc_log("gpio voltage=%d",SADC_CalculateVoltage(current_ch,regular_adc_value));
    }
    else
    {
        yc_log("vbat Voltage=%d",SADC_CalculateVoltage(current_ch,regular_adc_value));
    }
}
SADC_ClearIrq(irq_status);
```

不断在 VBAT 通道和 GPIO 通道间来回切换并 trig 然后计算电压并打印

单次采样开启定时器时(默认开启中断)

```
if(irq_status&SADC_EORC)
{
    if(current_ch==ADC_GPIO_CH)
    {
        yc_log("gpio voltage=%d",SADC_CalculateVoltage(current_ch,regular_adc_value));
    }
    else
    {
        yc_log("vbat Voltage=%d",SADC_CalculateVoltage(current_ch,regular_adc_value));
    }
}
SADC_ClearIrq(irq_status);
```

在中断中不断计算打印采样的电压值

开启看门狗功能时(默认开启 SADC 的看门狗中断)

```
if(irq_status&SADC_WDT)
{
    yc_log("adc wdt irq[regular_adc_value=%d]",regular_adc_value);
}
```

当采样原始值小于看门狗下限阈值或大于上限阈值时触发看门狗中断并打印电压值

17.4.2. sadc dma

- 演示 SADC 开启 DMA 功能，关键代码说明如下：

DMA 缓冲区数组和长度定义

```
#define ADC_BUF_LEN 3
typedef struct
{
    uint16_t adc_rbuf[ADC_BUF_LEN];
    uint16_t adc_ibuf[ADC_BUF_LEN];
}ADC_BUF_CB;
ADC_BUF_CB adc_buf;
```

SADC 定时器相关宏定义

```
#define TIMER_CH                SADC_TIMER6
#define TIMER_CLKDIV            (7)
```

配置触发采样的定时器和定时器时钟分频宏定义

定时器初始化配置

```
static void TIM_CoreParaConfig(TIM_PortEnum timx, uint8_t clkDiv, uint32_t period)
{
    _ASSERT(period<=0xffff);
    TIM_InitStruct TIM_InitStruct;
    TIM_InitStruct.timx = timx;
    TIM_InitStruct.period = period;
    TIM_InitStruct.clockdiv = clkDiv;
    TIM_InitStruct.autoLoadEn=ENABLE;
    /* timer core init */
    TIM_Init(&TIM_InitStruct);
    /* timer enable */
    TIM_Cmd(timx, ENABLE);
}
```

SADC 功能配置

```
static void ADC_Configuration(void)
{
    SADC_InitStruct SADC_InitStruct;
    SADC_InitStruct.continuous_mode=DISABLE;
    SADC_InitStruct.auto_inject=DISABLE;
```

```
    SADC_InitStruct.etsrc=SADC_SOFTWARE;
    SADC_InitStruct.etsic=SADC_SOFTWARE;
    SADC_Init(&SADC_InitStruct);

SADC_SetRegularChannel(1<<SADC_CHANNEL_GPIO22|1<<SADC_CHANNEL_GPIO23|1<<SADC_CHANNEL_GPIO24);
}
```

GPIO 配置

```
static void ADC_GpioConfig()
{
    GPIO_Config(GPIO_22,GPIO_MODE_HI_Z);
    GPIO_Config(GPIO_23,GPIO_MODE_HI_Z);
    GPIO_Config(GPIO_24,GPIO_MODE_HI_Z);
    GPIO_AnalogEn(GPIO_22,ENABLE);
    GPIO_AnalogEn(GPIO_23,ENABLE);
    GPIO_AnalogEn(GPIO_24,ENABLE);
}
```

配置多通道采样，配置三个通道分别采样 GPIO22、GPIO23、GPIO24

芯片初始化

```
SYSCTRL_ChipInit();
printport_init();
```

将规则组和注入组存放数据的数组地址配置入 DMA

```
SADC_DmaConfig(adc_buf.adc_rbuf,adc_buf.adc_rbuf+ADC_BUF_LEN-1,adc_buf.adc_ibuf+ADC_BUF_LEN-1);
```

SADC 定时器功能和中断配置

```
SADC_SetIrq(SADC_OVER,ENABLE);
SADC_RegularTrigsel(TIMER_CH);
TIM_CoreParaConfig((TIM_PortEnum)TIMER_CH, TIMER_CLKDIV, TIMER_1MS(TIMER_CLKDIV) * 100);
PLIC_SetPriority(SARADC_IRQn,2);
PLIC_EnableIRQ(SARADC_IRQn);
```

配置为 OVER 中断，当 DMA 缓冲区满时进入中断，并配置定时器参数

SADC 中断处理函数

```
void SADC_IRQHandler(void)
{
    uint8_t irq_status=SADC_GetIrqStatus();
    if(irq_status&SADC_OVER)
    {
        SADC_DmaCmd(DISABLE);
        for(uint16_t i=0;i<ADC_BUF_LEN;i++)
        {
            yc_log("adc data[%d] = %d %dmv",i,adc_buf.adc_rbuf[i],SADC_CalculateVoltage(SADC_CHANNEL_GPIO22,adc_buf.adc_rbuf[i]));
        }
        SADC_DmaCmd(ENABLE);
    }
    SADC_ClearIrq(irq_status);
}
```

定时器每次定时后触发单次采样，每次采样采集一次配置的所有通道，由于 DMA 缓冲区大小设置为配置的通道数量，所以每次定时器触发采样后触发 DMA 数组刚好填满，OVER 中断进入中断函数打印采集到的电压值

18. MIC

18.1. 宏定义、枚举及结构体说明

18.1.1. MIC 音频数据长度计算宏定义

```
#define AUDIO_DATA_MS(ms,sample_rate)                    ((ms)*((sample_rate)*(2)))
```

- ms：采样时长
- sample_rate：采样率，应填入采样率枚举

18.1.2. MIC 配置宏定义

```
#define ADC_FADE                                         ENABLE
#define ADC_HPF                                          ENABLE
#define ADC_DATA_TYPE                                    SIGNED_MODE
#define ADC_VOL_INIT                                     0x1000                  /* must less than ADC_VOL_MAX */
#define ADC_VOL_MAX                                      0x3fff                  /* max 14 bits = 0x3fff */
#define ADC_VOL_STEP                                     1                       /* 0x1~0xff, WARNING: set this to 0 and ADC_FADE ENABLE at same time will disable volume change */
```

- ADC_FADE：是否开启音量渐入/渐出功能配置：
  - 0：失能
  - 1：使能
- ADC_HPF：ADC 高通滤波器功能配置（建议开启）：
  - 0：失能
  - 1：使能
- ADC_DATA_TYPE：MIC ADC 输出数据类型配置，详见 MIC ADC 数据输出类型枚举
- ADC_VOL_INIT：初始化音量配置，范围为 0x0~0x3fff。前 2bit 为音量的整数部分，后 12bit 为音量 的小数部分

- ADC_VOL_MAX：最大音量，固定为 0x3fff
- ADC_VOL_STEP：配置 MIC 声音渐入/渐出功能音量渐变的步长

18.1.3. MIC 采样率枚举

```
typedef enum
{
    SAMPLE_RATE_8K      =  8,
    SAMPLE_RATE_16K     = 16,
    SAMPLE_RATE_24K     = 24,
    SAMPLE_RATE_32K     = 32,
    SAMPLE_RATE_44P1K   = 44,
    SAMPLE_RATE_48K     = 48,
} AUDIO_SampleRateEnum;
```

18.1.4. MIC 一级时钟分频系数枚举

```
typedef enum
{
    AUDIO_CLK_DIV_NONE,
    AUDIO_CLK_DIV_2,
    AUDIO_CLK_DIV_3
} AUDIO_ClkDivEnum;
```

18.1.5. MIC 二级时钟分频系数枚举

```
typedef enum
{
    ADC_CLK_DIV_NONE,
    ADC_CLK_DIV_2,
    ADC_CLK_DIV_4,
} AUDIO_AdcClkDivEnum;
```

18.1.6. MIC ADC 数据输出类型枚举

```
typedef enum
{
    MIC_SIGNED_MODE,
    MIC_UNSIGNED_MODE
```

```
} AUDIO_AdcDataModeEnum;
```

- MIC_SIGNED_MODE：输出有符号数据
- MIC_UNSIGNED_MODE：输出无符号数据

18.1.7. 数字 MIC 数据采集数据类型枚举

```
typedef enum
{
    MIC_RISING_SAMPLE,
    MIC_FALLING_SAMPLE
} AUDIO_AdcSampleEdgeEnum;
```

- MIC_RISING_SAMPLE：数字 MIC 采集时钟上升沿数据
- MIC_FALLING_SAMPLE：数字 MIC 采集时钟下降沿数据

18.1.8. 选择 DMA 输出 MIC 数据类型枚举

```
typedef enum
{
    MIC_MODE_ANA,
    MIC_MODE_PDM
} AUDIO_AdcModeEnum;
```

- MIC_MODE_ANA：DMA 输出模拟 MIC 采集的数据
- MIC_MODE_PDM：DMA 输出数字 MIC 采集的数据

18.1.9. MIC 初始化结构体

```
typedef struct
{
    AUDIO_SampleRateEnum      sampleRate;
    AUDIO_AdcSampleEdgeEnum pdmSampleEdge;
    AUDIO_AdcModeEnum          mode;
} AUDIO_AdcInitStruct;
```

- sampleRate：采样率设置，详见 MIC 采样率枚举

- pdm_data_mode：PDM 采集数据时机，详见数字 MIC 数据采集数据类型枚举
- mode：配置 DMA 输出数字 MIC 还是模拟 MIC 的数据，详见选择 DMA 输出 MIC 数据类型枚举

18.2. MIC 操作接口汇总

函数类型	函数名	说明
MIC 相关接口	AUDIO_AdcInit	MIC 初始化
	AUDIO_AdcDmaBufInit	MIC DMA 配置
	AUDIO_AdcCmd	MIC DMA 采样使能
	AUDIO_AdcSetVol	MIC 采样音量配置
	AUDIO_AdcSetMute	MIC 禁音使能
	AUDIO_AdcGetDataCount	获取 MIC DMA 数据长度
	AUDIO_AdcGetData	获取 MIC 数据

18.3. MIC 接口说明

18.3.1. void AUDIO_AdcInit(AUDIO_AdcInitStruct *adcInitStruct)

- 描述：MIC 初始化
- 参数：
  - adcInitStruct：详见 MIC 初始化结构体
- 返回值：none

18.3.2. void AUDIO_AdcDmaBufInit(uint8_t *buf,uint16_t bufSize)

- 描述：MIC DMA 缓冲区配置
- 参数：
  - buf：MIC DMA 起始地址
  - bufSize：MIC DMA 的长度
- 返回值：none

18.3.3. void AUDIO_AdcCmd(FunctionalState state)

- 描述：MIC 采样使能
- 参数：
  - state：MIC DMA 传输使能：
    - ENABLE： 使能
    - DISABLE：失能
- 返回值：none

18.3.4. void AUDIO_AdcSetVol(uint16_t vol)

- 描述：设置 MIC 采样音量配置
- 参数：
  - vol：MIC 音量配置，范围为 0x0~0x3fff

    通过 ADC_FADE 来设置音量渐入/渐出功能；通过 ADC_VOL_STEP 来设置音量渐入/渐出的步长；默认使用渐入/渐出功能
- 返回值：none

18.3.5. void AUDIO_AdcSetMute(FunctionalState newState)

- 描述：MIC 禁音功能使能
- 参数：
  - newState：使能或失能静音功能：
    - DISABLE：失能
    - ENABLE：使能
- 返回值：none

18.3.6. uint32_t AUDIO_AdcGetDataCount(void)

- 描述：获取 MIC 的 DMA 缓冲区可用数据长度
- 参数：none
- 返回值：MIC DMA 当前还未读取的数据长度

18.3.7. uint32_t AUDIO_AdcGetData(uint8_t *buf,uint32_t len)

- 描述：获取 MIC DMA 数据
- 参数：
  - buf：存放 DMA 数据的数组
  - len：读取的数据长度
- 返回值：实际读取的数据长度

18.4. demo 说明

- 演示 MIC 通过模拟 MIC 和数字 MIC 采样并通过 DAC 播放或 UART 输出原始数据，关键代码说明如下：

MIC 的宏定义和配置

```
#define SAMPLE_RATE                 SAMPLE_RATE_8K
#define MIC_MODE                    MIC_MODE_ANA
#define MIC_DATA_TO_UART            (0)

uint8_t mic_dma_buf[AUDIO_DATA_MS(200,SAMPLE_RATE)] = {0};
uint8_t read_buf[AUDIO_DATA_MS(50,SAMPLE_RATE)] = {0};
```

配置 MIC 的模式、采样率、输出模式和 DMA 缓冲区数组，默认输出模式是 DAC 输出 MIC 的采样数据

先初始化时钟和串口打印的 UART 端口

```
SYSCTRL_ChipInit();
printport_init();
```

判断是使用什么 MIC 模式，DMA 输出什么数据，选择初始化具体 GPIO

```
AUDIO_AdcInitStruct adcInitStruct;

adcInitStruct.sampleRate=SAMPLE_RATE;
if(MIC_MODE==MIC_MODE_PDM)
{
    GPIO_Config(GPIO_PDM_CLK_PIN,GPIO_MODE_MICCLK_PDM);
    GPIO_Config(GPIO_PDM_DATA_PIN,GPIO_MODE_MICDAT_PDM);
    adcInitStruct.mode=MIC_MODE_PDM;
    adcInitStruct.pdmSampleEdge=MIC_RISING_SAMPLE;
}
else
{
    GPIO_Config(GPIO_17,GPIO_MODE_HI_Z);
    GPIO_Config(GPIO_18,GPIO_MODE_HI_Z);
    GPIO_Config(GPIO_19,GPIO_MODE_HI_Z);
    adcInitStruct.mode=MIC_MODE_ANA;
}
AUDIO_AdcInit(&adcInitStruct);
AUDIO_AdcDmaBufInit(mic_dma_buf,sizeof(mic_dma_buf));
AUDIO_AdcSetVol(0x1000);
```

```
AUDIO_AdcCmd(ENABLE);
```

判断是 PDM 模式还是模拟 MIC 模式，配置 MIC 采样率，初始化 DMA 并设置音量然后使能 DMA，MIC 开始采集数据

DAC 输出模式

```
AUDIO_DacInit(SAMPLE_RATE);
AUDIO_DacDmaBufInit(mic_dma_buf, sizeof(mic_dma_buf));
AUDIO_DacSetVol(0);
AUDIO_DacCmd(ENABLE);
```

将采样率与 MIC 的采样率同步，并把 DAC 的 DMA 地址配置为 MIC 的 DMA 地址，设置音量并使能 DAC DMA

UART 输出模式

```
 while(1)
 {
    #if (MIC_DATA_TO_UART==1)
     if(AUDIO_AdcGetDataCount()>sizeof(read_buf))
     {
          AUDIO_AdcGetData(read_buf,sizeof(read_buf));
          UART_SendBuff(PRINT_PORT,read_buf,sizeof(read_buf));
     }
    #endif
}
```

如果 MIC 采样的数据大于 read_buf 缓冲区长度则将数据取出通过 UART 输出

19. DAC

19.1. 宏定义、枚举及结构体说明

19.1.1. 计算 DAC 输出数据量宏定义

```
#define AUDIO_DATA_MS(ms,sample_rate)                    ((ms)*((sample_rate)*(2)))
```

- ms：采样时长
- sample_rate：采样率，应填入采样率枚举

19.1.2. DAC 单次、连续输出模式宏定义

```
#define DAC_MODE_SINGLE                                  (0)
#define DAC_MODE_CYCLE                                   (1)
```

19.1.3. DAC 采样率枚举

```
typedef enum
{
    SAMPLE_RATE_8K          =  8,
    SAMPLE_RATE_16K         = 16,
    SAMPLE_RATE_24K         = 24,
    SAMPLE_RATE_32K         = 32,
    SAMPLE_RATE_44P1K       = 44,
    SAMPLE_RATE_48K         = 48
} AUDIO_SampleRateEnum;
```

与 MIC 的采样率枚举用法一致

19.2. DAC 接口汇总

函数类型	函数名	说明
DAC 相关接口	AUDIO_DacInit	DAC 初始化
	AUDIO_DacSetVol	DAC 输出音量配置

	AUDIO_DacDmaBufInit	DAC DMA 配置
	AUDIO_DacPutData	放置 DAC 输出数据
	AUDIO_DacPutZero	DAC 输出 0
	AUDIO_DacSetMode	DAC 模式设置
	AUDIO_DacSetMute	DAC 禁音使能
	AUDIO_DacCmd	DAC 输出使能
	AUDIO_DacGetAvailableBufSize	获取 DAC 缓冲区可填充数据的空间
	AUDIO_SetDacIrq	DAC 中断配置
	AUDIO_SetDacIrqLevel	DAC 中断水位设置
	AUDIO_ClearDacIrq	清除 DAC 中断标志位

19.3. DAC 接口说明

19.3.1. void AUDIO_DacInit(uint32_t sampleRate)

- 描述：DAC 初始化配置
- 参数：
  - sampleRate：DAC 采样率配置,详见 DAC 采样率枚举
- 返回值：none

19.3.2. void AUDIO_DacSetVol(uint8_t vol)

- 描述：设置 DAC 输出音量
- 参数：

- vol：DAC 音量设置，范围 0~7；0 为最大音量，7 为最小音量

- 返回值：none

19.3.3. void AUDIO_DacDmaBufInit(uint8_t *buf, uint32_t len)

- 描述：DAC DMA 缓冲区配置
- 参数：
  - buf：DAC DMA 起始地址配置
  - len：DMA 数据长度
- 返回值：none

19.3.4. uint32_t AUDIO_DacPutData(uint8_t *buf, uint32_t len)

- 描述：放置 DAC 数据
- 参数：
  - buf：存放 DAC 输出数据的数组起始地址
  - len：输出的数据长度
- 返回值：放置 DAC 数据的长度

19.3.5. uint32_t AUDIO_DacPutZero(uint32_t len)

- 描述：DAC 输出数据 0
- 参数：
  - len：输出的数据长度
- 返回值：实际输出的长度

19.3.6. void AUDIO_DacSetMode(uint8_t mode)

- 描述：设置 DAC 输出模式
- 参数：
  - mode：DAC 输出模式设置，详见 DAC_MODE_XX 宏定义组
- 返回值：none

19.3.7. void AUDIO_DacSetMute(FunctionalState mute)

- 描述：DAC 禁音使能
- 参数：
  - mute：DAC 静音功能使能
    - DISABLE：DAC 静音功能失能
    - ENABLE：DAC 静音功能使能
- 返回值：none

19.3.8. void AUDIO_DacCmd(FunctionalState state)

- 描述：DAC 输出使能
- 参数：
  - state：DAC DMA 使能
    - DISABLE：失能 DAC DMA，DAC 停止输出
    - ENABLE：使能 DAC DMA，DAC 开始输出
- 返回值：none

19.3.9. uint32_t AUDIO_DacGetAvailableBufSize(void)

- 描述：获取 DAC DMA 可填充数据的空间

- 参数：none
- 返回值：当前 DMA 空余空间

19.3.10. void AUDIO_SetDacIrq(FunctionalState state)

- 描述：DAC 中断使能
- 参数：
  - state：DAC 中断使能
    - DISABLE：失能 DAC 中断
    - ENABLE：使能 DAC 中断
- 返回值：none

19.3.11. void AUDIO_SetDacIrqLevel(uint16_t level)

- 描述：设置 DAC 中断的水位
- 参数：
  - level：当 DMA 读指针距离结束地址为 level 时触发 DAC 水位中断，必须为 4 的倍数
- 返回值：none

19.3.12. void AUDIO_ClearDacIrq(void)

- 描述：清除 DAC 中断标志位
- 参数：none
- 返回值：none

19.4. demo 说明

- 演示 DAC 在单次模式和循环模式下播放音频功能，关键代码说明如下：

DAC 配置宏定义

```
#define SAMPLING_RATE                   SAMPLE_RATE_8K
#define DAC_DMA_MODE                     DAC_MODE_CYCLE
#define DAC_STATUS_PLAY_START          0
#define DAC_STATUS_HALF_SIZE_IRQ      1
#define DAC_STATUS_BUF_END_IRQ         2
#define DAC_STATUS_PLAY_END            3

#define DAC_PACKET_LEN          AUDIO_DATA_MS(40,SAMPLING_RATE)
uint8_t dac_dma_buf[AUDIO_DATA_MS(80,SAMPLING_RATE)] = {0};
volatile uint8_t dac_status = DAC_STATUS_PLAY_START;
```

配置 DAC 的采样率，水位播放进度和音频播放进度状态宏定义，DAC 的 DMA 缓冲区大小，DAC 单包长度宏定义，配置 DAC 连续还是单次输出

先初始化时钟和串口打印的 UART 端口

```
SYSCTRL_ChipInit();
printport_init();
```

初始化 DAC GPIO 配置

```
void DAC_GpioInit(void)
{
    GPIO_Config(GPIO_DAC_OUT_P_PIN, GPIO_MODE_DAC_OUT);
    GPIO_Config(GPIO_DAC_OUT_N_PIN, GPIO_MODE_DAC_OUT);
}
```

DAC 的 DMA 初始化中断启用

```
AUDIO_DacInit(SAMPLING_RATE);
AUDIO_DacSetMode(DAC_DMA_MODE);
AUDIO_DacSetVol(0);
#if (DAC_DMA_MODE == DAC_MODE_CYCLE)
memset(dac_dma_buf, 0, sizeof(dac_dma_buf));
AUDIO_DacDmaBufInit(dac_dma_buf, sizeof(dac_dma_buf));
#endif
PLIC_SetPriority(GDAC_IRQn, 1);
PLIC_EnableIRQ(GDAC_IRQn);
```

DAC 连续模式(音频数据较大时)

```
while(1)
{
    if(dac_status == DAC_STATUS_PLAY_START)
    {
#if (DAC_DMA_MODE == DAC_MODE_CYCLE)
        packet_len = sizeof(dac_dma_buf);/* 默认声音文件大于 dma buf */
        AUDIO_DacPutData((uint8_t*)wav,packet_len);
        played_len = packet_len;
        AUDIO_DacSetIrqLevel(sizeof(dac_dma_buf)/2);
        dac_status = DAC_STATUS_HALF_SIZE_IRQ;
#endif
        AUDIO_DacCmd(ENABLE);
        AUDIO_DacSetIrq(ENABLE);
        yc_log("play start");
    }
    else if(dac_status == DAC_STATUS_PLAY_END)
    {
        yc_log("play end");
        delay_ms(1000);
        dac_status = DAC_STATUS_PLAY_START;
    }
}
```

播放开始时设置水位中断，将状态更新为播放过半，在播放过半时会进入中断，然后开启DMA 和中断，播放结束时延时 1000ms 后将音频播放进度的状态更新为开始重新播放音频数据

```
void GDAC_IRQHandler(void)
{
     AUDIO_DacClearIrq();
     #if (DAC_DMA_MODE == DAC_MODE_CYCLE)
     if(played_len < sizeof(wav))
     {
          if(sizeof(wav)-played_len > sizeof(dac_dma_buf)/2)
          {
               packet_len = sizeof(dac_dma_buf)/2;
               AUDIO_DacPutData((uint8_t*)&wav[played_len],packet_len);
          }
          else
          {
               packet_len = sizeof(wav)-played_len;
               AUDIO_DacPutData((uint8_t*)&wav[played_len],packet_len);
               AUDIO_DacPutZero(sizeof(dac_dma_buf)/2 - packet_len);
```

```
            }
            played_len += packet_len;
            if(dac_status == DAC_STATUS_HALF_SIZE_IRQ)
            {
                AUDIO_DacSetIrqLevel(0);
                dac_status = DAC_STATUS_BUF_END_IRQ;
            }
            else
            {
                AUDIO_DacSetIrqLevel(sizeof(dac_dma_buf)/2);
                dac_status = DAC_STATUS_HALF_SIZE_IRQ;
            }
        }
        else
        {
            AUDIO_DacCmd(DISABLE);
            AUDIO_DacSetIrq(DISABLE);
            played_len = 0;
            dac_status = DAC_STATUS_PLAY_END;
        }
        #endif
}
```

进入中断后，当音频数据的剩余数据量大于包长时，则将每包长度设置为 DMA 缓冲区的一半然后不断往 DMA 缓冲区中放入音频数据，然后通过水位播放进度的态位不断更新水位以确保播放和数据的连续性；如果音频数据剩余长度小于单包长度时，则填 0 填充剩余空间，最后已播放长度大于音频数据的长度时就关闭 DAC 输出和 DAC 的 DMA 使能，并更新音频数据播放进度状态。

DAC 单次输出模式(音频数据较小时或者 flash 中的音频文件可以直接播放的场景)

```
while(1)
{
    if(dac_status == DAC_STATUS_PLAY_START)
    {
        AUDIO_DacDmaBufInit((uint8_t*)wav, sizeof(wav));
        AUDIO_DacSetIrqLevel(0);
        dac_status = DAC_STATUS_BUF_END_IRQ;
        #endif
        AUDIO_DacCmd(ENABLE);
        AUDIO_DacSetIrq(ENABLE);
```

```
            yc_log("play start");
        }
        else if(dac_status == DAC_STATUS_PLAY_END)
        {
            yc_log("play end");
            delay_ms(1000);
            dac_status = DAC_STATUS_PLAY_START;
        }
}
```

将音频数据填入 DMA，然后开启中断更新水位，更新音频播放进度的状态，当播放结束时延时 1000ms 后更新音频播放进度的状态重新播放

```
void GDAC_IRQHandler(void)
{
    AUDIO_DacCmd(DISABLE);
    AUDIO_DacSetIrq(DISABLE);
    dac_status = DAC_STATUS_PLAY_END;
}
```

播放完成后在中断中关闭 DAC 和 DAC 的 DMA 更新音频播放进度的状态

20. USB

20.1. 宏定义、枚举及结构体说明

20.1.1. USB CDC

20.1.1.1. USB CDC 端点设置宏定义

```
#define TRANS_EP_NUM                3   /* 应用数据传输端点个数,用于申请及管理中转 buffer */
#define CDC_IN_EP                   VIRTUAL_IN_EP3     /* EP1 for data IN */
#define CDC_OUT_EP                  VIRTUAL_OUT_EP3    /* EP1 for data OUT */
#define CDC_CMD_EP                  VIRTUAL_IN_EP2     /* EP2 for CDC commands */
#define VIRTUAL_IN_EP1              0x84
#define VIRTUAL_OUT_EP1             0x04
#define VIRTUAL_IN_EP2              0x86
#define VIRTUAL_OUT_EP2             0x06
#define VIRTUAL_IN_EP3              0x88
#define VIRTUAL_OUT_EP3             0x08
```

- TRANS_EP_NUM：应用于数据传输端点的个数
- CDC_IN_EP：输入端点的宏定义
- CDC_OUT_EP：输出端点的宏定义
- CDC_CMD_EP：用于 CDC 命令的端点宏定义
- VIRTUAL_IN_EP1：将物理端点 1 的 IN 映射为虚拟端点的 IN 的宏定义，代码中为 0x84 则映射为端点 4 的 IN
- VIRTUAL_OUT_EP1：将物理端点 1 的 OUT 映射为虚拟端点的 OUT 的宏定义，代码中为 0x04 则映射为端点 4 的 OUT
- VIRTUAL_IN_EP2：将物理端点 2 的 IN 映射为虚拟端点的 IN 的宏定义，代码中为 0x86 则映射为端点 6 的 IN

- VIRTUAL_OUT_EP2：将物理端点 2 的 OUT 映射为虚拟端点的 OUT 的宏定义，代码中为 0x06 则映射为端点 6 的 OUT
- VIRTUAL_IN_EP3：将物理端点 3 的 IN 映射为虚拟端点的 IN 的宏定义，代码中为 0x88 则映射为端点 8 的 IN
- VIRTUAL_OUT_EP3：将物理端点 3 的 OUT 映射为虚拟端点的 OUT 的宏定义，代码中为 0x08 则映射为端点 8 的 OUT

20.1.1.2. USB CDC 数据包最大长度宏定义

```
#define CDC_DATA_MAX_PACKET_SIZE        64    /* Endpoint IN & OUT Packet size */
#define CDC_CMD_PACKET_SZE              8     /* Control Endpoint Packet size */
#define CDC_DATA_IN_PACKET_SIZE         CDC_DATA_MAX_PACKET_SIZE
#define CDC_DATA_OUT_PACKET_SIZE        CDC_DATA_MAX_PACKET_SIZE
```

20.1.1.3. USB CDC 描述符长度宏定义

```
#define USB_CDC_CONFIG_DESC_SIZ         (67)
#define USB_CDC_DESC_SIZ                (67-9)
```

20.1.1.4. USB CDC 应用层数据缓冲区长度宏定义

```
#define CDC_IN_BUF_SIZE                 (CDC_DATA_IN_PACKET_SIZE*10)
#define CDC_OUT_BUF_SIZE                (CDC_DATA_OUT_PACKET_SIZE*10)
```

20.1.1.5. USB CDC 描述符位置

USB 设备描述符位于.\ModuleDemo\USB\usb_cdc\user\cdc\src\usbd_desc.c 文件中数组名为 USBD_DeviceDesc，USB 配置描述于.\ModuleDemo\USB\usb_cdc\user\cdc\src\usbd_cdc_core.c 文件中数组名为 usbd_cdc_CfgDesc

20.1.1.6. USB CDC 接口应用函数结构体

```
typedef struct _Device_cb
{
    uint8_t (*Init)(void *pdev, uint8_t cfgidx);
    uint8_t (*DeInit)(void *pdev, uint8_t cfgidx);
    /* Control Endpoints*/
    uint8_t (*Setup)(void *pdev, USB_SETUP_REQ   *req);
    uint8_t (*EP0_TxSent)(void *pdev);
    uint8_t (*EP0_RxReady)(void *pdev);
    /* Class Specific Endpoints*/
    uint8_t (*DataIn)(void *pdev, uint8_t epnum);
    uint8_t (*DataOut)(void *pdev, uint8_t epnum);
    uint8_t (*SOF)(void *pdev);
    uint8_t (*IsoINIncomplete)(void *pdev);
    uint8_t (*IsoOUTIncomplete)(void *pdev);

    uint8_t   *(*GetConfigDescriptor)(uint8_t speed, uint16_t *length);
#ifdef USB_OTG_HS_CORE
    uint8_t   *(*GetOtherConfigDescriptor)(uint8_t speed, uint16_t *length);
#endif

#ifdef USB_SUPPORT_USER_STRING_DESC
    uint8_t   *(*GetUsrStrDescriptor)(uint8_t speed, uint8_t index,   uint16_t *length);
#endif

} USBD_Class_cb_TypeDef;
```

USB 不同接口应用的初始化，反初始化，令牌包解析接收，端点 0 发送，端点 0 接收就绪，指定端点输入输出，SOF 包接收状态，同步传输发送接收等函数进行实现

```
USBD_Class_cb_TypeDef USBD_CDC_cb =
    {
        usbd_cdc_Init,
        usbd_cdc_DeInit,
        usbd_cdc_Setup,
        NULL,                           /* EP0_TxSent */
        usbd_cdc_EP0_RxReady,
        usbd_cdc_DataIn,
        usbd_cdc_DataOut,
        usbd_cdc_SOF,
        NULL,
        NULL,
        USBD_cdc_GetCfgDesc,
#ifdef USE_USB_OTG_HS
        USBD_cdc_GetOtherCfgDesc, /* use same cobfig as per FS */
```

```
#endif                                  /* USE_USB_OTG_HS   */
};
```

- usbd_cdc_Init：USB CDC 初始化函数
- usbd_cdc_DeInit：USB CDC 反初始化函数
- usbd_cdc_Setup：USB CDC 令牌包接收解析函数
- usbd_cdc_EP0_RxReady：USB CDC 端点 0(控制端点)接收数据
- usbd_cdc_DataIn：USB CDC 输入函数(设备向主机输入数据)
- usbd_cdc_DataOut：USB CDC 输出函数(主机向设备输出数据)
- usbd_cdc_SOF：接收 SOF 包状态函数
- USBD_cdc_GetCfgDesc：获取 USB CDC 配置描述符函数

20.1.1.7. USB CDC 应用数据缓冲区结构体

```
struct APP_DATA_STRUCT_DEF
{
    /* 通过 PC 接收数据(将会通过串口发出的数据)信息 */
    struct {
        uint8_t app_in_buf[TRANS_EP_NUM][CDC_IN_BUF_SIZE];      /* 接收数据缓存 */
        volatile uint32_t app_in_buf_wptr[TRANS_EP_NUM];        /* 缓存接收数据起始偏移地址 */
        volatile uint32_t app_in_buf_rptr[TRANS_EP_NUM];        /* 缓存读取数据起始偏移地址 */
        volatile uint8_t app_in_ep_busy[TRANS_EP_NUM];
    } in_struct;
    /* 通过串口接收数据(将会通过 USB 发送给 PC 的数据)信息 */
    struct{
        uint8_t app_out_buf[TRANS_EP_NUM][CDC_OUT_BUF_SIZE];    /* 接收数据缓存 */
        volatile uint32_t app_out_buf_wptr[TRANS_EP_NUM];       /* 缓存接收数据起始偏移地址 */
        volatile uint32_t app_out_buf_rptr[TRANS_EP_NUM];       /* 缓存读取数据起始偏移地址 */
    } out_struct;
    uint8_t COM_config_cmp;  /* 串口状态(1:串口处于连接状态；0：串口处于断开状态) */
};
```

20.1.1.8. USB CDC 用于处理类请求的结构体

```
typedef struct _CDC_IF_PROP
{
  uint16_t (*pIf_Init)      (void);
  uint16_t (*pIf_DeInit)    (void);
  uint16_t (*pIf_Ctrl)      (uint32_t Cmd, uint8_t* Buf, uint32_t Len);
  uint16_t (*pIf_DataTx)    (uint8_t* Buf, uint32_t Len);
  uint16_t (*pIf_DataRx)    (uint8_t* Buf, uint32_t Len);
}
CDC_IF_Prop_TypeDef;
extern CDC_IF_Prop_TypeDef APP_FOPS;
```

USB CDC 处理类请求的函数 API 在 usbd_cdc_vcp.c 文件中

20.1.2. USB HID

20.1.2.1. USB VID 和 PID 宏定义

```
#define USBD_VID                    0x3151
#define USBD_PID                    0x6012
```

20.1.2.2. USB HID 端点数量及虚拟映射宏定义

```
#define TRANS_EP_NUM                1    /* 应用数据传输端点个数,用于申请及管理中转 buffer */
#define HID_IN_EP                   0x84
#define HID_OUT_EP                  0x04
```

- TRANS_EP_NUM：设置传输的端点数量
- HID_IN_EP：将物理端点 1 的 IN 映射为虚拟端点的 IN 的宏定义，代码中为 0x84 则映射为端点 4 的 IN
- HID_OUT_EP：将物理端点 1 的 OUT 映射为虚拟端点的 OUT 的宏定义，代码中为 0x04 则映射为端点 4 的 OUT

20.1.2.3. USB HID 报告 ID 大小数量宏定义

```
#define HID_REPORT_ID               0xf2
#define HID_REPORT_COUNT            0x3f
```

```
3  #define HID_REPORT_SIZE                 8
```

20.1.2.4. USB HID 数据包最大长度宏定义

```
1  #define HID_EP_MAX_PACKET               64
2  #define HID_EPIN_SIZE                   HID_EP_MAX_PACKET
3  #define HID_EPOUT_SIZE                  HID_EP_MAX_PACKET
```

20.1.2.5. USB HID 描述符长度宏定义

```
1  #define USB_HID_CONFIG_DESC_SIZ         41
2  #define USB_HID_DESC_SIZ                9
3  #define HID_MOUSE_REPORT_DESC_SIZE    33
4  #define HID_INPUT_REPORT_DESC_SIZE    26
```

- USB_HID_CONFIG_DESC_SIZ：USB 配置描述符大小
- USB_HID_DESC_SIZ：USB HID 描述符大小
- HID_MOUSE_REPORT_DESC_SIZE：USB HID 报告描述符大小(MOUSE)
- HID_INPUT_REPORT_DESC_SIZE：USB HID 报告描述符大小(INPUT)

20.1.2.6. USB HID 应用层数据缓冲区长度宏定义

```
1  #define APP_IN_BUF_LEN                  (HID_EP_MAX_PACKET * 10)
2  #define APP_OUT_BUF_LEN                 (HID_EP_MAX_PACKET * 10)
```

20.1.2.7. USB HID 描述符位置

USB 设备描述符位于.\ModuleDemo\USB\usb_hid\user\hid\src\usbd_desc.c 文件中数组名为 USBD_DeviceDesc，USB 配置描述符,USB HID 报告描述符在.\ModuleDemo\USB\usb_hid\user\hid \src\usbd_cdc_core.c 文件中，配置描述符名为 USBD_HID_CfgDesc，报告描述符名为 HID_INPUT_ReportDesc，此 USB 报告描述符仅用于数据传输而没有其他功能

而 USB HID 的键鼠 demo 中，键盘和鼠标分别使用一个报告描述符，使用两个接口来实现键鼠复合功能

键盘的报告描述符

```
__ALIGN_BEGIN static uint8_t HID_kaypad_ReportDesc[USB_HID_KEYPAD_REPORTDESC_SIZE] __ALIGN_END =
    {
        0x05,0x01,// USAGE_PAGE (Generic Desktop)
        0x09,0x06,// USAGE (Keyboard)
        0xA1,0x01,// COLLECTION (Application)

        0x05,0x07,//   USAGE_PAGE (Keyboard Key Codes)
        0x19,0xE0,//   USAGE_MINIMUM (Keyboard LeftControl)
        0x29,0xE7,//   USAGE_MAXIMUM (Keyboard Right GUI)
        0x15,0x00,//   LOGICAL_MINIMUM (0)
        0x25,0x01,//   LOGICAL_MAXIMUM (1)
        0x75,0x01,//   REPORT_SIZE (1)
        0x95,0x08,//   REPORT_COUNT (8)
        0x81,0x02,//   INPUT (Data, Var, Abs)
        0x95,0x01,//   REPORT_COUNT (1)
        0x75,0x08,//   REPORT_SIZE (8)
        0x81,0x01,//   INPUT (Constant, Array, Abs)

        0x95,0x05,//   REPORT_COUNT (5)
        0x75,0x01,//   REPORT_SIZE (1)
        0x05,0x08,//   USAGE_PAGE (LEDs)
        0x19,0x01,//   USAGE_MINIMUM (Num Lock)
        0x29,0x05,//   USAGE_MAXIMUM (Kana)
        0x91,0x02,//   OUTPUT (Data, Var, Abs)
        0x95,0x01,//   REPORT_COUNT (1)
        0x75,0x03,//   REPORT_SIZE (3)
        0x91,0x01,//   OUTPUT (Constant, Array, Abs)

        0x95,0x06,//   REPORT_COUNT (6)
        0x75,0x08,//   REPORT_SIZE (8)
        0x15,0x00,//   LOGICAL_MINIMUM (0)
        0x25,0x68,//   LOGICAL_MAXIMUM (104)
        0x05,0x07,//   USAGE_PAGE (Keyboard Key Codes)
        0x19,0x00,//   USAGE_MINIMUM (0)
        0x29,0x68,//   USAGE_MAXIMUM (104)
        0x81,0x00,//   INPUT (Data, Array, Abs)
        0xC0,
```

```
};
```

鼠标报告描述符

```
__ALIGN_BEGIN static uint8_t HID_mouse_ReportDesc[USB_HID_MOUSE_REPORTDESC_SIZE]
__ALIGN_END =
{
/* Mouse pointer */
0x05, 0x01, //        USAGE_PAGE (Generic Desktop)
0x09, 0x02, //        USAGE (Mouse)
0xA1, 0x01, //        COLLECTION (Application)
0x85, 0x01, //        REPORT_ID (1)
0x09, 0x01, //        USAGE (Pointer)
0xA1, 0x00, //        COLLECTION (Physical)
/* The five buttons of the mouse */
0x05, 0x09, //        USAGE_PAGE (Button)
0x19, 0x01, //        USAGE_MINIMUM (Button 1)
0x29, 0x05, //        USAGE_MAXIMUM (Button 5)
0x15, 0x00, //        LOGICAL_MINIMUM (0)
0x25, 0x01, //        LOGICAL_MAXIMUM (1)
0x95, 0x05, //        REPORT_COUNT (5)
0x75, 0x01, //        REPORT_SIZE (1)
0x81, 0x02, //        INPUT (Data,Var,Abs)
0x95, 0x01, //        REPORT_COUNT (1)
0x75, 0x03, //        REPORT_SIZE (3)
0x81, 0x01, //        INPUT (Constant,Array,Abs)
/* The X and Y axes of the mouse */
0x05, 0x01, //        USAGE_PAGE (Generic Desktop)
0x09, 0x30, //        USAGE (X)
0x09, 0x31, //        USAGE (Y)
0x16, 0x01,0xF8,    //        LOGICAL_MINIMUM (-2047)
0x26, 0xFF, 0x07, //        LOGICAL_MAXIMUM (2047)
0x75, 0x10, //        REPORT_SIZE (16)
0x95, 0x02, //        REPORT_COUNT (2)
0x81, 0x06, //        INPUT (Data,Var,Rel)
/* Mouse wheel movement amount */
0x09, 0x38, //        USAGE (Wheel)
0x15, 0x81, //        LOGICAL_MINIMUM (-127)
0x25, 0x7F, //        LOGICAL_MAXIMUM (127)
0x75, 0x08, //        REPORT_SIZE (8)
0x95, 0x01, //        REPORT_COUNT (1)
0x81, 0x06, //        INPUT (Data,Var,Rel)
/* User-defined purpose */
0x05, 0x0C, //        USAGE_PAGE (Consumer)
0x0A, 0x38, 0x02, //        USAGE (AC Pan)
```

```
0x95, 0x01, //        REPORT_COUNT (1)
0x81, 0x06, //        REPORT_SIZE (8)
0xC0, //     END_COLLECTION
0xC0 //      END_COLLECTION
};
```

20.1.2.8. USB HID 接口应用函数结构体

```
typedef struct _Device_cb
{
      uint8_t (*Init)(void *pdev, uint8_t cfgidx);
      uint8_t (*DeInit)(void *pdev, uint8_t cfgidx);
      /* Control Endpoints*/
      uint8_t (*Setup)(void *pdev, USB_SETUP_REQ   *req);
      uint8_t (*EP0_TxSent)(void *pdev);
      uint8_t (*EP0_RxReady)(void *pdev);
      /* Class Specific Endpoints*/
      uint8_t (*DataIn)(void *pdev, uint8_t epnum);
      uint8_t (*DataOut)(void *pdev, uint8_t epnum);
      uint8_t (*SOF)(void *pdev);
      uint8_t (*IsoINIncomplete)(void *pdev);
      uint8_t (*IsoOUTIncomplete)(void *pdev);

      uint8_t   *(*GetConfigDescriptor)(uint8_t speed, uint16_t *length);
#ifdef USB_OTG_HS_CORE
      uint8_t   *(*GetOtherConfigDescriptor)(uint8_t speed, uint16_t *length);
#endif

#ifdef USB_SUPPORT_USER_STRING_DESC
      uint8_t   *(*GetUsrStrDescriptor)(uint8_t speed, uint8_t index,   uint16_t *length);
#endif
} USBD_Class_cb_TypeDef;
```

```
USBD_Class_cb_TypeDef USBD_HID_cb =
      {
            USBD_HID_Init,
            USBD_HID_DeInit,
            USBD_HID_Setup,
            NULL,                   /* EP0_TxSent */
            NULL,                   /* EP0_RxReady */
            USBD_HID_DataIn,        /* DataIn */
            USBD_HID_DataOut,       /* DataOut */
            NULL,                   /* SOF */
            NULL,
```

```
        NULL,
        USBD_HID_GetCfgDesc,
#ifdef USB_OTG_HS_CORE
        USBD_HID_GetCfgDesc, /* use same config as per FS */
#endif
};
```

- USBD_HID_Init：USB HID 初始化函数
- USBD_HID_DeInit：USB HID 反初始化函数
- USBD_HID_Setup：USB HID 令牌包接收解析函数
- USBD_HID_DataIn：USB HID 输入函数(设备向主机输入数据)
- USBD_HID_DataOut：USB HID 输出函数(主机向设备输出数据)
- USBD_HID_GetCfgDesc：获取 USB HID 配置描述符函数

20.1.2.9. USB HID 应用数据缓冲区结构体

```
struct APP_DATA_STRUCT_DEF
{
    struct {
        uint8_t app_in_buf[TRANS_EP_NUM][APP_IN_BUF_LEN];
        volatile uint32_t app_in_buf_wptr[TRANS_EP_NUM];
        volatile uint32_t app_in_buf_rptr[TRANS_EP_NUM];
        volatile uint8_t app_in_ep_busy[TRANS_EP_NUM];
    } in_struct;

    struct{
        uint8_t app_out_buf[TRANS_EP_NUM][APP_OUT_BUF_LEN];
        volatile uint32_t app_out_buf_wptr[TRANS_EP_NUM];
        volatile uint32_t app_out_buf_rptr[TRANS_EP_NUM];
    } out_struct;
} ;
```

20.1.3. USB printer

20.1.3.1. USB printer 端点数量及虚拟映射宏定义

```
#define TRANS_EP_NUM                    1      /* 应用数据传输端点个数,用于申请及管理中转 buffer */
```

```
#define PRINTER_IN_EP                   0x84
#define PRINTER_OUT_EP                  0x04
```

- TRANS_EP_NUM：设置传输的端点格式
- PRINTER_IN_EP：将物理端点 1 的 IN 映射为虚拟端点的 IN 的宏定义，代码中为 0x84 则映射为端点 4 的 IN
- PRINTER_OUT_EP：将物理端点 1 的 OUT 映射为虚拟端点的 OUT 的宏定义，代码中为 0x04 则映射为端点 4 的 OUT

20.1.3.2. USB printer 描述符长度宏定义

```
#define USB_PRINTER_CONFIG_DESC_SIZ                 (32)
#define USB_PRINTER_DESC_SIZ                        (USB_PRINTER_CONFIG_DESC_SIZ-9)
#define USB_SIZ_DEVICE_DESC                         18
```

- USB_PRINTER_CONFIG_DESC_SIZ：USB 配置描述符大小
- USB_PRINTER_DESC_SIZ：USB printer 描述符大小
- USB_SIZ_DEVICE_DESC：USB 设备描述符

20.1.3.3. USB printer 数据包最大长度宏定义

```
#define PRINTER_DATA_MAX_PACKET_SIZE                64    /* Endpoint IN & OUT Packet size */
#define PRINTER_DATA_IN_PACKET_SIZE                 PRINTER_DATA_MAX_PACKET_SIZE
#define PRINTER_DATA_OUT_PACKET_SIZE                PRINTER_DATA_MAX_PACKET_SIZE
```

20.1.3.4. USB printer 应用层数据缓冲区长度宏定义

```
#define PRINTER_APP_OUT_BUF_SIZE (PRINTER_DATA_MAX_PACKET_SIZE*10)
#define PRINTER_APP_IN_BUF_SIZE                     (PRINTER_DATA_MAX_PACKET_SIZE*5)
```

20.1.3.5. USB printer 描述符位置

USB 设备描述符位于.\ModuleDemo\USB\usb_printer\user\printer\src\usbd_desc.c 文件中数组名为 USBD_DeviceDesc，USB 配置描述符,USB HID 报告描述符在.\ModuleDemo\USB\usb_printer\user\printer\src\usbd_printer_core.c 文件中，配置描述符名为 usbd_printer_CfgDesc，还有打印机 ID 描述符名为 print_id_descriptor

20.1.3.6. USB printer 接口应用函数结构体

```
typedef struct _Device_cb
{
    uint8_t (*Init)(void *pdev, uint8_t cfgidx);
    uint8_t (*DeInit)(void *pdev, uint8_t cfgidx);
    /* Control Endpoints*/
    uint8_t (*Setup)(void *pdev, USB_SETUP_REQ   *req);
    uint8_t (*EP0_TxSent)(void *pdev);
    uint8_t (*EP0_RxReady)(void *pdev);
    /* Class Specific Endpoints*/
    uint8_t (*DataIn)(void *pdev, uint8_t epnum);
    uint8_t (*DataOut)(void *pdev, uint8_t epnum);
    uint8_t (*SOF)(void *pdev);
    uint8_t (*IsoINIncomplete)(void *pdev);
    uint8_t (*IsoOUTIncomplete)(void *pdev);

    uint8_t   *(*GetConfigDescriptor)(uint8_t speed, uint16_t *length);
#ifdef USB_OTG_HS_CORE
    uint8_t   *(*GetOtherConfigDescriptor)(uint8_t speed, uint16_t *length);
#endif

#ifdef USB_SUPPORT_USER_STRING_DESC
    uint8_t   *(*GetUsrStrDescriptor)(uint8_t speed, uint8_t index,   uint16_t *length);
#endif

} USBD_Class_cb_TypeDef;
```

USB 不同接口应用的初始化，反初始化，令牌包解析接收，端点 0 发送，端点 0 接收就绪，指定端点输入输出，SOF 包接收状态，同步传输发送接收等函数进行实现

```
USBD_Class_cb_TypeDef USBD_PRINTER_cb =
    {
        usbd_printer_Init,
        usbd_printer_DeInit,
```

```
    usbd_printer_Setup,
    NULL, /* EP0_TxSent, */
    usbd_printer_EP0_RxReady,
    usbd_printer_DataIn,
    usbd_printer_DataOut,
    usbd_printer_SOF,
    NULL,
    NULL,
    USBD_printer_GetCfgDesc,
#ifdef USE_USB_OTG_HS
    USBD_printer_GetOtherCfgDesc, /* use same cobfig as per FS */
#endif                                          /* USE_USB_OTG_HS    */
};
```

- usbd_printer_Init：USB printer 初始化函数
- usbd_printer_DeInit：USB printer 反初始化函数
- usbd_printer_Setup：USB printer 令牌包接收解析函数
- usbd_printer_EP0_RxReady：USB printer 端点 0(控制端点)接收数据
- usbd_printer_DataIn：USB printer 输入函数(设备向主机输入数据)
- usbd_printer_DataOut：USB printer 输出函数(主机向设备输出数据)
- usbd_printer_SOF：接收 SOF 包后处理函数
- USBD_printer_GetCfgDesc：获取 USB printer 配置描述符函数

20.1.3.7. USB printer 应用数据缓冲区结构体

```
struct APP_DATA_STRUCT_DEF
{
    /* 通过 PC 接收数据(将会通过串口发出的数据)信息 */
    struct
    {
        uint8_t app_in_buf[TRANS_EP_NUM][PRINTER_APP_IN_BUF_SIZE];  // 接收数据缓存
        volatile uint32_t app_in_buf_wptr[TRANS_EP_NUM];                // 缓存接收数据起始偏移地址
        volatile uint32_t app_in_buf_rptr[TRANS_EP_NUM];                // 缓存读取数据起始偏移地址
        volatile uint8_t app_in_ep_busy[TRANS_EP_NUM];
    } in_struct;
```

```
    /* 通过串口接收数据(将会通过 USB 发送给 PC 的数据)信息 */
    struct
    {
        uint8_t app_out_buf[TRANS_EP_NUM][PRINTER_APP_OUT_BUF_SIZE]; // 接收数据缓存
        volatile uint32_t app_out_buf_wptr[TRANS_EP_NUM];                // 缓存接收数据起始偏移地址
        volatile uint32_t app_out_buf_rptr[TRANS_EP_NUM];                // 缓存读取数据起始偏移地址
    } out_struct;
} ;
```

20.2. USB 接口汇总

函数类型	USB 类型	函数名	说明
USB 相关接口	USB_CDC	usbd_cdc_Init	USB CDC 类初始化
		USBD_cdc_GetCfgDesc	获取 USB CDC 配置描述符
		usbd_cdc_DeInit	USB CDC 反初始化
		usbd_cdc_Setup	USB CDC 接口请求处理
		usbd_cdc_EP0_RxReady	USB CDC 收到类请求后回复函数
		CDC_GetEpIndex	获取实际物理端点
		usbd_cdc_DataIn	USB 底层发送函数
		usbd_cdc_DataOut	USB 底层接收函数
		CDC_DataIn	USB 应用层发送函数
		CDC_DataOut	USB 应用层接收函数
		CDC_GetInDataLen	获取应用层发送缓冲区未发送长度
		CDC_GetOutDataLen	获取应用层接收缓冲区未读取长度

		usbd_cdc_SOF	USB CDC 类处理 SOF 包
	USB_HID	USBD_HID_Init	USB HID 类初始化
		USBD_HID_DeInit	USB HID 类反初始化
		USBD_HID_GetCfgDesc	获取 USB HID 配置描述符
		USBD_HID_Setup	USB HID 接口请求处理
		HID_GetEpIndex	获取实际物理端点
		USBD_HID_DataIn	USB 底层发送函数
		USBD_HID_DataOut	USB 底层接收函数
		HID_DataIn	USB 应用层发送函数
		HID_DataOut	USB 应用层接收函数
		HID_GetInDataLen	获取应用层发送缓冲区未发送长度
		HID_GetOutDataLen	获取应用层接收缓冲区未读取长度
	USB_printer	usbd_printer_Init	USB printer 类初始化
		usbd_printer_DeInit	获取 USB printer 配置描述符
		USBD_printer_GetCfgDesc	USB printer 反初始化
		usbd_printer_Setup	USB printer 接口请求处理
		usbd_printer_EP0_RxReady	USB printer 收到类请求后回复函数
		PRINTER_GetEpIndex	获取实际物理端点
		usbd_printer_DataIn	USB 底层发送函数
		usbd_printer_DataOut	USB 底层接收函数

		PRINTER_DataIn	USB 应用层发送函数
		PRINTER_DataOut	USB 应用层接收函数
		PRINTER_GetInDataLen	获取应用层发送缓冲区未发送长度
		PRINTER_GetOutDataLen	获取应用层接收缓冲区未读取长度
		usbd_printer_SOF	USB printer 类处理 SOF 包

20.3. USB 接口说明

20.3.1. USB CDC

20.3.1.1. static uint8_t usbd_cdc_Init(void *pdev,uint8_t cfgidx)

- 描述：USB CDC 类初始化
- 参数：
  - pdev：USB 控制器核心结构体
  - cfgidx：配置索引
- 返回值：USB 设备状态

20.3.1.2. static uint8_t usbd_cdc_DeInit(void *pdev,uint8_t cfgidx)

- 描述：USB CDC 类反初始化
- 参数：
  - pdev：USB 控制器核心结构体
  - cfgidx：配置索引
- 返回值：USB 设备状态

20.3.1.3. static uint8_t usbd_cdc_Setup(void *pdev, USB_SETUP_REQ *req)

- 描述：USB CDC 类接口请求处理
- 参数：
  - pdev：USB 控制器核心结构体
  - req：USB 接收到的请求
- 返回值：USB 设备状态

20.3.1.4. static uint8_t usbd_cdc_EP0_RxReady(void *pdev)

- 描述：收到 USB CDC 类请求后的回复函数
- 参数：
  - pdev：USB 控制器核心结构体
- 返回值：USB 设备状态

20.3.1.5. static uint8_t *USBD_cdc_GetCfgDesc(uint8_t speed, uint16_t *length)

- 描述：获取 USB CDC 类配置描述符
- 参数：
  - speed：USB 的速度模式
  - length：USB CDC 配置描述符长度
- 返回值：USB CDC 配置描述符

20.3.1.6. static uint8_t CDC_GetEpIndex(uint8_t epNum)

- 描述：通过虚拟端点号获取物理端点号
- 参数：

- epNum：实际物理端点映射的虚拟端点号

- 返回值：实际物理端点号

20.3.1.7. static uint8_t usbd_cdc_DataIn(void *pdev, uint8_t epnum)

- 描述：发送 IN 包，该函数将应用层发送缓冲区数据写入底层 DMA
- 参数：
  - pdev：USB 控制器核心结构体
  - epnum：实际物理端点号
- 返回值：USB 设备状态

20.3.1.8. static uint8_t usbd_cdc_DataOut(void *pdev, uint8_t epnum)

- 描述：接收 OUT 包，该函数将底层 DMA 中数据写入应用层缓冲区数据。当应用层的缓冲区剩余空间小于两倍包长时将设备的端点的接收阻塞，当应用层缓冲区数据被读取时触发数据继续接收并解除阻塞。通过阻塞来防止应用层缓冲区的数据未读取就被覆盖。
- 参数：
  - pdev：USB 控制器核心结构体
  - epnum：实际物理端点号
- 返回值：USB 设备状态

20.3.1.9. uint32_t CDC_GetInDataLen(uint8_t ep_num)

- 描述：获取应用层发送缓冲区的未发送数据的长度
- 参数：
  - ep_num：实际物理端点映射的虚拟端点号
- 返回值：应用层发送缓冲区内还未发送的数据长度

20.3.1.10.uint32_t CDC_GetOutDataLen(uint8_t ep_num)

- 描述：获取应用层接收缓冲区未读取的数据长度
- 参数：
  - ep_num：实际物理端点映射的虚拟端点号
- 返回值：应用层接收缓冲区未读取的数据长度

20.3.1.11.uint32_t CDC_DataIn(uint8_t ep, uint8_t *buf, uint32_t len)

- 描述：将数据写入应用层发送缓冲区
- 参数：
  - ep：实际物理端点映射的虚拟端点号
  - buf：将要发送的数据的数组
  - len：写入的数据长度
- 返回值：实际写入的数据长度

20.3.1.12.uint32_t CDC_DataOut(uint8_t ep, uint8_t *buf, uint32_t len)

- 描述：将数据从应用层接收缓冲区取出到指定数组
- 参数：
  - ep：实际物理端点映射的虚拟端点号
  - buf：将要接收的数据的数组
  - len：读取的数据长度
- 返回值：实际读取的数据长度

20.3.1.13.static uint8_t usbd_cdc_SOF(void *pdev)

- 描述：USB CDC 类处理 SOF 包

- 参数：
  - pdev：USB 控制器核心结构体
- 返回值：USB 设备状态

20.3.2. USB HID

20.3.2.1. static uint8_t USBD_HID_Init(void *pdev,uint8_t cfgidx)

- 描述：USB HID 类初始化
- 参数：
  - pdev：USB 控制器核心结构体
  - cfgidx：配置索引
- 返回值：USB 设备状态

20.3.2.2. static uint8_t USBD_HID_DeInit(void *pdev,uint8_t cfgidx)

- 描述：USB HID 类反初始化
- 参数：
  - pdev：USB 控制器核心结构体
  - cfgidx：配置索引
- 返回值：USB 设备状态

20.3.2.3. static uint8_t USBD_HID_Setup(void *pdev,USB_SETUP_REQ *req)

- 描述：USB HID 类接口请求处理
- 参数：
  - pdev：USB 控制器核心结构体

- req：USB 接收到的请求

- 返回值：USB 设备状态

20.3.2.4. static uint8_t *USBD_HID_GetCfgDesc(uint8_t speed, uint16_t *length)

- 描述：获取 USB HID 配置描述符

- 参数：

  - speed：USB 的速度模式

  - length：USB CDC 配置描述符长度

- 返回值：USB HID 配置描述符

20.3.2.5. static uint8_t HID_GetEpIndex(uint8_t epNum)

- 描述：通过虚拟端点号求得物理端点号

- 参数：

  - epNum：实际物理端点映射的虚拟端点号

- 返回值：实际物理端点号

20.3.2.6. static uint8_t usbd_HID_DataIn(void *pdev, uint8_t epnum)

- 描述：发送 IN 包，将应用层缓冲区数据写入底层 DMA

- 参数：

  - pdev：USB 控制器核心结构体

  - epnum：实际物理端点号

- 返回值：USB 设备状态

20.3.2.7. static uint8_t usbd_HID_DataOut(void *pdev, uint8_t epnum)

- 描述：接收 OUT 包，该函数将底层 DMA 中数据写入应用层缓冲区数据。当应用层的缓冲区剩余空间小于两倍包长时将设备的端点的接收阻塞，当应用层缓冲区数据被读取时触发数据继续接收并解除阻塞。通过阻塞来防止应用层缓冲区的数据未读取就被覆盖。
- 参数：
  - pdev：USB 控制器核心结构体
  - epnum：实际物理端点号
- 返回值：USB 设备状态

20.3.2.8. uint32_t HID_GetInDataLen(uint8_t ep_num)

- 描述：获取应用层发送缓冲区还未发送的数据长度
- 参数：
  - ep_num：实际物理端点映射的虚拟端点号
- 返回值：应用层发送缓冲区内还未发送的数据长度

20.3.2.9. uint32_t HID_GetOutDataLen(uint8_t ep_num)

- 描述：获取应用层接收缓冲区未读取的数据长度
- 参数：
  - ep_num：实际物理端点映射的虚拟端点号
- 返回值：应用层接收缓冲区未读取的数据长度

20.3.2.10.uint32_t HID_DataIn(uint8_t ep, uint8_t *buf, uint32_t len)

- 描述：将数据写入应用层发送缓冲区
- 参数：
  - ep：实际物理端点映射的虚拟端点号

- buf：将要发送的数据的数组
- len：写入的数据长度

- 返回值：实际写入的数据长度

20.3.2.11.uint32_t HID_DataOut(uint8_t ep, uint8_t *buf, uint32_t len)

- 描述：将数据从应用层接收缓冲区取出到指定数组
- 参数：
  - ep：实际物理端点映射的虚拟端点号
  - buf：将要接收的数据的数组
  - len：读取的数据长度
- 返回值：实际读取的长度

20.3.3. USB printer

20.3.3.1. static uint8_t usbd_printer_Init(void *pdev,uint8_t cfgidx)

- 描述：USB printer 类初始化
- 参数：
  - pdev：USB 控制器核心结构体
  - cfgidx：配置索引
- 返回值：USB 设备状态

20.3.3.2. static uint8_t usbd_printer_DeInit(void *pdev,uint8_t cfgidx)

- 描述：USB printer 类反初始化
- 参数：
  - pdev：USB 控制器核心结构体

- cfgidx：配置索引

- 返回值：USB 设备状态

20.3.3.3. static uint8_t usbd_printer_Setup(void *pdev, USB_SETUP_REQ *req)

- 描述：USB printer 类处理接口请求
- 参数：
  - pdev：USB 控制器核心结构体
  - req：USB 接收到的请求
- 返回值：USB 设备状态

20.3.3.4. static uint8_t usbd_printer_EP0_RxReady(void *pdev)

- 描述：USB printer 类接收到类请求后的回复函数
- 参数：
  - pdev：USB 控制器核心结构体
- 返回值：USB 设备状态

20.3.3.5. static uint8_t *USBD_cdc_GetCfgDesc(uint8_t speed, uint16_t *length)

- 描述：获取 USB printer 类配置描述符
- 参数：
  - speed：USB 的速度模式
  - length：USB CDC 配置描述符长度
- 返回值：USB printer 类配置描述符

20.3.3.6. static uint8_t PRINTER_GetEpIndex(uint8_t epNum)

- 描述：通过虚拟端点号获取物理端点号
- 参数：
  - epNum：实际物理端点映射的虚拟端点号
- 返回值：实际物理端点号

20.3.3.7. static uint8_t usbd_printer_DataIn(void *pdev, uint8_t epnum)

- 描述：发送 IN 包，该函数将应用层发送缓冲区写入底层 DMA
- 参数：
  - pdev：USB 控制器核心结构体
  - epnum：实际物理端点号
- 返回值：USB 设备状态

20.3.3.8. static uint8_t usbd_printer_DataOut(void *pdev, uint8_t epnum)

- 描述：接收 OUT 包，该函数将底层 DMA 中数据写入应用层缓冲区数据。当应用层的缓冲区剩余空间小于两倍包长时将设备的端点的接收阻塞，当应用层缓冲区数据被读取时触发数据继续接收并解除阻塞。通过阻塞来防止应用层缓冲区的数据未读取就被覆盖。
- 参数：
  - pdev：USB 控制器核心结构体
  - epnum：实际物理端点号
- 返回值：USB 设备状态

20.3.3.9. uint32_t PRINTER_GetInDataLen(uint8_t ep_num)

- 描述：获取应用层发送缓冲区还未发送的数据长度

- 参数：
  - ep_num：实际物理端点映射的虚拟端点号
- 返回值：应用层发送缓冲区内还未发送的数据长度

20.3.3.10.uint32_t PRINTER_GetOutDataLen(uint8_t ep_num)

- 描述：获取应用层接收缓冲区未读取的数据长度
- 参数：
  - ep_num：实际物理端点映射的虚拟端点号
- 返回值：应用层接收缓冲区内未读取的数据长度

20.3.3.11.uint32_t PRINTER_DataIn(uint8_t ep, uint8_t *buf, uint32_t len)

- 描述：将数据写入应用层发送缓冲区
- 参数：
  - ep：实际物理端点映射的虚拟端点号
  - buf：将要发送的数据的数组
  - len：写入的数据长度
- 返回值：实际写入的数据长度

20.3.3.12.uint32_t PRINTER_DataOut(uint8_t ep, uint8_t *buf, uint32_t len)

- 描述：将数据从应用层接收缓冲区取出到指定数组
- 参数：
  - ep：实际物理端点映射的虚拟端点号
  - buf：将要接收的数据的数组

- len：读取的数据长度

- 返回值：实际读取的长度

20.3.3.13.static uint8_t usbd_printer_SOF(void *pdev)

- 描述：USB printer 类处理 SOF 包

- 参数：

  - pdev：USB 控制器核心结构体

- 返回值：USB 设备状态

20.4. demo 说明

20.4.1. USB CDC

- 演示 USB CDC 类设备数据回环，关键代码说明如下：

USB 接收和各个端点的发送缓冲区

```
volatile uint8_t usb_rxbuf[Usb_Rx_Bufsize] = {0};
volatile uint8_t usb_txbuf0[Usb_Endpoint0tx_Bufsize] = {0};
volatile uint8_t usb_txbuf1[Usb_Endpoint1tx_Bufsize] = {0};
volatile uint8_t usb_txbuf2[Usb_Endpoint2tx_Bufsize] = {0};
volatile uint8_t usb_txbuf3[Usb_Endpoint3tx_Bufsize] = {0};
```

先初始化时钟和串口打印的 UART 端口

```
SYSCTRL_ChipInit();
printport_init();
```

对 USB 初始化

```
void USB_Init()
{
    SYSCTRL_ClkEn(CLK_USB, ENABLE);
    CORE_USB->host_md.bit.dp_pu1p5 = 1;
    CORE_USB->ctrl.reg = 0;
    CORE_USB->ctrl.reg = 0x3c;
    CORE_USB->usb_rx_saddr.reg = (uint32_t)(usb_rxbuf);
```

```
    CORE_USB->usb_rx_buflen.reg = sizeof(usb_rxbuf);
    CORE_USB->usb_tx_saddr0.reg = (uint32_t)(usb_txbuf0);
    CORE_USB->usb_tx_saddr1.reg = (uint32_t)(usb_txbuf1);
    CORE_USB->usb_tx_saddr2.reg = (uint32_t)(usb_txbuf2);
    CORE_USB->usb_tx_saddr3.reg = (uint32_t)(usb_txbuf3);
    CORE_USB->usb_rxptr.reg = (uint32_t)(usb_rxbuf);
    CORE_USB->host_md.bit.dma_en = 1;
    CORE_USB->usb_irq_mask.bit.mask = 0xfd7;
    CORE_USB->usb_epmap.bit.ep1 = VIRTUAL_OUT_EP1;
    CORE_USB->usb_epmap.bit.ep2 = VIRTUAL_OUT_EP2;
    CORE_USB->usb_epmap.bit.ep3 = VIRTUAL_OUT_EP3;
    memset(&USB_OTG_dev, 0, sizeof(USB_OTG_dev));
    USBD_Init(&USB_OTG_dev, USB_OTG_FS_CORE_ID, &USR_desc, &USBD_CDC_cb, &USRD_cb);
}
```

配置 D+上拉 1.5 千欧，配置 USB 为全速模式并且使能 USB 模块；设置 USB 的接收缓冲区和 USB 的端点发送缓冲区；使能 DMA 功能，除复位中断，接收缓冲区就绪中断还有端点发送完成中断以外的中断全部屏蔽

使能 USB 中断

```
PLIC_SetPriority(USB_IRQn, 3);
PLIC_EnableIRQ(USB_IRQn);
```

不断接收发送数据

```
packetLen = CDC_GetOutDataLen(CDC_OUT_EP);
if (packetLen > 0)
{
    packetLen = packetLen>CDC_DATA_MAX_PACKET_SIZE?
CDC_DATA_MAX_PACKET_SIZE:packetLen;
    CDC_DataOut(CDC_OUT_EP, Rxbuf, packetLen);
    TotalRcvLen += packetLen;
    yc_log("packetLen=%d TotalRcvLen=%d", packetLen, TotalRcvLen);
    CDC_DataIn(CDC_IN_EP,Rxbuf,packetLen);
}
```

获取应用层缓冲区是否为空，不为空则接收数据，并把接收到的数据发送回去，并统计每包的数据长度和接收的数据总长度

在 USB 中断中的工作

```
uint32_t USBD_OTG_ISR_Handler(USB_OTG_CORE_HANDLE *pdev)
```

在中断中调用该函数，当触发复位中断时将设备地址设为 0；当收到不同 USB 请求命令时回应不同的数据；数据过长时，发送完一包数据后进入中断，会将数据分次发送。

20.4.2. USB HID

- 演示 USB HID 类设备数据回环和键鼠功能，关键代码说明如下：

USB 接收和各个端点的发送缓冲区

```
volatile uint8_t usb_rxbuf[Usb_Rx_Bufsize] = {0};
volatile uint8_t usb_txbuf0[Usb_Endpoint0tx_Bufsize] = {0};
volatile uint8_t usb_txbuf1[Usb_Endpoint1tx_Bufsize] = {0};
volatile uint8_t usb_txbuf2[Usb_Endpoint2tx_Bufsize] = {0};
volatile uint8_t usb_txbuf3[Usb_Endpoint3tx_Bufsize] = {0};
```

先初始化时钟和串口打印的 UART 端口

```
SYSCTRL_ChipInit();
printport_init();
```

对 USB 初始化

```
void USB_Init()
{
    SYSCTRL_ClkEn(CLK_USB, ENABLE);
    CORE_USB->host_md.bit.dp_pu1p5 = 1;
    CORE_USB->ctrl.reg = 0;
    CORE_USB->ctrl.reg = 0x3c;
    CORE_USB->usb_rx_saddr.reg = (uint32_t)(usb_rxbuf);
    CORE_USB->usb_rx_buflen.reg = sizeof(usb_rxbuf);
    CORE_USB->usb_tx_saddr0.reg = (uint32_t)(usb_txbuf0);
    CORE_USB->usb_tx_saddr1.reg = (uint32_t)(usb_txbuf1);
    CORE_USB->usb_tx_saddr2.reg = (uint32_t)(usb_txbuf2);
    CORE_USB->usb_tx_saddr3.reg = (uint32_t)(usb_txbuf3);
    CORE_USB->usb_rxptr.reg = (uint32_t)(usb_rxbuf);
    CORE_USB->host_md.bit.dma_en = 1;
    CORE_USB->usb_irq_mask.bit.mask = 0xfd7;
    CORE_USB->usb_epmap.bit.ep1 = VIRTUAL_OUT_EP1;
    CORE_USB->usb_epmap.bit.ep2 = 2;
    CORE_USB->usb_epmap.bit.ep3 = 3;
```

```
    memset(&USB_OTG_dev, 0, sizeof(USB_OTG_dev));
    USBD_Init(&USB_OTG_dev, USB_OTG_FS_CORE_ID, &USR_desc, &USBD_HID_cb, &USRD_cb);
}
```

配置 D+上拉 1.5 千欧，配置 USB 为全速模式并且使能 USB 模块；设置 USB 的接收缓冲区和 USB 的端点发送缓冲区；使能 DMA 功能，除复位中断，接收缓冲区就绪中断还有端点发送完成中断以外的中断全部屏蔽

使能 USB 中断

```
PLIC_SetPriority(USB_IRQn, 3);
PLIC_EnableIRQ(USB_IRQn);
```

不断发送接收数据

```
if (HID_GetOutDataLen(HID_OUT_EP) >= HID_EPOUT_SIZE)
{
    HID_DataOut(HID_OUT_EP, hid_data, sizeof(hid_data));
    hid_data[0] = HID_REPORT_ID;
    HID_DataIn(HID_IN_EP, hid_data, HID_EPIN_SIZE);
}
```

获取应用层缓冲区是否为空，不为空则接收数据，并把接收到的数据发送回去并在包头加上报告 ID

在 USB 中断中的工作

```
uint32_t USBD_OTG_ISR_Handler(USB_OTG_CORE_HANDLE *pdev)
```

在中断中调用该函数，当触发复位中断时将设备地址设为 0；当收到不同 USB 请求命令时回应不同的数据；数据过长时，发送完一包数据后进入中断，会将数据分次发送。

20.4.3. 键鼠 demo

- 演示键盘切换大小写，鼠标移动的数据，关键代码说明如下：

```
uint8_t keyboard_capslock[8] = {0x00, 0x00, 0x39, 0x00, 0x00, 0x00, 0x00, 0x00};
uint8_t keyboard_uplift[8]   = {0};
uint8_t mouse_leftshift[8]   = {0x01, 0x00, 0xfa, 0xff, 0x00, 0x00, 0x00, 0x00};
```

```
uint8_t mouse_rightshift[8]    = {0x01, 0x00, 0x06, 0x00, 0x00, 0x00, 0x00, 0x00};
uint8_t mouse_upshift[8]         = {0x01, 0x00, 0x00, 0x00, 0xfa, 0xff, 0x00, 0x00};
uint8_t mouse_downshift[8]     = {0x01, 0x00, 0x00, 0x00, 0x06, 0x00, 0x00, 0x00};
```

初始化流程基本与仅用于数据传输的 HID demo 一致

串口接收到 k 的大小写字符后就切换键盘的大小写，接收到 m 的大小写字符后就使鼠标指针在屏幕中移动矩形轨迹

```
switch (UART_ReceiveData(PRINT_PORT))
{
    case 'k':
    case 'K':
        HID_DataIn(KEYBOARD_IN_EP, keyboard_capslock, KEYBOARD_EPIN_SIZE);
        HID_DataIn(KEYBOARD_IN_EP, keyboard_uplift, KEYBOARD_EPIN_SIZE);
        yc_log("casplock");
          break;
    case 'm':
    case 'M':
        for (int i = 0; i < 50; i++)
        {
            HID_DataIn(MOUSE_IN_EP, mouse_upshift, MOUSE_EPIN_SIZE);
        }
        for (int i = 0; i < 50; i++)
        {
            HID_DataIn(MOUSE_IN_EP, mouse_rightshift, MOUSE_EPIN_SIZE);
        }
        for (int i = 0; i < 50; i++)
        {
            HID_DataIn(MOUSE_IN_EP, mouse_downshift, MOUSE_EPIN_SIZE);
        }
        for (int i = 0; i < 50; i++)
        {
            HID_DataIn(MOUSE_IN_EP, mouse_leftshift, MOUSE_EPIN_SIZE);
        }
        yc_log("mouse test");
        break;
    default:
        break;
}
```

检测 HID，键盘，鼠标接收到的数据并打印

```
if (HID_GetOutDataLen(HID_OUT_EP) >= HID_EPOUT_SIZE)
```

```
{
    yc_usb_log("hid receive %d bytes data", HID_EPOUT_SIZE);
    HID_DataOut(HID_OUT_EP, hid_receive_data, HID_EPOUT_SIZE);
     HID_DataIn(HID_IN_EP, hid_receive_data, HID_EPIN_SIZE);
}

int hid_len = HID_GetOutDataLen(KEYBOARD_OUT_EP);
if (hid_len > 0)
{
    yc_log("keyboard receive %d bytes data", hid_len);
    HID_DataOut(KEYBOARD_OUT_EP, keyboard_receive_data, hid_len);
    yc_log_buf("keyboard_receive_data", keyboard_receive_data, hid_len);
}

hid_len = HID_GetOutDataLen(MOUSE_OUT_EP);
if (hid_len > 0)
{
    yc_log("mouse receive %d bytes data", hid_len);
    HID_DataOut(MOUSE_OUT_EP, mouse_receive_data, hid_len);
    yc_log_buf("mouse_receive_data", mouse_receive_data, hid_len);
}
```

在 USB 中断中的工作

```
uint32_t USBD_OTG_ISR_Handler(USB_OTG_CORE_HANDLE *pdev)
```

在中断中调用该函数，当触发复位中断时将设备地址设为 0；当收到不同 USB 请求命令时回应不同的数据；数据过长时，发送完一包数据后进入中断，会将数据分次发送。

20.4.4. USB printer

- 演示 USB 接收和各个端点的发送缓冲区，关键代码说明如下：

```
volatile uint8_t usb_rxbuf[Usb_Rx_Bufsize] = {0};
volatile uint8_t usb_txbuf0[Usb_Endpoint0tx_Bufsize] = {0};
volatile uint8_t usb_txbuf1[Usb_Endpoint1tx_Bufsize] = {0};
volatile uint8_t usb_txbuf2[Usb_Endpoint2tx_Bufsize] = {0};
volatile uint8_t usb_txbuf3[Usb_Endpoint3tx_Bufsize] = {0};
```

先初始化时钟和串口打印的 UART 端口

```
SYSCTRL_ChipInit();
```

```
printport_init();
```

对 USB 初始化

```
void USB_Init()
{
    SYSCTRL_ClkEn(CLK_USB, ENABLE);
    CORE_USB->host_md.bit.dp_pu1p5 = 1;
    CORE_USB->ctrl.reg = 0;
    CORE_USB->ctrl.reg = 0x3c;
    CORE_USB->usb_rx_saddr.reg = (uint32_t)(usb_rxbuf);
    CORE_USB->usb_rx_buflen.reg = sizeof(usb_rxbuf);
    CORE_USB->usb_tx_saddr0.reg = (uint32_t)(usb_txbuf0);
    CORE_USB->usb_tx_saddr1.reg = (uint32_t)(usb_txbuf1);
    CORE_USB->usb_tx_saddr2.reg = (uint32_t)(usb_txbuf2);
    CORE_USB->usb_tx_saddr3.reg = (uint32_t)(usb_txbuf3);
    CORE_USB->usb_rxptr.reg = (uint32_t)(usb_rxbuf);
    CORE_USB->host_md.bit.dma_en = 1;
    CORE_USB->usb_irq_mask.bit.mask = 0xfd7;
    CORE_USB->usb_epmap.bit.ep1 = VIRTUAL_OUT_EP1;
    CORE_USB->usb_epmap.bit.ep2 = 2;
    CORE_USB->usb_epmap.bit.ep3 = 3;
    memset(&USB_OTG_dev, 0, sizeof(USB_OTG_dev));
    USBD_Init(&USB_OTG_dev, USB_OTG_FS_CORE_ID, &USR_desc, &USBD_HID_cb, &USRD_cb);
}
```

配置 D+上拉 1.5 千欧，配置 USB 为全速模式并且使能 USB 模块；设置 USB 的接收缓冲区和 USB 的端点发送缓冲区；使能 DMA 功能，除复位中断，接收缓冲区就绪中断还有端点发送完成中断以外的中断全部屏蔽

使能 USB 中断

```
PLIC_SetPriority(USB_IRQn, 3);
PLIC_EnableIRQ(USB_IRQn);
```

在循环中

```
packetLen = PRINTER_GetOutDataLen(PRINTER_OUT_EP);
if (packetLen > 0)
{
    packetLen =
(packetLen>=PRINTER_DATA_MAX_PACKET_SIZE)?PRINTER_DATA_MAX_PACKET_SIZE:packetLen;
```

```
    PRINTER_DataOut(PRINTER_OUT_EP, Rxbuf, packetLen);
    totalLen += packetLen;
    yc_log("packetLen:%d rxlen:%d", packetLen, totalLen);
}
```

统计主机发送的数据大小，并输出每包接收了多少数据和总共接收了多少数据

在 USB 中断中的工作

```
uint32_t USBD_OTG_ISR_Handler(USB_OTG_CORE_HANDLE *pdev)
```

在中断中调用该函数，当触发复位中断时将设备地址设为 0；当收到不同 USB 请求命令时回应不同的数据；数据过长时，发送完一包数据后进入中断，会将数据分次发送。

21. OTA

21.1. 宏定义、枚举及结构体说明

21.1.1. OTA 配置结构体

```
typedef struct
{
    bool     SetAppFlag;              //跳 APP 标志
    bool     isBufferFull;            //buf 满
    uint32_t pos;                     //当前帧数偏移
    uint32_t offset;                  //4K 的偏移地址
    uint32_t updateaddr;              //写 flash 地址
    uint32_t address;                 //写 flash 地址偏移地址
    uint32_t Sumlength;
    uint32_t CheckSum;
    uint8_t buffer[OTA_BUFFER_LEN]; //升级 buf 4K
}OTA_CONFIG;
```

21.1.2. OTA 固件大小及 A,B 区大小宏定义(A B 区方式 OTA 升级)

```
#define OTA_OFFSET_ADDR          0x0
#define OTA_BLOCK_A_ADDR         0x01000     /* 4K 对齐   A Block 的大小为 100k */
#define OTA_BLOCK_B_ADDR         0x1a000     /* 100K -->end addr :0x33000*/
#define VENDER_BLOCK_ADDR        0x33000     /* 52k reserved */
```

```
#define OTA_FIRMWARE_LEN        100*1024    /*100K*/
#define FLASH_PAGE_LEN          256
```

21.1.3. OTA 固件大小及 APP 起始地址宏定义(BOOT 方式 OTA 升级)

```
#define OTA_OFFSET_ADDR         0x0
#define OTA_APP_ADDR            0x23000     /* APP 固件起始地址 */
#define OTA_FIRMWARE_LEN        100*1024    /*100K*/
#define FLASH_PAGE_LEN          256
```

21.1.4. OTA 升级模式枚举

```
 typedef enum
{
    USB_MODE     =    0x01,
    BLE_MODE     =    0x02,
}OTA_MODE_Def;
```

21.2. 接口说明

21.2.1. void ota_config_init(void)

- 描述：初始化 OTA 的配置

21.2.2. void RunNewApp(bool appflag)

- 描述：如果固件数据传输完成后并写入 flash 后，将芯片复位并运行新的固件
- 参数：
  - appflag：OTA 流程是否结束，当结束时该位置一并开始运行新固件的流程

21.2.3. void GoToApp(uint8_t set_app_flag,uint8_t entry_ota_flag)

- 描述：如果固件数据传输完成后并写入 flash 后或没有进入 OTA 模式，则从 BOOT 固件区跳转到 APP 固件区；跳转到 APP 前需要将开启过的模块关闭，并关闭对应模块的中断然后再关闭全局中断（该函数主要在 BOOT 方式进行 OTA 升级时使用）
- 参数：
  - set_app_flag：OTA 流程是否结束，当结束时该位置一并开始运行新 APP 固件的流程
  - entry_ota_flag：判断是否进入 OTA 模式，没有进入 OTA 模式则直接跳转到 APP

21.2.4. void Show_Current_Block(void)

- 描述：从 UART 打印目前固件运行在 flash 的 A 区还是 B 区

21.2.5. void OTA_Process(OTA_MODE_Def ota_mode)

- 描述：通过参数判断是通过 BLE 进行 OTA 升级还是通过 USB 进行 OTA 升级，该函数中通过参数判断运行 USB 的 OTA 协议处理流程还是运行 BLE 的 OTA 协议处理流程
  - 参数：
    - ota_mode：OTA_MODE_Def 枚举，详见 21.1.3 OTA 升级模式枚举

21.3. demo 说明

OTA 升级关键代码说明如下

21.3.1 A B 区方式 OTA 升级

此 OTA 方式的演示 demo 为 ota_boot_app，以下是关键代码说明：

芯片串口初始化

```
SYSCTRL_ChipInit();
SYSTICK_SetClk1M();
printport_init();
```

初始 OTA 配置并打印目前的固件运行在 A 区还是 B 区，如果代码运行在 A 区，则将新固件写入 B 区，OTA 升级完成后，运行 B 区的固件；如果代码运行在 B 区，则将新固件写入 A 区，OTA 升级完成后，运行 A 区的固件

```
ota_config_init();
Show_Current_Block();
```

OTA 的主要流程

```
while(1)
    {
        if(UART_GetRxCount(PRINT_PORT) > 0)
        {
            RecvData = UART_ReceiveData(PRINT_PORT);
            yc_log("RecvData = %c.\n", RecvData);

            if(RecvData == 'B' || RecvData == 'b')
            {
                yc_log("Ble Init.\n");
                Ble_ParamInit();
                ota_mode_flag=BLE_MODE;
            }
            else if(RecvData == 'U' || RecvData == 'u')
            {
                yc_log("USB Init.\n");
                USB_Init();
                ota_mode_flag=USB_MODE;
            }
        }
        RunNewApp(ota_config.SetAppFlag);
        OTA_Process(ota_mode_flag);
    }
```

UART 输入“U”或者“u”时，则通过 USB 进行 OTA 升级，先将 USB 初始化，ota_mode_flag 赋值为 USB_MODE；UART 输入“B”或者“b”时，则通过 BLE 进行 OTA 升级，先将 BLE 初始化，ota_mode_flag 赋值为 BLE_MODE；OTA_Process 函数运行对应的 OTA 协议流程。当 BLE 或者 USB 的 OTA 流程完成后会将 ota_config.SetAppFlag 置 1 运行新固件

21.3.2 BOOT 方式 OTA 升级

此 OTA 方式的相关 demo 为 ota_boot 和 ota_app。

ota_boot demo 代码说明：

```
SYSCTRL_ChipInit();
SYSTICK_SetClk1M();
printport_init();
```

初始 OTA 配置并将按键对应的 GPIO 设置为上拉

```
ota_config_init();
GPIO_Config(BLE_OTA_PIN,GPIO_MODE_PULL_UP);
GPIO_Config(USB_OTA_PIN,GPIO_MODE_PULL_UP);
```

从 FLASH 读取 OTA 升级标志位

```
FLASH_Read(FLASH_OTA_FLAG_ADDR,1,&flash_ota_flag);
```

OTA 的主要流程

```
 if (flash_ota_flag==FLASH_OTA_FLAG_USB)
{
    ota_mode_flag=OTA_USB_MODE;
}
else if (flash_ota_flag==FLASH_OTA_FLAG_BLE)
{
    ota_mode_flag=OTA_BLE_MODE;
}
else if (GPIO_ReadDataBit(BLE_OTA_PIN)==0)
{
    delay_ms(50);
    if (GPIO_ReadDataBit(BLE_OTA_PIN)==0)
    {
        ota_mode_flag=OTA_BLE_MODE;
    }
}
else if (GPIO_ReadDataBit(USB_OTA_PIN)==0)
{
    delay_ms(50);
    if (GPIO_ReadDataBit(USB_OTA_PIN)==0)
    {
        ota_mode_flag=OTA_USB_MODE;
```

```
        }
    }
    if(ota_mode_flag == OTA_USB_MODE)
    {
        yc_log("USB Init.\n");
        USB_Init();
    }
    else if(ota_mode_flag == OTA_BLE_MODE)
    {
        yc_log("Ble Init.\n");
        Ble_ParamInit();
    }
    while((ota_mode_flag == OTA_USB_MODE || ota_mode_flag == OTA_BLE_MODE) &&
ota_config.SetAppFlag==0)
    {
        OTA_Process(ota_mode_flag);
    }
    GoToApp(ota_mode_flag);

```

首先检查使用USB或BLE进行OTA升级的标志位是否被置起,若FLASH_OTA_FLAG_USB被置起，则通过 USB 进行 OTA 升级，先将 USB 初始化，ota_mode_flag 赋值为 USB_MODE；若 FLASH_OTA_FLAG_BLE 被置起，则通过 BLE 进行 OTA 升级，先将 BLE 初始化，ota_mode_flag 赋值为 BLE_MODE；OTA_Process 函数运行对应的 OTA 协议流程，当 USB 或者 BLE 的 OTA 流程完成后会将 ota_config.SetAppFlag 置一然后跳转到 APP 固件。

如果没有标志位被置起，则检测用于触发 OTA 升级（USB 或 BLE 方式）的按键状态，按键必须在芯片复位之前被按下并保持至 OTA 升级流程启动。BLE_OTA_PIN 被按下，先将 BLE 初始化，ota_mode_flag 赋值为 BLE_MODE；USB_OTA_PIN 被按下，先将 USB 初始化，ota_mode_flag 赋值为 USB_MODE；OTA_Process 函数运行对应的 OTA 协议流程，当 BLE 或者 USB 的 OTA 流程完成后会将 ota_config.SetAppFlag 置一然后跳转到 APP 固件。

若没有标志位被置起且没有按键被按下，则直接跳转到 APP 固件。

ota_app demo 代码说明：

芯片串口初始化

```
SYSCTRL_ChipInit();

printport_init();
```

OTA 的主要流程

```
    while(1)
    {
        if(UART_GetRxCount(PRINT_PORT) > 0)
        {
            RecvData = UART_ReceiveData(PRINT_PORT);
            yc_log("RecvData = %c.\n", RecvData);

            if(RecvData == 'B' || RecvData == 'b')
            {
                yc_log("Ble OTA.\n");
                flash_ota_flag=FLASH_OTA_FLAG_BLE;
                FLASH_Write(FLASH_OTA_FLAG_ADDR,1,&flash_ota_flag);
                SYSCTRL_SoftReset();
            }
            else if(RecvData == 'U' || RecvData == 'u')
            {
                yc_log("USB OTA.\n");
                flash_ota_flag=FLASH_OTA_FLAG_USB;
                FLASH_Write(FLASH_OTA_FLAG_ADDR,1,&flash_ota_flag);
                SYSCTRL_SoftReset();
            }
            else if (RecvData == 'S' || RecvData == 's')
          {
             yc_log("lpm sleep");
             LPM_Sleep(RTC_WAKEUP_TIME,SLEEP_MODE_DEEP,0x1f,WAKEUP_FROM_SLEEP,LPM
_prepareActionBeforeSleep,LPM_postActionWakeup);
             yc_log("wakeup");
             wakeup_FlagAnalysis();
          }

        }
    }
```

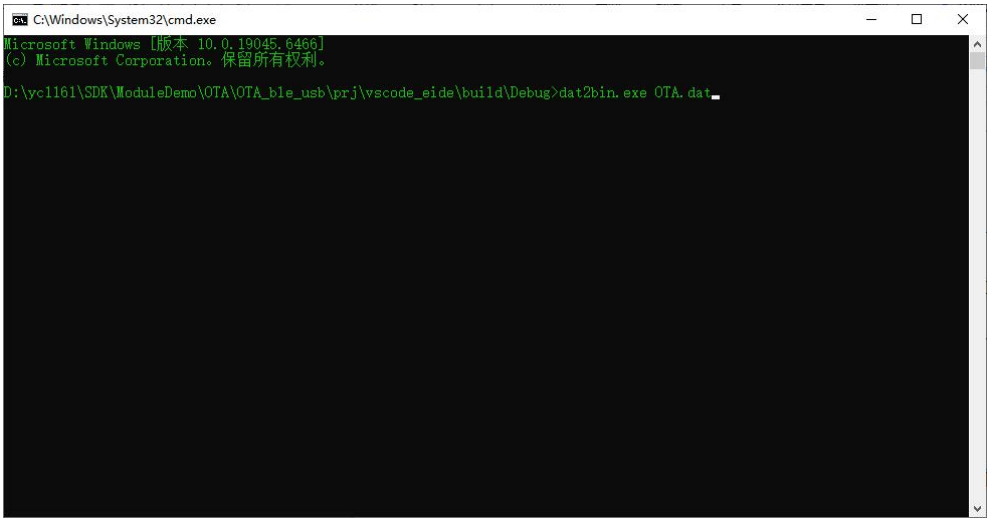
UART 输入“B”或者“b”时，通过 BLE 进行 OTA 升级，将 flash_ota_flag 赋值为 FLASH_OTA_FLAG_BLE 并写入 FLASH；UART 输入“U”或者“u”时，通过 USB 进行 OTA

升级，将 flash_ota_flag 赋值为 FLASH_OTA_FLAG_USB 并写入 FLASH；然后程序会复位芯片，接着进入 ota_boot 流程；UART 输入”S”或者”s”进入 lpm sleep。

21.4. OTA 升级流程和注意事项

OTA 固件注意事项

OTA 升级的 bin 文件与编译生成的 bin 文件不同，需要手动制作，在工程路径\prj\vscode_eide\build\Debug 下中有以后缀为.dat 的文件，需要通过该文件转为.bin 文件，这种转换出来的 bin 文件才可以通过 OTA 正常使用，yc1161\SDK\util\tool 路径下有 dat2bin.exe 程序和 dat2bin.py 脚本，通过程序或脚本将.dat 文件转换为.bin 文件，将该程序放至.dat 文件的同目录下并在该路径下打开 cmd，如图将.dat 文件转换为.bin 文件



OTA A B 区升级注意事项

A B 区固件的起始地址要 4K 对齐，方便 flash 擦除读写，目前 demo 中的 flash 操作都是扇区擦除扇区写入

OTA BOOT 升级注意事项

如果通过 BOOT 方式进行 OTA 升级，则需要注意以下事项：

flash 中 APP 区起始地址需要满足以下条件：

(flash_addr)%4096==0 并且(flash_addr-4096)%34==0

OTA 升级时下载的固件为 flash_app 工程中的文件，该文件也需要通过程序将 dat 文件转换为 bin 文件才可以正常下载使用

flash_app 工程使用的链接文件与正常 demo 不同，如

yc1161\SDK\ModuleDemo\OTA\ota_app\user 路径下的 link_script.ld 文件

link_script.ld 链接文件中的 flash 起始地址需要修改，如下图

```
MEMORY
{
    flash          (rxai!w) : ORIGIN = 0x01020000, LENGTH = 512K
    retention_ram  (wxa!ri) : ORIGIN = 0x10050, LENGTH = 8K-0x50
    ram            (wxa!ri) : ORIGIN = 0x12000, LENGTH = 57K
    ramcode        (rxai!w) : ORIGIN = 0x20400, LENGTH = 15k
}
```

该起始地址需要将 flash 物理地址加上偏移后转换为 CPU(XIP 地址)，转换公式如下：

CPU(XIP)_addr=(flash_addr+24)/34*32|0x1000000

而在 boot 工程下的跳转地址为链接文件中的 flash 起始地址加上 0x80，如下图

```
01020000 <flash_start-0x80>:
   ...

01020080 <flash_start>:
/**
 * Reset Handler
 */
    .globl flash_start
flash_start:
    j chip_init
1020080:    0300006f            j    10200b0 <chip_init>
    nop
1020084:    00000013            nop
```

跳转到对应 app 固件地址（即转换后的 flash 起始地址）的代码

```
((void(*)(void))(((PARAM_MCU_ADDR(OTA_APP_ADDR+24))+0x80)|0x1000000))();
```

21.4.1. OTA 通讯协议

OTA USB 通讯协议如下

上位机发送命令说明						
	CMD_OTA_START	0xba	0xc0	2BYTES PACKET NUM	3BYTES 总数据长度	
	CMD_OTA_DATA	0xba	0xc1	2BYTES PACKET ID	2BYTES LENGTH	OTA DATA（可变）
					数据CheckSum换成占3字节16进制，小端格式填入	
	CMD_OTA_END	0xba	0xc2	2BYTES PACKET ID	3BYTES CHECKSUM	3BYTES 总数据长度，除去跳过字节数
	CMD_OTA_RESET	0xba	0xff			
下位机发送命令说明						
	CMD_OTA_START_H	0xab	0xc0			
	CMD_OTA_DATA_H	0xab	0xc1	2BYTES PACKET ID		
	CMD_OTA_END_H	0xab	0xc2	2BYTES PACKET ID	55(OTA_SUCCESS); 其它(OTA_FAIL)	例如：ABC2 7D01 55 成功
* 2个、3个字节的内容都是低字节在前，高字节在后						

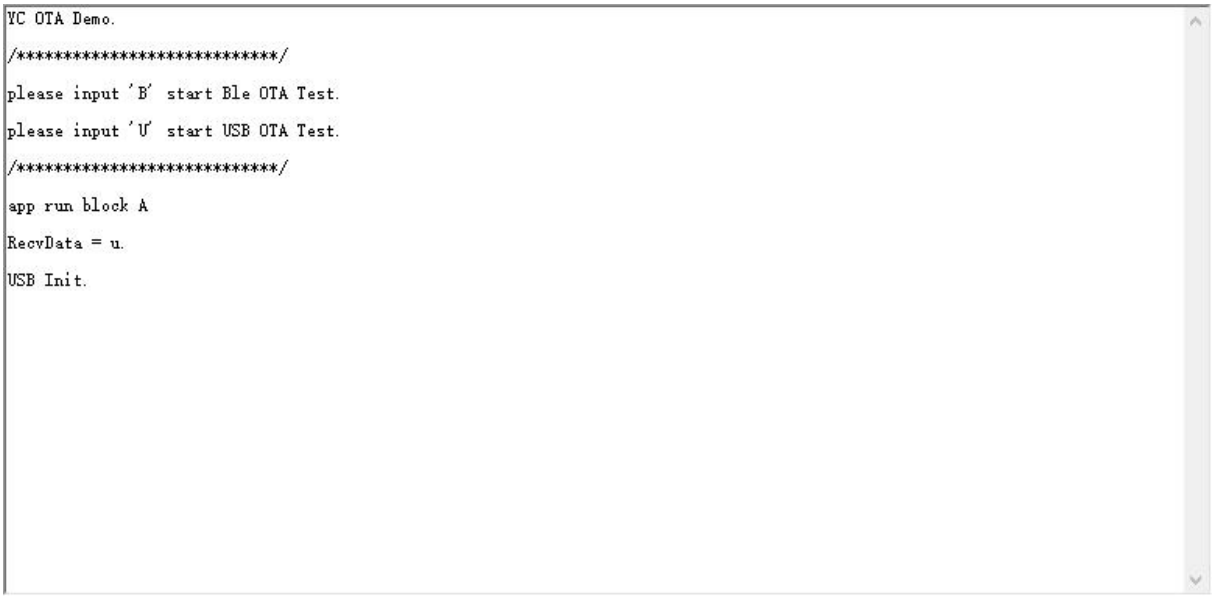
APP		芯片
重复发送CMD_OTA_START，直到收到CMD_OTA_START_H	------> <-------	收到CMD_OTA_START后发送CMD_OTA_START_H
收到包判断包序号与上一包发送的包序号相同则重复上一包序号ot继续发送下一个序号ota数据；否则发送CMD_OTA_RESET	------> <-------	收到CMD_OTA_DATA数据，发送CMD_OTA_DATA_H
	.	
	.	
	.	
	.	
	.	
ota数据发送完毕，发送CMD_OTA_EXIT	-------->	收到CMD_OTA_EXIT，计算checksum，与收到的checksum比较；checksum相同则回发OTA_SUCCESS包；否则回发OTA_FAIL包；
收到CMD_OTA_EXIT_H，并在界面中显示是否成功；否则显示失败	<--------	

OTA BLE 通讯协议

OTA BLE 的通讯协议参考路径 yc1161\DOC 下的 OTA BLE 通讯协议说明.pdf 文件

21.4.2. USB 方式 OTA 升级

UART 输入 u 或者 U 后进行 USB 初始化

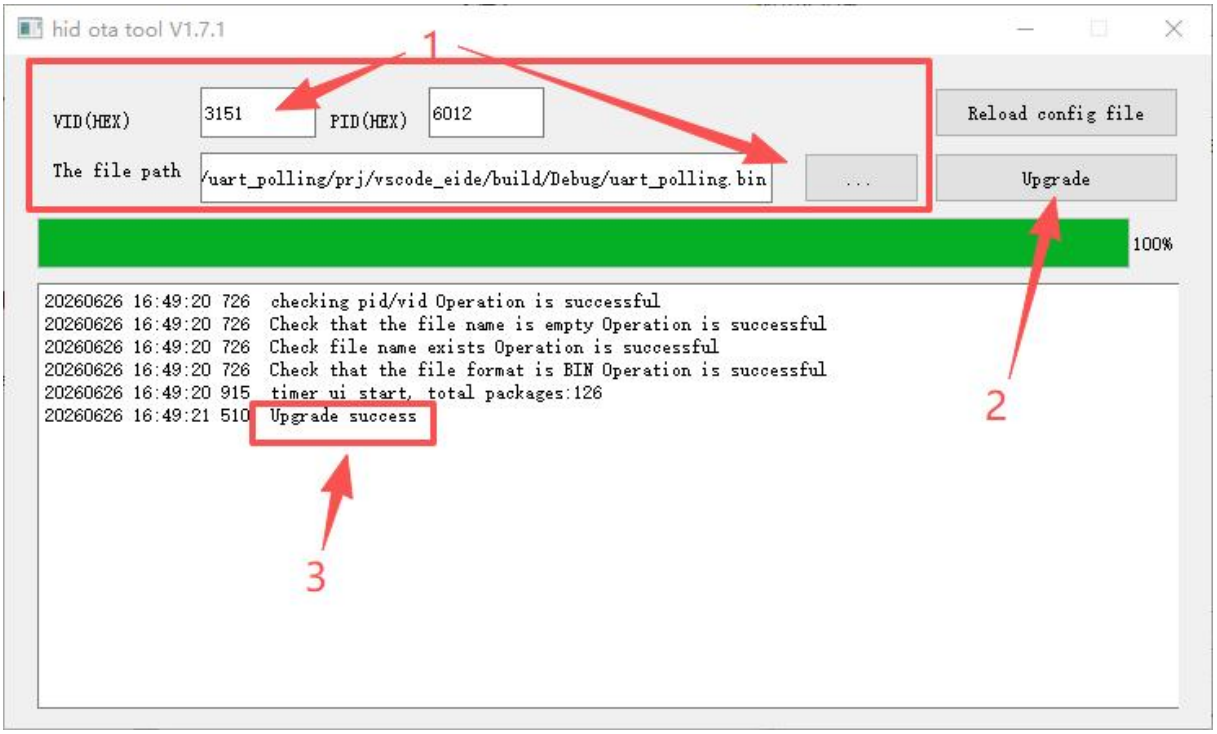


然后打开 yc1161\SDK\util\tool 路径下的 hid_ota_tool.exe 程序

步骤一：填写 VID 和 PID

步骤二：点击 Upgrade

步骤三：查看到 Upgrade success 即为成功



21.4.3. BLE 方式 OTA 升级

UART 输入 b 或者 B 后进行 BLE 初始化

通过浏览器扫描二维码下载 OTA APP



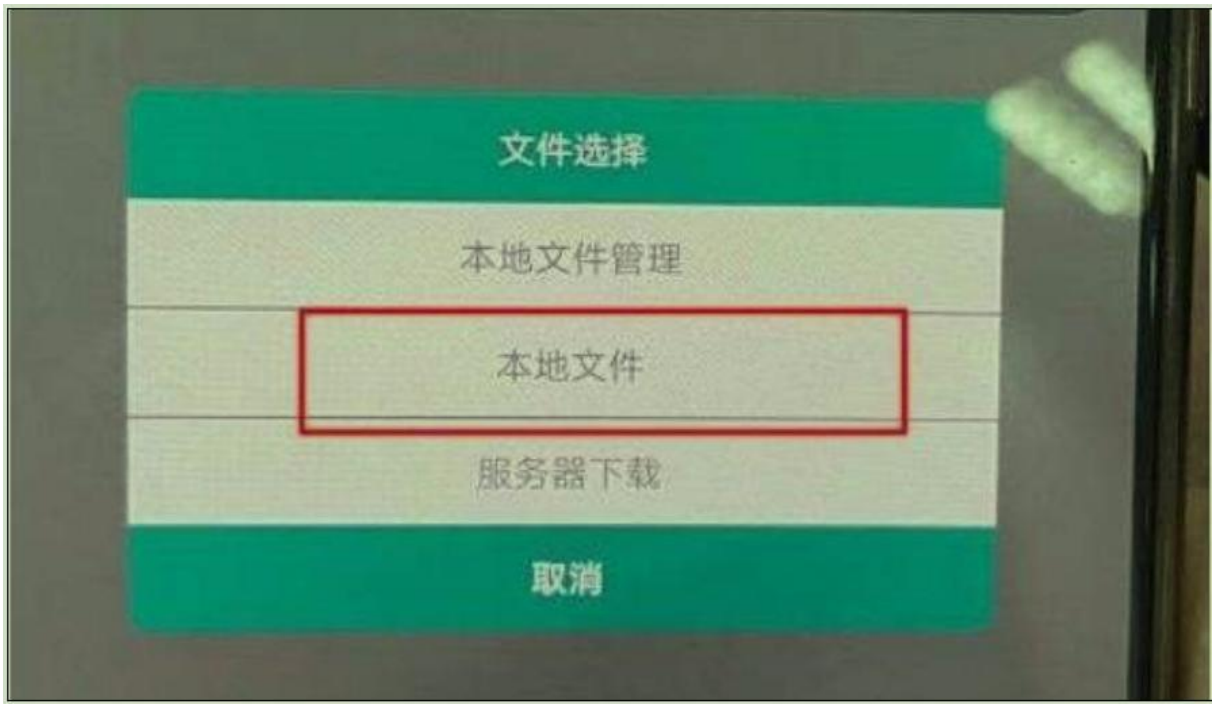
将生产的.bin 文件放入如下图的文件路径下



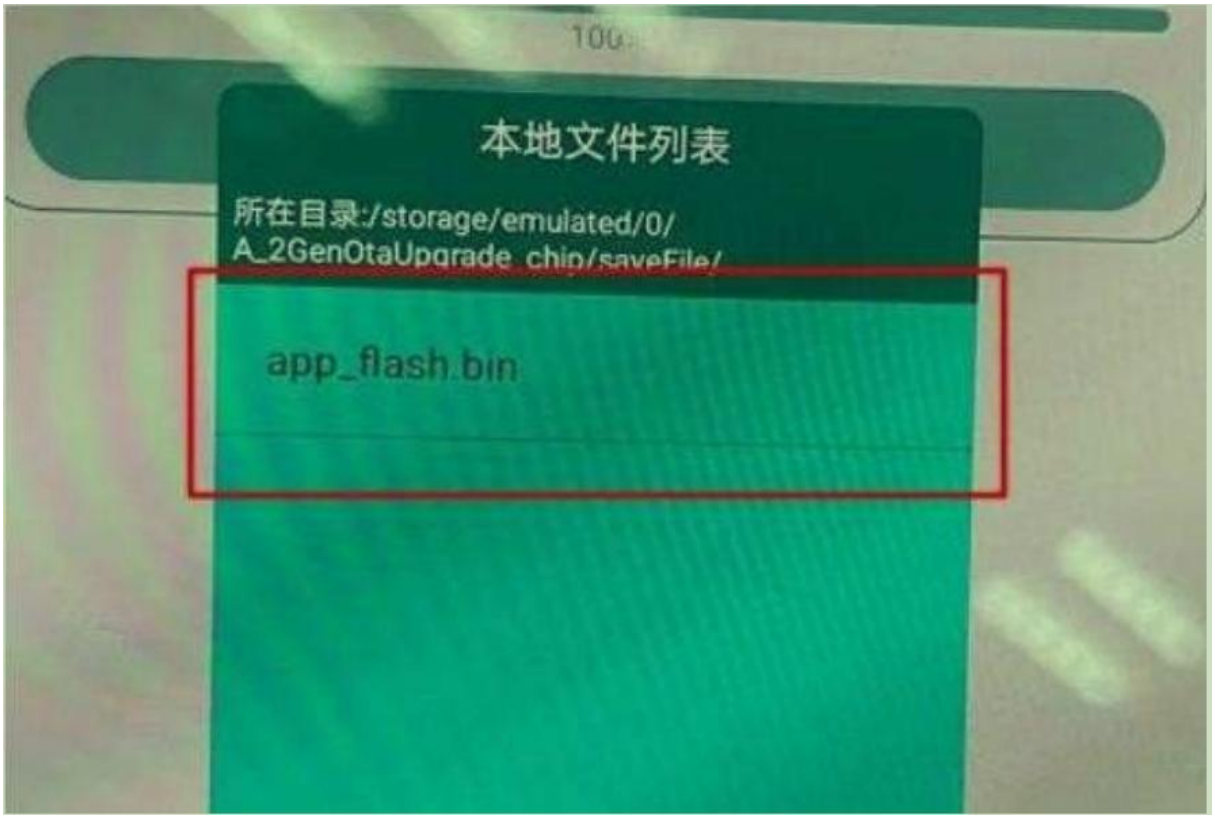
打开 APP，点击选择/下载升级固件



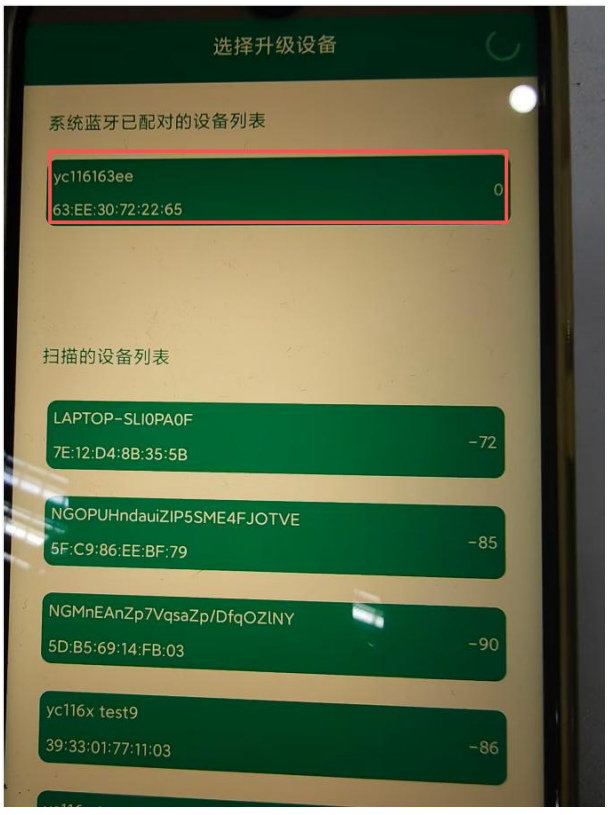
选择本地文件(只有选择该选项才能正常下载)



选择放入的升级固件



点击选择设备&开始升级，找到要升级的芯片蓝牙设备名，点击后直接开始升级



升级成功



升级成功后会跳转到新的固件

22. BLE

22.1. 宏定义、枚举及结构体说明

22.1.1. BLE 广播数据最大长度

```
35 #define ADV_MAX_LEN                    (31)  //BLE 广播数据，最大长度不超过 31 字节
```

22.1.2. RSSI 窗口大小

```
36 #define RSSI_WINDOW_SIZE               (16)  //计算真实 rssi 的窗口大小,取 5-16
```

22.1.3. BLE 地址类型

```
#define NON_RESOLV_ADDR_TYPE       (0x00)
#define RESOLV_ADDR_TYPE           (0x40)
#define RESERVED_ADDR_TYPE         (0x80)
#define STATIC_ADDR_TYPE           (0xc0)
```

22.1.4. NVRAM 相关

```
#define NVRAM_FLASH_ADDR      (0x3F000)      //last 4KB of 2M-bitflash
#define NVRAM_SIZE            (34)//固定值
#define NVRAM_NUM             (5)//用户自定义修改，默认 5 组
#define NVRAM_LEN             (NVRAM_SIZE*NVRAM_NUM)
```

22.1.5. NVRAM 数据结构体

```
typedef struct
{
    uint8_t num;
    uint8_t type;
    union
    {
        uint8_t addr[6];
        uint8_t irk[16];
        uint8_t ediv_rand[16];
    };
    uint8_t key[16];
}NvramItem;
```

- num：序号
- type：数据类型
- union:联合体
  - addr：设备地址，6 字节
  - irk：IRK，16 字节
  - ediv_rand：EDIV/RAND 数据，16 字节。
- key：密钥

22.1.6. BLE 广播类型枚举

```
typedef enum
{
    ADV_IND = 0,
    ADV_DIRECT_IND,
    ADV_NONCONN_IND,
    ADV_SCAN_IND = 6,
    ADV_EXT_IND,
    AUX_ADV_IND = 7,
    AUX_SYNC_IND = 7,
    AUX_CHAIN_IND = 7,
    AUX_SYNC_SUBEVENT_IND = 7,
    ADV_DECISION_IND = 9,
}BLE_AdvTypeEnum;
```

22.1.7. BLE 地址类型枚举

```
typedef enum
{
    ADDR_PUBLIC = 0,
    ADDR_RANDOM = 1,
}BLE_AdvAddrType;
```

22.1.8. BLE 配对模式枚举

```
typedef enum
{
    PAIR_NONE               = 0x00,
    PAIR_JUST_WORK          = 0x01,
    PAIR_PASSKEY            = 0x02,
    PAIR_PASSKEY_INPUT = 0x03,
    PAIR_SC_JUST_WORK    = 0x81,
    PAIR_SC_PASSKEY       = 0x83,
    PAIR_SC_NUMERIC       = 0x82
}BLE_PairModeEnum
```

22.2. ble 接口汇总

函数类型	头文件名	函数名	说明
ble 相关接口	yc_bt	BT_GetLibVersion	获取蓝牙库版本
		BT_BleSetMac	设置 BLE 地址
		BT_BleSetName	设置 BLE 名称
		BT_BleSetAdvParams	设置 Ble 广播数据
		BT_BleSetScanData	设置 BLE 扫描包数据
		BT_BleSetAdvInterval	设置 ble 广播间隔
		BT_SetVisibility	BLE 可发现(开广播)
		BT_SetScanEnable	BLE 开扫描
		BT_SetBleConnAddr	BLE 主连接到指定地址
		BT_BleConnectParmUpdateReq	BLE 更新连接参数
		BT_BleUpdateMtu	BLE 更新 MTU 数据
		BT_BleDisconnectIND	BLE 断开连接指令
		BT_BleSetMtu	BLE 设置本地 MTU
		BT_BleSlaveSendData	BLE 从机发送数据
		BT_BleMasterSendData	BLE 主机发送数据
		BT_BleSetPairMode	BLE 设置配对模式
		BT_NvramLoad	从 Flash 读 NVRAM 配置
		BT_SetTXPower	设置 TX 功率
		BT_SetLLParam	设置 LL 参数

		BT_NvramFindFromList	从列表中查找 NVRAM
		BT_BleNvramSave	更新 nvram 数据
		BT_NvramUpdateToFlash	更新 Flash 中 Nvram 数据
		BT_GetRssi	获取 Rssi 值
	yc_ipc	IPC_HandleRxPacket	BLE 处理接收数据包
		IPC_ParseEvtCallback	解析事件回调函数，根据事件类型进行处理
	yc_user	U_BleSetMac	设置蓝牙地址,通过 flash id 的 crc16 值生成唯一地址
		U_BleSetDeviceName	设置蓝牙设备名称,通过将设备名称和蓝牙地址拼接起来,防止设备名称重复
		U_BleGenAdvScanData	生成广播数据和扫描响应数据
		U_BleParseScanAdvData	解析 ble 扫描到的对端广播数据，用于 ble 做主时连接使用
		U_smp_passkey_wait_input	待并处理配对时的 Passkey 输入

		U_smp_passkey_wait_input_timeout	passkey 输入超时等待
		U_ATT_GenericAccess_CbRead	ATT 处理读属性回调
		U_ATT_User_ServiceCbWrite	ATT 处理写数据回调
		U_BleServiceParse	解析当前连接设备 ATT 服务
		U_BleMasterParseNotifyData	解析主机端接收到的 Notify/Indicate 数据

22.3. 接口说明

22.4. yc_bt

22.4.1. uint16_t BT_GetLibVersion(void)

- 描述：获取蓝牙库版本
- 参数：none
- 返回值：蓝牙库版本号

22.4.2. void BT_BleSetMac(uint8_t * bleAddr)

- 描述：设置 BLE MAC 地址
- 参数：
  - bleAddr：6 个字节 MAC 地址
- 返回值：none

22.4.3. void BT_BleSetName(uint8_t* bleName, uint8_t len)

- 描述：设置 BLE 名称
- 参数：
  - bleName：蓝牙 4.0 名称，不能包含'\0'
  - len：名称长度小于 16 字节
- 返回值：none

22.4.4. void BT_BleSetAdvParams(BLE_AdvTypeEnum type,BLE_AdvAddrType own_addr_type, BLE_AdvAddrType peer_addr_type, uint8_t *advData, uint8_t len)

- 描述：设置 Ble 广播数据
- 参数：
  - type：广播类型
  - own_addr_type,peer_addr_type：00:public address, 01: random address
  - AdvData：广播数据或者回连地址
  - len：名称长度小于 16 字节
- 返回值：none

22.4.5. void BT_BleSetScanData(uint8_t* scanData, int len)

- 描述：设置 BLE 扫描包数据
- 参数：
  - scanData : 扫描数据指针, len :数据长度小于 32 字节
- 返回值：none

22.4.6. void BT_BleSetAdvInterval(uint32_t interval)

- 描述：设置 ble 广播间隔
- 参数：
  - interval：设置间隔,单位 us
- 返回值：none

22.4.7. void BT_SetVisibility(uint8_t discoverable)

- 描述：BLE 可发现(开广播)
- 参数：
  - discoverable: ENABLE 或 DISABLE
- 返回值：none

22.4.8. void BT_SetScanEnable(uint8_t discoverable,uint32_t scan_interval,uint16_t scan_window)

- 描述：BLE 开扫描，用于 ble 做主接收从机端广播数据
- 参数：
  - discoverable: ENABLE 或 DISABLE
  - scan_interval：扫描间隔,单位 us
  - scan_window：扫描窗口,单位 us
- 返回值：none

22.4.9. void BT_SetBleConnAddr(uint8_t *addr)

- 描述：BLE 连接到指定地址
- 参数：

- addr: 目标地址

- 返回值：none

22.4.10. void BT_BleConnectParmUpdateReq(uint8_t *data,uint8_t len)

- 描述：BLE 更新连接参数
- 参数：
  - data:
    - byte0-byte1：最小连接间隔，最小连接间隔=(byte0+(byte1<<8))*1.25ms
    - Byte2-byte3：最大连接间隔，最小连接间隔=(byte2+(byte3<<8))*1.25ms
    - Byte4-byte5：从机延迟[0~499]
    - Byte6-byte7：连接监督超时[10~3200]，监督超时 =(byte6+(byte7<<8))*1.25ms，监督超时>(1+slaveLatency)*(maxConnectionInterval)
  - len：数据长度，必须是 8 字节
- 返回值：none

22.4.11. void BT_BleUpdateMtu(void)

- 描述：BLE 更新 MTU 数据,仅在连接成功后执行操作
- 参数：none
- 返回值：none

22.4.12. void BT_BleSetMtu(uint16_t mtu)

- 描述：BLE 设置本地 MTU

- 参数：
  - mtu :mtu 范围(23~517)
- 返回值：none

22.4.13. void BT_BleSetPairMode(BLE_PairModeEnum mode)

- 描述：BLE 设置配对模式
- 参数：
  - mode ：
    - PAIR_NONE
    - PAIR_JUST_WORK
    - PAIR_PASSKEY
    - PAIR_SC_JUST_WORK
    - PAIR_SC_PASSKEY
    - PAIR_SC_NUMERIC
- 返回值：none

22.4.14. _Bool BT_BleSlaveSendData(dev_handle_t dhandle,uint16_t handle, uint8_t * bleData, uint16_t len)

- 描述：BLE 从机发送数据
- 参数：
  - dhandle：ble 连接句柄
  - bleData：ble 数据
  - len：数据长度，数据长度小于 253

- 返回值：true/false

22.4.15. _Bool BT_BleMasterSendData(dev_handle_t dhandle,uint16_t handle, uint8_t * bleData, uint16_t len)

- 描述：BLE 主机发送数据
- 参数：
  - handle：ble 连接句柄
  - bleData：ble 数据
  - len：数据长度，数据长度小于 253
- 返回值：true/false

22.4.16. void BT_SetTXPower(char db)

- 描述：设置 tx 功率
- 参数：
  - db : TX 功率
    - -3=-3db,3=3db,5=5db,7=7db,10=10db,其它等于 0db
- 返回值：none

22.4.17. void BT_SetLLParam(uint8_t *p)

- 描述：设置 LL 参数,仅在未连接状态下配置
- 参数：
  - p：LL 参数指针，8 字节特征，5 字节版本
- 返回值：none

22.4.18. void BT_NvramLoad(void)

- 描述：从 Flash 读 NVRAM 配置
- 参数：none
- 返回值：none

22.4.19. uint8_t BT_NvramFindFromList(void)

- 描述：从列表中查找 nvram
- 参数：none
- 返回值：
  - index, if not found, return NVRAM_NUM

22.4.20. void BT_BleNvramSave(void)

- 描述：更新 nvram 数据
- 参数：none
- 返回值：none

22.4.21. uint8_t BT_NvramUpdateToFlash(void)

- 描述：更新 Flash 中 Nvram 数据
- 参数：none
- 返回值：SUCCESS or ERROR

22.4.22. void BT_BleDisconnectIND(uint8_t reason)

- 描述：BLE 断开连接指令
- 参数：
  - reason：断开连接原因
- 返回值：none

22.4.23. int8_t BT_GetRssi(void)

- 描述：获取真实的 Rssi 值,仅在开 ble rx 的状态下使用。
- 参数：none
- 返回值：RSSI 值

22.5. yc_ipc

22.5.1. void IPC_HandleRxPacket(void)

- 描述：BLE 处理接收数据包
- 参数：none
- 返回值：none

22.5.2. void IPC_ParseEvtCallback(uint32_t dataPtr)

- 描述：解析事件回调函数，根据事件类型进行处理
- 参数：
  - dataPtr：接收数据包指针
- 返回值：none

22.6. yc_user

22.6.1. void U_BleSetMac(uint8_t * bleAddr)

- 描述：设置蓝牙地址,通过 flash id 的 crc16 值生成唯一地址
- 参数：
  - bleAddr：蓝牙地址指针
- 返回值：none

22.6.2. void U_BleSetDeviceName(char *name_in,uint8_t *bleAddr)

- 描述：设置蓝牙设备名称,通过将设备名称和蓝牙地址拼接起来,防止设备名称重复
- 参数：
  - name_in：设备名称指针
  - bleAddr：蓝牙地址指针
- 返回值：none

22.6.3. void U_BleGenAdvScanData(uint8_t *adv_data,uint8_t *scan_rsp, uint8_t *name,uint8_t name_len,uint8_t *mac)

- 描述：生成广播数据和扫描响应数据
- 参数：
  - adv_data：广播数据指针
  - scan_rsp：扫描响应数据指针
  - name：设备名称指针
  - name_len：设备名称长度
  - mac :蓝牙地址指针
- 返回值：none

22.6.4. void U_BleParseScanAdvData(uint8_t *p)

- 描述：解析 ble 扫描到的对端广播数据，用于 ble 做主时连接使用
- 参数：
  - p：LL 参数指针

- 返回值：none

22.6.5. int U_smp_passkey_wait_input(dev_handle_t dhandle)

- 描述：待并处理配对时的 Passkey 输入
- 参数：
  - ahandle：ble 连接句柄
- 返回值：
  - 输入状态：
    - -1：当前不处于等待 Passkey 输入状态
    - 0：还没收到完整输入
    - -1：已经收到了有效 Passkey，并继续了配对流程

22.6.6. void U_smp_passkey_wait_input_timeout(dev_handle_t dhandle)

- 描述：passkey 输入超时等待
- 参数：
  - dhandle：ble 连接句柄
- 返回值：none

22.6.7. int U_ATT_GenericAccess_CbRead(dev_handle_t dhandle, uint16_t att_handle, const struct ble_gatt_attr *attr, void *buf, uint16_t len, uint16_t offset)

- 描述：：ATTGenericAccess 处理读属性回调函数
- 参数：

- dhandle：ble 连接句柄
- att_handle：当前被读取的 att 属性句柄
- attr：当前被访问的属性对象
- buf：输出缓冲区
- len：请求读取的最大长度

- 返回值：读取后实际返回的字节数

22.6.8. int U_ATT_User_ServiceCbWrite(dev_handle_t dhandle, uint16_t att_handle, const struct ble_gatt_attr *attr, void *buf, uint16_t len, uint16_t offset)

- 描述：ATT User_Service 处理写回调函数
- 参数：
  - dhandle：ble 连接句柄
  - att_handle：当前被读取的 att 属性句柄
  - attr：当前被访问的属性对象
  - buf：输出缓冲区
  - len：请求读取的最大长度
- 返回值：写数据处理结果

22.6.9. void U_BleMasterParseNotifyData(dev_handle_t dhandle,uint8_t *rxp)

- 描述：解析主机端接收到的 Notify/Indicate 数据
- 参数：
  - dhandle：ble 连接句柄

- rxp：数据包缓存区指针

- 返回值：none

22.7. demo 说明

22.7.1. ble_demo

- 演示 ble 作为从机连接及进入休眠，关键代码说明如下：

上电启动，初始化系统、串口和蓝牙从机，并判断是否为 deepsleep 唤醒，deepsleep 唤醒则打印唤醒原因；若为 lpm 睡眠唤醒，则只初始化串口和打印唤醒原因

```
85 void app_init(void)
86 {
87
88 if((LpmRecoverStruct->wakeup_flag&(WAKEUP_FLAG_WAKE|WAKEUP_FLAG_RAM_CRC_ERR))
==WAKEUP_FLAG_WAKE &&
89  \LpmRecoverStruct->rg_lpm_ldo_dig_lpm_vosel.bit.sleep_mode==SLEEP_MODE_LPM)
90     {
91         //带蓝牙 lpm
92         #ifndef YC_BT_BB_LOG_SWITCH
93         printport_init();
94         #endif
95         yc_log("lpm wake
from %s",LpmRecoverStruct->rg_lpm_ldo_dig_retmem_vsel.bit.wake_mode ==
WAKEUP_FROM_SLEEP ? "sleep" : "main");
96         wakeup_FlagAnalysis();
97     }
98     else
99     {
100        SYSCTRL_ChipInit();
101        SYSTICK_SetClk1M();
102        printport_init();
103        if(LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_WAKE)
104        {
105            yc_log("deep wake
from %s",LpmRecoverStruct->rg_lpm_ldo_dig_retmem_vsel.bit.wake_mode ==
WAKEUP_FROM_SLEEP ? "sleep" : "main");
106            wakeup_FlagAnalysis();
107        }
108        else
```

```
            {
                yc_log("YC16xx ble demo!");
            }
            Ble_SlaveParamInit();
            rtc_cali_start(RTC_CALI_CNT);
        }
}
```

蓝牙从机初始化

```
void Ble_SlaveParamInit(void)
{
    uint16_t version = BT_GetLibVersion();
    yc_log("bt_lib_version:%04x",version);
    init_le();
    //设置 BLE 地址，public or random.
    U_BleSetMac((uint8_t*)ble_addr_public);
    yc_bt_log_buf("BLE_ADDR_Public:", ble_addr_public, sizeof(ble_addr_public));
    //设置 BLE 名称
    U_BleSetDeviceName((char *)ble_name,(uint8_t*)ble_addr_public);

    yc_bt_log("ble_device_name:%s", ble_name);
    //生成 BLE 广播数据和扫描响应包
    U_BleGenAdvScanData(gen_ble_adv_normal,gen_ble_scan_rsp,(char *)ble_name,sizeof(ble_name),(uint8_t*)ble_addr_public);
    yc_bt_log_buf("BLE_adv_normal :", gen_ble_adv_normal, sizeof(gen_ble_adv_normal));
    yc_bt_log_buf("BLE_scan_rsp     :", gen_ble_scan_rsp, sizeof(gen_ble_scan_rsp));
    //使能广播
    BT_BleSetAdvParams(ADV_IND,ADDR_PUBLIC,ADDR_PUBLIC, (uint8_t *)gen_ble_adv_normal, ADV_MAX_LEN);
    //使能扫描响应
    BT_BleSetScanData((uint8_t *)gen_ble_scan_rsp, ADV_MAX_LEN);
    //设置广播间隔
    BT_BleSetAdvInterval(BLE_ADV_INTERVAL);
    yc_bt_log("BLE_ADV_Interval:%d", BLE_ADV_INTERVAL);
    //设置本地 MTU
    BT_BleSetMtu(BLE_LOCAL_MTU);
    //从 Flash 读 NVRAM 配置
    BT_NvramLoad();
    //设置 LL 参数
    BT_SetLLParam((uint8_t*)ll_param);
    //设置 BLE 配对模式
    BT_BleSetPairMode(PAIR_JUST_WORK);
    yc_bt_log("BLE_Pair_Mode:%s", "PAIR_JUST_WORK");
    //设置发射功率
```

```
    BT_SetTXPower(0);
    //设置 BLE 可发现
    BT_SetVisibility(ENABLE);
    //查询蓝牙状态
    //yc_bt_log("Bt state = 0x%x", BT_GetStatus());
}
```

串口发送指令进入 lpm sleep 或 deep sleep 模式，lpm sleep 模式可以维持蓝牙关播或蓝牙连接，支持蓝牙事件（链接请求、断开链接或收到蓝牙数据）唤醒、定时唤醒和 IO 唤醒；deep sleep 模式不可以维持蓝牙关播或蓝牙连接，支持定时唤醒和 IO 唤醒。

```
 void UART_DebugTest()
{
        if(UART_GetRxCount(PRINT_PORT))
        {
            switch(UART_ReceiveData(PRINT_PORT))
            {
                case 'l':
                    yc_log("lpm sleep");
                    lpm_mode = SLEEP_MODE_LPM;
                    break;
                case 's':
                    yc_log("deep sleep");
                    lpm_mode = SLEEP_MODE_DEEP;
                    break;
                default:
                    break;
            }
        }

}

void LPM_Dispatch(void)
{
    if(lpm_mode != SLEEP_MODE_NONE)
    {
        if(lpm_mode == SLEEP_MODE_LPM)
        {

if(LPM_Sleep(0,SLEEP_MODE_LPM,0x11,WAKEUP_FROM_SLEEP,LPM_prepareActionBeforeSleep,LPM_postActionWakeup))
            {
                lpm_mode = SLEEP_MODE_NONE;
```

```
                    yc_log("lpm wakeup");
                }
                else
                {
                    yc_log("try lpm sleep fail");
                }
            }
            else
            {
if(LPM_Sleep(00000,SLEEP_MODE_DEEP,0,WAKEUP_FROM_MAIN,LPM_prepareActionBeforeSleep,LPM_postActionWakeup))
                {
                    lpm_mode = SLEEP_MODE_NONE;
                    yc_log("deep wakeup");
                }
                else
                {
                    yc_log("try deep sleep fail");
                }
            }
        }
}
```

在 while 循环里处理 ble 接收数据包

```
int main(void)
{
    app_init();
    while(1)
    {
        IPC_HandleRxPacket();
        UART_DebugTest();
        LPM_Dispatch();
    }

}

```

22.7.2. ble_master_demo

- 演示 ble 作为主机连接及进入休眠，关键代码说明如下：

上电启动，初始化系统、串口和蓝牙主机，并判断是否为 deepsleep 唤醒，deepsleep 唤醒则打印唤醒原因；若为 lpm 睡眠唤醒，则只初始化串口和打印唤醒原因

```
void app_init(void)
{
    if((LpmRecoverStruct->wakeup_flag&(WAKEUP_FLAG_WAKE|WAKEUP_FLAG_RAM_CRC_ERR))
==WAKEUP_FLAG_WAKE &&
     \ LpmRecoverStruct->rg_lpm_ldo_dig_lpm_vosel.bit.sleep_mode==SLEEP_MODE_LPM)
    {
        //带蓝牙 lpm
        #ifndef YC_BT_BB_LOG_SWITCH
        printport_init();
        #endif
        yc_log("lpm wake
from %s",LpmRecoverStruct->rg_lpm_ldo_dig_retmem_vsel.bit.wake_mode ==
WAKEUP_FROM_SLEEP ? "sleep" : "main");
        wakeup_FlagAnalysis();
    }
    else
    {
        SYSCTRL_ChipInit();
        SYSTICK_SetClk1M();
        printport_init();
        if(LpmRecoverStruct->wakeup_flag&WAKEUP_FLAG_WAKE)
        {
            yc_log("deep wake
from %s",LpmRecoverStruct->rg_lpm_ldo_dig_retmem_vsel.bit.wake_mode ==
WAKEUP_FROM_SLEEP ? "sleep" : "main");
            wakeup_FlagAnalysis();
        }
        else
        {
            yc_log("YC16xx ble demo!");
        }
        Ble_MasterParamInit();
        rtc_cali_start(RTC_CALI_CNT);
    }

}
```

蓝牙主机初始化

```
void Ble_MasterParamInit(void)
{
```

```
    uint16_t version = BT_GetLibVersion();
    yc_log("bt_lib_version:%04x",version);

    init_le();
    ble_user_list_ptr = ble_user_list;
    //设置 BLE 地址，public or random.
    BT_BleSetMac((uint8_t*)ble_addr_public);
    //设置 BLE 名称
    U_BleSetDeviceName((char *)ble_name,(uint8_t*)ble_addr_public);
    yc_bt_log("ble_device_name:%s", ble_name);
    //设置本地 MTU
    BT_BleSetMtu(BLE_LOCAL_MTU);
    //从 Flash 读 NVRAM 配置
    BT_NvramLoad();
    //设置 LL 参数
    BT_SetLLParam((uint8_t*)ll_param);
    //设置 ble master 连接地址
    BT_SetBleConnAddr((uint8_t*)ble_addr_con);
    //设置 BLE 扫描使能
    BT_SetScanEnable(ENABLE,BLE_SCAN_INTERVAL,BLE_SCAN_WINDOW);
    //查询蓝牙状态
    //yc_bt_log("Bt state = 0x%x", BT_GetStatus());

}
```

串口发送指令进入 lpmsleep 或 deepsleep 模式

```
 void UART_DebugTest()
{
        if(UART_GetRxCount(PRINT_PORT))
        {
            switch(UART_ReceiveData(PRINT_PORT))
            {
                case 'l':
                    yc_log("lpm sleep");
                    lpm_mode = SLEEP_MODE_LPM;
                    break;
                case 's':
                    yc_log("deep sleep");
                    lpm_mode = SLEEP_MODE_DEEP;
                    break;
                default:
                    break;
            }
        }
```

```
293
294 }
295
296 void LPM_Dispatch(void)
297 {
298     if(lpm_mode != SLEEP_MODE_NONE)
299     {
300         if(lpm_mode == SLEEP_MODE_LPM)
301         {
302
    if(LPM_Sleep(0,SLEEP_MODE_LPM,0x11,WAKEUP_FROM_SLEEP,LPM_prepareActionBeforeSleep,LPM_postActionWakeup))
303             {
304                 lpm_mode = SLEEP_MODE_NONE;
305                 yc_log("lpm wakeup");
306             }
307             else
308             {
309                 yc_log("try lpm sleep fail");
310             }
311         }
312         else
313         {
314
    if(LPM_Sleep(00000,SLEEP_MODE_DEEP,0,WAKEUP_FROM_MAIN,LPM_prepareActionBeforeSleep,LPM_postActionWakeup))
315             {
316                 lpm_mode = SLEEP_MODE_NONE;
317                 yc_log("deep wakeup");
318             }
319             else
320             {
321                 yc_log("try deep sleep fail");
322             }
323         }
324     }
325 }
```

在 while 循环里处理 ble 接收数据包

```
326 int main(void)
327 {
328     app_init();
329     while(1)
330     {
```

```
331             IPC_HandleRxPacket();
332             UART_DebugTest();
333             LPM_Dispatch();
334         }
335
336     }
337
```